tramdag documentation

Contents

tramdag — Interpretable Neural Causal Models (TRAM-DAGs) in PyTorch

> **Status: 1.0 release candidate.** This README documents the **unreleased 1.0** API (term classes, `callbacks=`, `flow.shift_curve`, `VC(center="col")`); `0.3.0` on PyPI predates all of it. Until 1.0 ships, install from git to follow the docs below.

TRAM-DAGs model each variable of a structural causal model with a (transformation-model) flow: one triangular normalizing flow from iid standard-logistic latents to the observed variables. The triangular adjacency structure is exactly your causal DAG. Fit it **once** on observational data and answer all three rungs of Pearl's causal hierarchy — observational (L1), interventional (L2, the do-operator), and counterfactual (L3, Pearl abduction) — while keeping **interpretable effects**: every linear-shift coefficient is a log-odds ratio, exactly as in classical proportional-odds models.

> Beate Sick & Oliver Dürr, *Interpretable Neural Causal Models with TRAM-DAGs*, CLeaR 2025 (arXiv:2503.16206). This repo is the reference implementation (PyTorch, built on zuko); all of the paper's experiments are replicated here with pinned tests.

5-minute showcase: the Colab badge above fits the paper's bimodal benchmark live and walks L1 → L2 → L3, every answer checked against analytic ground truth. Further notebooks are available at `notebooks/` like the didactic walkthrough of the model: `notebooks/intro_tram_dag.py`.

Install

```
uv add tramdag                  # latest release (PyPI); pip install tramdag works too
uv add "tramdag[plots]"         # with matplotlib, for plot_dag / plot_marginals / plot_training
uv add "tramdag @ git+https://github.com/tensorchiefs/tramdag.git@main"   # dev version (track ma
uv sync                         # or: dev setup from a clone (tests, experiments)
```

Pin the dev install to a commit for reproducibility, e.g. `...tramdag.git@<sha>`.

30 seconds of API

```
from tramdag import CausalFlowDAG, ContinuousNode, OrdinalNode, I, LS, CS

spec = {  # the spec IS the labelled DAG
    "X1": ContinuousNode(),
    "X2": ContinuousNode(I("X1")),
```

```
    "T": OrdinalNode(2, LS("X1") + CS("X2")),  # 2 levels, column coded 0..1
    "Y": OrdinalNode(4, I("X1") + CS("X2") + LS("T")),
}
# train_df / val_df: DataFrames with one column per node
flow = CausalFlowDAG(spec)  # validates acyclicity, builds the flow

# fit() is one minibatch Adam loop with Keras-shaped extras: validation_data=
# (or validation_split=0.1) records the per-epoch validation NLL in
# flow.history["val"], verbose= prints progress; schedules and early stopping
# attach through optimizer= and callbacks= (docs/fitting.md)
from tramdag.callbacks import EarlyStopping

flow.fit(
    train_df,
    epochs=4000,
    batch_size=512,
    validation_data=val_df,
    verbose=100,
    # keep the best-validation weights; stop once the best is 200 epochs old
    callbacks=[EarlyStopping(patience=200)],
)

# all-`ls` model? fit it classically instead: deterministic float64 L-BFGS,
# exact MLE matching statsmodels/R (see docs/fitting.md)
flow.fit_classical(train_df)  # raises on cs/ci/vc specs

flow.log_prob(df)  # L1: joint log-likelihood per row
flow.sample(1000)  # L1: observational sampling
flow.sample(1000, do={"T": 1})  # L2: interventional (graph mutilation)
flow.pmf(df, node="Y", do={"T": 1})  # L2: analytic interventional PMF
flow.density(df, node="X2", grid=grid, do={"X1": 0.5})  # ... and density, continuous nodes

u = flow.abduct(df)  # L3 step 1: latents from observations
cf = flow.sample(do={"T": 1}, u=u)  # L3 steps 2+3: counterfactuals

flow.ls_coefficients()  # interpret: per-edge log-odds-ratios (LS terms)
# per-parent partial effects of an additive complex intercept (centered):
# flow.intercept_contributions(df, "Y") on a CI("A", "B", allow_interaction=False)

# heterogeneous treatment effects: a small, penalized effect head beta(x)*T
# (VC term) with a first-class read-out — see docs/varying-coefficients.md
# e.g. CS("X1", "X2") + VC("X1", t="T") ->
# flow.varying_coef(df, "Y")               # beta(x): deterministic, y-free

flow.scores(df, node="Y")  # per-observation scores dl_i/dtheta
flow.effect_modifier_scan(df, "Y", t="T")  # which VC modifiers? (CUSUM
# scan from a cheap all-ls fit) — docs/scores.md

flow.save("flow.pt")
flow = CausalFlowDAG.load("flow.pt")
```

See the DAG

```
from tramdag import plot_dag
plot_dag(spec)   # or plot_dag(flow) — layers left to right, one edge style per term
```

Continuous nodes are ellipses, ordinal nodes rounded boxes with their level count; LS is thin gray, CS thick blue, a complex intercept dashed orange, a VC treatment edge red with its modifiers dotted. Needs the `plots` extra.

The model in detail: spec → math → networks

Per node, the transformation is additive on the latent (log-odds) scale — one intercept term I plus any number of shifts (notation: `docs/notation.md`):

$u = h_\vartheta(x) + \Sigma\ \beta \cdot x_{pa} + \Sigma\ g(x_{pa}) + (\beta_0 + b_\vartheta(x_{mod})) \cdot x_t$

term	math	what gets built	interpretability
`I()` / bare `I` / omitted	$h_\vartheta(x)$ — constant ϑ	`SimpleIntercept`: one free parameter vector, no network	the baseline transform
`I("A")`	$h_\vartheta(a)(x)$ — ϑ bends with the parent	`ComplexIntercept`: NN `[8, 8]` → `n_params`	the parent reshapes the whole distribution; no single coefficient
`I("A","B")` (default `allow_interaction=True`)	$h_\vartheta(a,b)(x)$	**one joint** NN over both parents — they interact in ϑ	maximal flexibility
`I("A","B", allow_interaction=False)`	$h_\vartheta(a)+\vartheta(b)(x)$	one NN **per parent**, parameter vectors summed in coefficient space	per-parent partial effects via `flow.intercept_contributions`
`LS("A")`	$\beta \cdot a$	`Linear(width, 1)`, no bias — **one parameter per feature column** (one for a continuous parent, `levels` for a one-hot ordinal, identified only as differences `w[k]-w[0]`)	`exp(β)` is an odds ratio
`CS("A")`	$g(a)$, additive	`ComplexShift`: NN `[64, 128, 64]` → `1`	plot `g`
`CS("A","B")`	$g(a,b)$ — joint	one NN over the concatenated features	interaction *in the shift*
`CS("A") + CS("B")`	$g_1(a) + g_2(b)$	two NNs, scalars added	GAM-style, each effect plottable
`VC("A","B", t="T")`	$(\beta_0 + b_\vartheta(a,b)) \cdot x_t$	scalar β_0 + zero-initialised penalized NN `[16]` → `1`; the treatment value multiplies, it never enters the net	$\beta_0 \approx$ constant effect, `flow.varying_coef` reads $\beta(x)$

A node takes **at most one `I` term with parents** — a term list is therefore always purely additive on the latent scale, and interactions exist only *inside* a term. Lists and `+` sums are interchangeable: `[LS("A"), CS("B")]` ≡ `LS("A") + CS("B")`.

Three knobs on the terms:

- **`transform=` on `I`** picks the class of h_ϑ for a continuous node — `"bernstein"` (default, 20 coefficients, tails extrapolate with the boundary slope), `"spline"` (monotone RQ spline, 23 params at `bins=8`, fixed tail slope) or `"affine"` (2 params: the latent is exactly logistic). Ordinal nodes have no transform to pick: their intercept is the cutpoint vector, $P(x \le k) = \sigma(\vartheta_k - \text{shift})$.
- **`input_transform=` on `CI`/`CS`/`VC`** — `"minmax"`, `"standardize"`, or a callable `fn(x, train)` — transforms that term's continuous network inputs with statistics frozen at the first fit; `LS` and the `VC` treatment stay raw, so their coefficients keep their units.

- **`units=` on `I/CS/VC`** sizes the term's network, e.g. `units=[16]` for one hidden layer of 16 neurons. The defaults match the PyTorch reference this package grew out of (buehlpa/TramDag, `tram_models.py`), **not** the TRAM-DAG paper's own R nets — so a replication sets `units=` and `activation=` explicitly, as the configs in `experiments/paper/` do.

Feature widths: a continuous parent enters raw (1 column), an ordinal parent one-hot (`levels` columns). Abduction is exact for continuous nodes and truncated-logistic for ordinal ones, so `flow.sample(u=flow.abduct(df))` reproduces `df` exactly / level-exactly.

There are two ways to fit the model: a stochastic optimizer (`fit`) and the classical route (`fit_classical`). For all-`ls` models — where each node-conditional is a classical transformation model — the classical fit is deterministic and takes seconds (measured ~10 s vs ~200 s for Adam on the CI runner — before the 2026-09 epoch cut made the Adam route ~3.5x faster; CI re-measures on the next push); see `docs/fitting.md`.

Validation

Two layers, deliberately separate.

The framework's own test suite (`tests/`) measures the library against three inline data-generating processes it carries itself, so it needs no research code to run. Its strongest claim is an equality, not a similarity: an all-`ls` outcome node *is* an ordered-logit model, so the flow's MLE must match `statsmodels OrderedModel` on the same design matrix — it does, and the varying-coefficient acceptance bars (effect recovery, propensity centering) are pinned the same way. What the tests guarantee, and how each ground truth is obtained, is documented in `tests/README.md`.

The paper replications (`experiments/`) are separate: one self-contained script per dataset, with its hyperparameters in a sibling YAML file and its expected results committed under each area's `ground_truth/`. A dedicated workflow runs them and compares.

experiment	paper	demonstrates
`triangle.py` (`linear-ls`, `linear-cs`, `atan-cs`, `sin-cs`)	§6.1, C.3	LS coefficient recovery ($\beta = 2, -0.2, +0.3$), CS curve $\equiv -f(x_2)$ for non-monotone f
`triangle_mixed.py` (`linear-ls`, `exp-cs`)	§6.2	mixed data L1/L2 + the C.4 odds-ratio check (OR ≈ 7.4)
`vaca.py`	§5.1–5.2	the bimodal L1 case a default CNF misses; L2 $p(x_3 \mid do(x_2))$ against analytic means
`carefl.py`	§5.3	L3 counterfactual curves vs **analytic** truth
`validate_ls.py`	—	flow ≡ `statsmodels` ≡ R `MASS::polr` to ~4 decimals with `fit_classical` (a converged Adam `fit` gets ~1e-3), on a frozen synthetic cohort with a known true effect

Sign note: ordinal shifts are *subtracted* here but *added* in the paper, so fitted ordinal weights are the paper's with flipped sign (each `truth.json` records both conventions). A figure-by-figure account of what is reproduced, and what is not (the competing CNF/NSF baselines), is in `experiments/paper/PAPER_COVERAGE.md`.

Training speed — schedules, per-node freezing, L-BFGS and device benchmarks: `docs/training-speed.md`.

Paper replication — every hyperparameter of the eight `experiments/paper` variants with its source in the paper's R code, the deviations, and the numbers (paper / previous protocol / now): `docs/paper-replication.md`.

Testing policy

See the `tests/README.md` file for more details.

Layout

```
src/tramdag/            spec.py terms.py transforms.py conditioners.py
                        nodes.py flow.py fitting.py readouts.py scores.py
                        callbacks.py              <- the framework, and nothing else
tests/                  unit tests, identities, acceptance bars, three inline DGPs
experiments/            research code, one directory per area, each self-contained:
                          paper/       the replications + generators + frozen data
                          benchmarks/  training-speed and machine measurements
                          misc/        the classical-MLE validation
                        paper/misc: <name>.py + <name>.yaml, data/,
                        ground_truth/, tests/, results/ (benchmarks reads the
                        other areas' data and pins no ground truth)
notebooks/              four executed examples: didactic intro, Colab demo,
                        additive-vs-joint intercepts, varying coefficients
docs/                   architecture.md + adr/ (the 1.0 design and its
                        refusals), code-map.md, fitting.md, notation.md,
                        training-speed.md, paper-replication.md,
                        varying-coefficients.md, scores.md
```

Implementation conventions (latent-scale signs, raw/one-hot parent encoding, log-space ordinal likelihood, seeding) are documented in `CLAUDE.md` and pinned by tests.

Citation

If you use `tramdag`, please cite the method paper:

```bibtex
@inproceedings{sick2025tramdag,
  title     = {Interpretable Neural Causal Models with TRAM-DAGs},
  author    = {Sick, Beate and D{\"u}rr, Oliver},
  booktitle = {Proceedings of the 4th Conference on Causal Learning and Reasoning (CLeaR)},
  series    = {Proceedings of Machine Learning Research},
  volume    = {275},
  year      = {2025},
}
```

Notation

This page defines the notation. Every guide, docstring and notebook in this repository follows it.

Symbols

Random variables are capital Latin letters, for example Y. Their realizations — observed or sampled values — are lowercase Latin letters, for example y. The cumulative distribution function of Y is F_Y and the density is f_Y. A hat marks an estimate: $\hat{F}$ is the fitted distribution, F the true and often unknown one.

Sets and vectors are bold, for example $\mathbf{X} = [X_1, \dots, X_J]$.

A single optimized parameter is a lowercase Greek letter, for example ϑ or β. A vector of parameters is bold lowercase, for example $\boldsymbol{\vartheta} = (\vartheta_1, \dots, \vartheta_M)^\top$. A tuple that collects several parameter groups — for example all weight matrices of a network — is bold capital, for example $\boldsymbol{\Theta}$.

A function that a parameter vector defines carries it as a subscript: h_ϑ is the monotone transformation with coefficients ϑ.

The model's symbols

Symbol	Meaning	In code
X_j, Y	the variables of the DAG	node names in the spec
$\text{pa}(X_j)$	the causal parents of X_j	`node_parents`
U_j	the standard-logistic latent of node j	`u = flow.abduct(df)`
h_ϑ	the monotone transformation of a node	the `I` term; `theta` in docstrings
ϑ_k	the ordinal cutpoints, elements of ϑ	`ordinal_cutpoints`
β	a linear-shift coefficient; e^β is an odds ratio	`LS`; `flow.ls_coefficients()`
g_j	a complex shift, an NN of one term's parents	`CS`
β_0	the constant treatment effect of a `VC` term	`beta0`
b_Θ	the penalized effect-modification network	the `VC` net; `units=` sets its layers
λ	the L2 weight on b_Θ	`penalty=`
σ	the logistic function	`torch.sigmoid`

Every node's model is one additive formula on the latent scale:

$$u \;=\; h(x \mid \text{pa}(x)) \;=\; h_{\vartheta(\cdot)}(x) \;+\; \sum_j \beta_j\, x_j \;+\; \sum_k g_k(x_k) \;+\; (\beta_0 + b_\Theta(x_{\text{mod}}))\, x_t.$$

For a continuous node the shifts are added, as written. For an ordinal node the shift is subtracted inside the sigmoid: $P(Y \le k \mid \text{pa}) = \sigma(\vartheta_k - \text{shift})$.

Plain-text fallback

Docstrings and terminal output cannot render Greek subscripts. There, `theta` stands for ϑ, `h_theta` for h_ϑ, `b_theta` for b_Θ and `beta0` for β_0.

Fitting a TRAM-DAG: how training works

The technical reference for training a `CausalFlowDAG`: the likelihood, the two fitting paths (`fit`, `fit_classical`) and their hooks.

How the flow is built: one module, one sub-model per node

A `CausalFlowDAG` is a single `torch.nn.Module` holding **one independent sub-model per variable** — an intercept producing the transform parameters `θ`, the monotone 1-D transform `h` (no learnable weights of its own, only range buffers), and one shift module per shift term. The nodes share **no parameters**: one module bundles them and one optimizer trains them, but the DAG structure lives entirely in *which parents each node reads* — there is no edge weight matrix or shared trunk. Which class implements which term is the code map; parent features enter continuous-raw / ordinal-one-hot.

How the likelihood is computed

A TRAM-DAG maps iid standard-logistic latents `U` to the observed `X` in causal order. Node `i` reads only its parents (earlier variables, as *data*). Therefore the Jacobian of `U → X` is **triangular**, and its log-determinant is the sum of the per-node 1-D terms. The joint log-likelihood therefore **decomposes per node**:

```
log p(x) = Σ_i log p(x_i | pa(x_i))
```

`CausalFlowDAG.node_log_prob` computes one term per node and `log_prob` sums them. For a node, given `θ, shift` from `theta_shift`:

- **continuous** — change of variables through the monotone transform:

  ```
  u = h(x; θ) + shift
  log p(x | pa) = log f_logistic(z) + log |dz/dx|
  ```

  That is, the term is the standard-logistic density at the latent `u` (`StandardLogistic.log_prob`) plus the transform's log-derivative. `ut.forward` returns this log-derivative as `ladj`. This is the 1-D Jacobian term that makes the result a proper density, not only a score.

- **ordinal** — an ordered-logit / proportional-odds head, `P(x ≤ k) = σ(θ_k − shift)`. `ordinal_log_prob` evaluates it as the log of the cutpoint-interval probability. The computation runs in log-space via `logsigmoid`/`log1mexp`, because the naive sigmoid difference underflows to exactly-zero gradients in float32.

The training loss is the summed per-node **mean** NLL over the batch (`Σ_i mean_rows(−log p(x_i | pa))`). In contrast, `log_prob` returns the per-row joint, which scores whole observations.

Consequence used by both optimizers: because parents enter as data, the per-node gradients are independent. Therefore a joint fit of the summed loss is identical to a separate fit of each node. This independence licenses per-node learning rates and freezing (a callback, below) and the all-`ls` classical fit.

Path A — stochastic optimization (`fit`)

`CausalFlowDAG.fit` is the general-purpose trainer. Any `cs`/`ci` edge requires it. Mechanics:

- **One optimizer over all parameters** — `Adam(lr=learning_rate)` by default, or any `torch.optim.Optimizer` you pass as `optimizer=` (exactly per-node training, see the consequence above; `per_node_adam` builds the per-node parameter groups when you want per-node rates).
- **Minibatches**: a fresh `torch.randperm` shuffle each epoch (`seed=` seeds it). The loss is the summed per-node mean NLL on the batch, plus the `VC` penalty.
- **`calibrate(train_df)`**, called by the first `fit`: every term freezes its own data-dependent state — the intercept maps the train `range_q`/`1-range_q` quantiles onto its transform's domain, and every term-level `input_transform=` freezes its statistics. Calibration never touches the weights. A checkpoint carries the flag, so a loaded model is never recalibrated. The calibrated start is a separate, always-explicit step: `flow.init_marginals(train_df)` resets every Bernstein/ordinal simple intercept to its column's empirical marginal — `logit(F_hat)` in both cases, as control points or as cutpoints; a pure init that leaves the MLE unchanged (spline, affine and `range_q=0` transforms have none) — any time, including on a trained or loaded flow.
- **Validation, Keras-shaped** — `validation_data=` (a DataFrame) or `validation_split=` (a float: the LAST fraction of `train_df`, no shuffle, and only the head calibrates — no leakage) makes `fit` compute the per-node validation NLL after every epoch, once, into `flow.history["val"]` (`validation_batch_size=` chunks the pass). The shipped callbacks read it there. `flow.history["lr"]` records the optimizer's rate after every epoch (`{node: lr}` with `per_node_adam`), so a schedule's decisions are on record without a callback of your own. `verbose=N` prints every Nth epoch plus the final one (0, the default, is silent).

- **`callbacks=`** — one entry or a list. A `tramdag.callbacks.Callback` hooks `on_fit_begin` / `on_epoch_end` / `on_fit_end` (its docstring is the contract — timing, the stop rule, the VC re-centering order); a bare callable in the list is an `on_epoch_end` hook, `cb(flow, epoch, optimizer)`, and any `True` stops the fit — this is where schedules, snapshots and coefficient trajectories live. The common recipes ship in `tramdag.callbacks`: `EarlyStopping` (best-validation weights restored automatically; optional patience) and `PerNodePlateau` + `per_node_adam` (per-node decay and freezing), all reading `history["val"]`.
- **Centered `VC` propensities are a column**: `VC(center="ps")` names the training-frame column holding the out-of-fold `P(t=1|pa_t)` per row — it splits and minibatches with the frame (see varying-coefficients.md).

Training strategies

Every strategy below is `fit` plus a callback or a few lines of your own — pick by model class, each with a copy-paste example (they assume a built `flow = CausalFlowDAG(spec)` and pandas `train_df`/`val_df`). The empirical rule of thumb: **all-`ls` models train to the MLE and keep the final weights; flexible (CI/CS/VC) models validate and keep the best weights** (they overfit observational confounding at the MLE, see the finding below).

Strategy	When
exact MLE — `fit_classical` (Path B, below)	all-`ls` spec; deterministic, seconds
plain Adam	all-`ls` with a shift `fit_classical` refuses, quick looks
multi-phase Adam	a tighter MLE without a scheduler
best-validation weights — `EarlyStopping()`	any CI/CS/VC model; the recommended recipe (register it — fit has no default)
… + patience — `EarlyStopping(patience=)`	also stop once the best is that many epochs old
global plateau schedule	decaying one shared rate beats picking one
per-node plateau — `PerNodePlateau`	nodes converge at different speeds; self-stopping

Plain Adam — one loop, constant rate, final weights:

```
flow = CausalFlowDAG(spec, seed=0)
flow.fit(train_df, epochs=500, learning_rate=1e-3, batch_size=256, verbose=100)
```

Multi-phase Adam — re-calling `fit` continues training, so decreasing rates are a loop. This is the `validate_ls` protocol, whose three phases land within ~1e-5 of statsmodels — a single converged constant-rate run gets ~1e-3:

```
for epochs, lr in [(800, 1e-2), (700, 1e-3), (500, 1e-4)]:
    flow.fit(train_df, epochs=epochs, learning_rate=lr)
```

Best-validation weights — the flexible-model default, one import, one registration (the restore happens automatically at fit end; without `patience` the fit runs its full budget):

```
from tramdag.callbacks import EarlyStopping

flow.fit(
    train_df,
    epochs=4000,
    validation_data=val_df,   # or validation_split=0.1
    verbose=50,
```

```
    callbacks=EarlyStopping(),
)
```

`validation_split` takes the LAST rows unshuffled — shuffle the DataFrame first if its row order means anything.

… plus patience — the same callback also stops the fit once the best epoch is `patience` old:

```
flow.fit(train_df, epochs=4000, validation_split=0.1,
         callbacks=EarlyStopping(patience=200))
```

Per-node plateau — one rate per node (`per_node_adam` tags one parameter group per node; valid because the per-node gradients are independent), each decaying and finally freezing on its own validation NLL; the fit stops when every node froze. The demo notebook runs it end to end; before 0.4 it was `fit(schedule="plateau", freeze_patience=)`, measured in `experiments/benchmarks/bench_training.py`:

```
from tramdag.callbacks import PerNodePlateau, per_node_adam

flow.fit(train_df, epochs=4000, validation_split=0.1,
         optimizer=per_node_adam(flow, lr=1e-2),
         callbacks=PerNodePlateau())   # patience=15, freeze=50
```

Freezing helps and parallelizing the node loop does not: freezing deletes whole epochs, while node-level overlap only time-slices the cores that each node's batched BLAS ops already saturate — measured as contention, not speedup. Only when per-node kernels under-utilize the hardware (tiny nodes on a big GPU) could overlap pay, and there the tool is fusing same-shaped nodes, not threads.

Global plateau schedule — anything else is a few lines of your own: a learning-rate schedule is torch's, stepped from the hook on the validation NLL `fit` already computed (so this too needs `validation_data=` or `validation_split=`); the snapshot half is what `EarlyStopping` does inside, written out:

```
import copy

import torch

opt = torch.optim.Adam(flow.parameters(), lr=1e-2)
plateau = torch.optim.lr_scheduler.ReduceLROnPlateau(opt, factor=0.3, patience=30)
best = {"nll": float("inf"), "state": None}

def on_epoch(flow, epoch, opt):
    nll = sum(flow.history"val".values())   # fit computed it
    plateau.step(nll)
    if nll < best["nll"]:
        best.update(nll=nll, state=copy.deepcopy(flow.state_dict()))
    return opt.param_groups0 < 1e-5      # stop once the rate bottomed out

flow.fit(train_df, epochs=4000, batch_size=512, validation_data=val_df,
         optimizer=opt, callbacks=on_epoch)
flow.load_state_dict(best["state"])
```

(One difference to `EarlyStopping`: a post-fit `load_state_dict` skips the VC re-centering, so on a spec with a `VC` term put the restore in a `Callback` subclass's `on_fit_end` instead.)

The exact-MLE and warm-start strategies are Path B, below.

Benchmarks and schedule trade-offs are in `training-speed.md`. The worked walkthrough is `notebooks/intro_tram_dag.py`.

Path B — classical optimization (`fit_classical`)

`CausalFlowDAG.fit_classical` is the dedicated optimizer for **all-`ls`** models, where every node-conditional is a classical transformation model (ordered-logit / Colr). It raises on any `cs`/`ci`/`vc` term.

- **Full-batch, float64, L-BFGS** (strong-Wolfe line search). There are no minibatches, no schedule, and no early stopping. Therefore the fit is **deterministic** (same init → bit-identical) and lands on the **exact MLE** — `fit_classical` matches `statsmodels`/R to ~4 decimals; a converged Adam `fit` gets within ~1e-3.
- **Solver budget** (`max_iter=400`, `tol=1e-9`, `history_size=50`): one L-BFGS run with torch's own stopping rule — it ends when the NLL or the parameters move by less than `tol`, or at `max_iter`. The report's `n_iter` is torch's count and `converged` says whether a tolerance, not the cap, ended the run. `history_size` is the L-BFGS memory.
- **float64 is a transient compute mode**: the fit runs in double and restores float32 afterwards; checkpoints stay float32.
- **Convergence**: the flag is true when torch's `tolerance_change` ended the run before `max_iter` did, and it is *advisory*. A Bernstein intercept and weakly-identified directions (rare one-hot levels, a flat treatment-effect ridge) continue to drift along zero-curvature valleys after the likelihood is at the optimum. Correctness comes from a comparison with classical software (`python -m misc.validate_ls classical`), not from the flag.
- Read the fitted coefficients with `ls_coefficients()`.

Warm-start handoff: classical fit, then keep training

`fit_classical` leaves the model at the MLE in float32, ready for any normal operation — and continuing with `fit()` from there **stays put**, which is both a check that the classical solution really is the optimum and a way to use it as a fast, principled initialization:

```
flow.fit_classical(train_df)                          # exact MLE, seconds
before = flow.ls_coefficients()["y"]
flow.fit(train_df, epochs=300, learning_rate=1e-3)  # a gentle Adam phase ...
after = flow.ls_coefficients()["y"]                   # ... barely moves
```

A small drift means the classical fit was already at the optimum. The same handoff warm-starts a `VC` term's `beta0` — the measured recipe is in varying-coefficients.md.

Memory and disk during fitting

Neither fitting path writes to disk: parameters, optimizer state and the `history` dict live in RAM, and whatever a callback records is yours. The only disk I/O in the module is the explicit `save()`/`load()`; the `results/` artifacts in this repo come from the experiment scripts, not the library.

Optimizer choice

Today: **Adam** for flexible models, **L-BFGS** (float64) for all-`ls`. The per-node decomposition makes optimizer swaps cheap through `optimizer=`; candidates (IRLS for the `ls` path, per-node mixing, modern Adam variants) are benchmarked with `experiments/benchmarks/bench_training.py` before adoption.

How fast can a CausalFlowDAG train?

This document benchmarks learning-rate schedules, per-node freezing, batch sizes, devices, and LBFGS. The benchmark ran in June 2026 on an Apple-silicon Mac mini with torch 2.12 (CPU unless noted). To reproduce it, run `experiments/benchmarks/bench_training.py` (`cd experiments`

`&& uv run python -m benchmarks.bench_training`, or `--quick` for one seed on cpu); the full grid takes ≈ 35 min. For a quick **cross-machine** comparison, use the self-contained `experiments/benchmarks/perf_machine.py`. It runs fixed 200-epoch workloads on all available devices and writes a machine fingerprint to JSON, and it needs nothing but `pip install tramdag`. The raw CSV is a local run artifact and is not committed.

The recipes, and where they live now

The recipes this benchmark compares are **callbacks** since 0.4 — training strategies, not part of the model (see fitting.md):

recipe	behavior	today
constant	one lr for the whole run	`flow.fit(train, epochs=, learning_rate=)`
plateau+freeze	**per-node**: a node whose own validation NLL hasn't improved by `min_delta` (1e-4 default; the stroke run uses 1e-5, see the benchmark) for `patience` epochs gets its lr × 0.3, floored at 1e-3 × the start; once decayed ≥ 100× and flat for `freeze` epochs it leaves training (rate 0). Valid because the per-node losses have independent gradients.	`tramdag.callbacks.PerNodePlateau`, a callback over one parameter group per node (`per_node_adam`), measured in `experiments/benchmarks/bench_training.py`
global plateau	torch's `ReduceLROnPlateau` on the summed validation NLL — the paper reference's rule	`experiments/paper/helpers.py::fit_paper`

`"onecycle"` and `"cosine"` lost to `"plateau"` on every workload and were removed in 0.4; their measured rows below stay as the record.

The exact-MLE path: for an exact comparison with classical methods (`statsmodels`, R `polr`/`tram`) no recipe is needed —

```
flow.fit(train_df, epochs=4000, learning_rate=1e-2, batch_size=512)  # constant lr
flow.fit(train_df, epochs=2000, learning_rate=1e-3, batch_size=512)  # 2nd phase
```

is still the exact-MLE path (`experiments/misc/validate_ls.py` runs a three-phase variant of it, 800/700/500 epochs at 1e-2/1e-3/1e-4, batch 256 — cut from 4000/2000/1000 in 2026-09: the same MLE, named-coef gap to statsmodels 1.6e-5 vs 1.8e-5, ~3.5x less wall clock). `fit` keeps the final weights. The guard test `tests/test_fit_hooks.py::test_torch_plateau_scheduler_preserves_exact_m` shows that a schedule through the hooks still lands the all-`ls` fit on the classical MLE within the usual tolerances.

LBFGS ships as its own method, `fit_classical`, which supersedes the hand-rolled recipe this report benchmarked. The float32 variant measured here was fast (< 2 s) but seed-fragile (Finding #2); `fit_classical`'s float64 upcast fixed that.

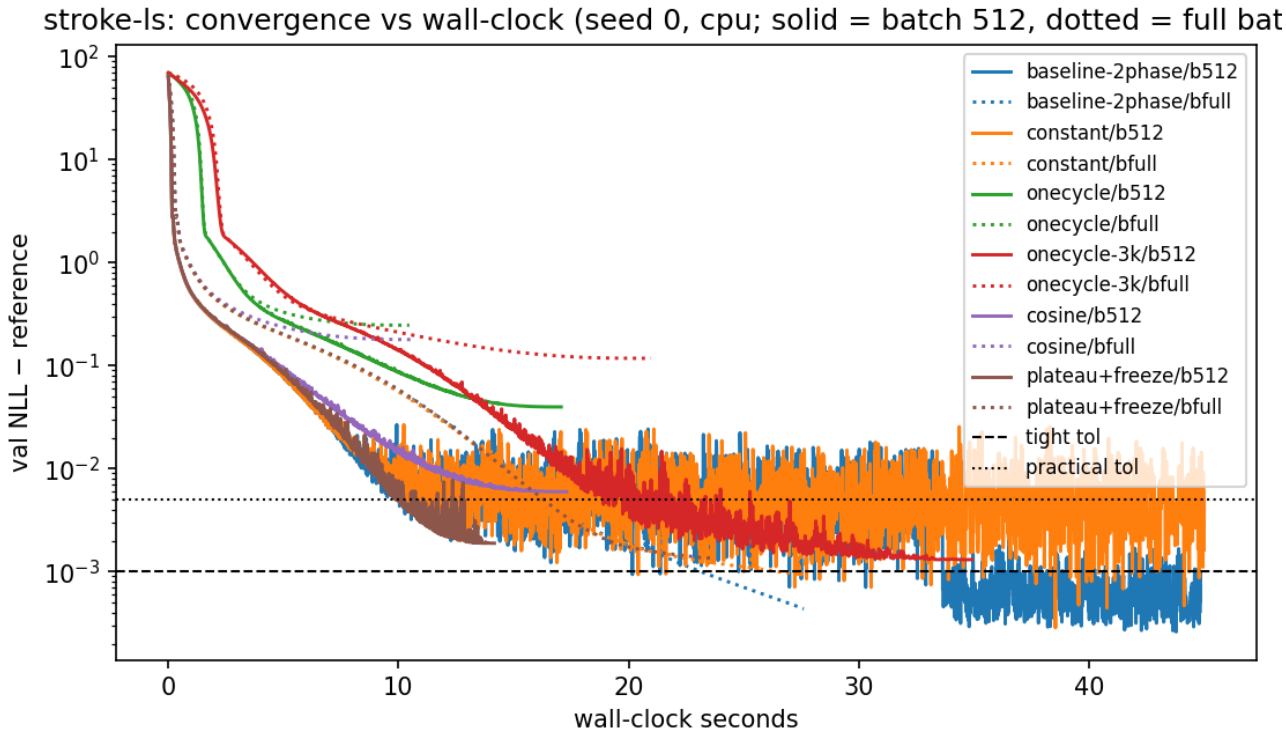
Method: time-to-target, not loss-go-down

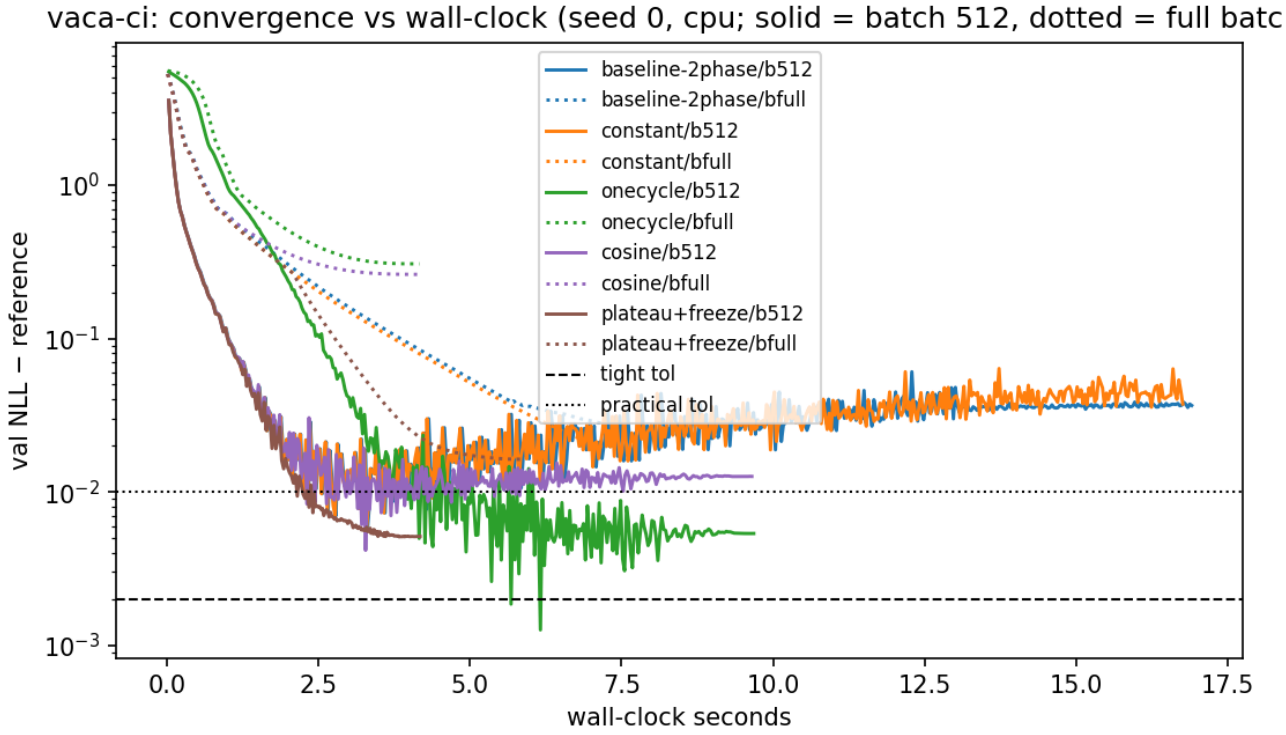
Each config runs once. The benchmark's callback records per-epoch validation NLL *and* wall-clock time. From these records, we measure the seconds until the fit is within a fixed gap of a cached long-run reference (3 torch seeds, medians):

workload	model / data	reference NLL	tight tol	practical tol
stroke-ls	all-`ls` 5-node DAG, frozen `experiments/misc/data/magic-mrclean/ls` (n=1275, full-data MLE)	10.3042 (train)	+1e-3	+5e-3
vaca-ci	all-`ci` flow, frozen `experiments/paper/data/vaca` (n=5000, 90/10 split)	4.9632 (val)	+2e-3	+1e-2

Tight ≈ exact-MLE equivalence (statsmodels/R-polr match). *Practical* ≈ coefficient-equivalent: a fit with gap ≈ 3e-3 already matches the R reference coefficients within the tolerances of `experiments/misc/validate_ls.py`. The workloads are unchanged since the measurement (same frozen data, same specs); re-running reproduces the machine-independent stroke-ls reference NLL 10.3042 exactly.

Results





The table shows median seconds to target (batch 512, cpu). A “—” entry means that the fit never reached the target within the budget.

config	stroke-ls practical	stroke-ls tight	vaca-ci practical	vaca-ci tight	self-stops
baseline two-phase (old default)	9.0	**21.4**	2.1	2.8[1]	no (runs 40 s / 15 s)
constant 1e-2	9.1	21.5[3]	2.2	2.8[1]	no
onecycle (1500 / 300 ep)[2]	—	—	3.5	4.5	no
onecycle (3000 ep)[2]	16.8	— (gap 1–2e-3)			no
cosine[2]	—	—	2.2	3.5[1]	no
plateau + freeze	**8.9**	— (gap 2e-3)	**2.0**	2.9	**yes — 13 s / 4 s total**
LBFGS (full-batch)	**1.6** (2/3 seeds)	— (gap 4–8e-3)	n/a	n/a	yes

[1] transient: the val-NLL curve dips through the target and then drifts away. Stroke needs the 1e-3 phase to *stay*. Vaca shows mild overfitting. Final gap for the vaca baseline is 0.037. The old 520-epoch budget **underfits** vaca by ~0.03 nats. Plateau+freeze *stays* at its target. [2] `onecycle` and `cosine` were removed from `fit()` in 0.4 — they lost to plateau on every workload here — so these three rows cannot be re-measured. [3] constant lr at batch 512 stalls at gap 3–7e-3. Only the lr-decay phase closes the last decade. This is why the two-phase recipe existed.

Findings

1. **Per-node plateau decay + freezing is the best default-style trainer.** It has the same time-to-accuracy as the hand-tuned two-phase schedule, but it needs **no budget tuning**. It decays the lr of each node off its own validation curve. It freezes converged nodes, which is a real FLOP saving because the per-node NLLs have independent gradients. And it **stops itself**: 13 s total vs 40 s for the baseline on stroke-ls, 4 s vs 15 s on vaca-ci, at equal or better final NLL.
2. **LBFGS is spectacular but not robust.** Full-batch LBFGS reaches coefficient-level accuracy on the classical all-`ls` model in **< 2 s** (vs 9 s for Adam) on 2/3 seeds. The third seed stalls at gap 8e-3. An Adam warm start made it *worse* (different basin) on every seed. Use it as a fast first shot with the plateau trainer as fallback, not as the default.
3. **OneCycle is a "spend exactly this budget" scheduler.** Accuracy arrives only at the end of its anneal. At 1500 epochs it misses everything. At 3000 epochs it lands gap 1–2e-3. But you must know the right budget in advance, and this requirement is the problem that we try to remove.
4. **Full-batch loses on time-to-target** despite ~1.6× higher epoch throughput. It makes too few optimizer steps per second of compute at these n. Batch 512 is a good default. Very large batches (16k) only improved raw throughput at n=50k.
5. **MPS (Apple GPU) is 3–4× slower than the M-series CPU** at these model sizes (verified correct: identical reconstruction). Kernel-launch overhead dominates sub-millisecond ops. Stay on CPU locally. CUDA on Colab-class GPUs is a different regime.
6. **The old defaults waste or under-spend.** Stroke: 4000 epochs budgeted, converged work done after ~1500 (freezing recovers the difference automatically). Vaca: 520 epochs budgeted, ~0.03 nats short of converged. Fixed budgets are wrong in both directions. Adaptive stopping fixes both.

Recommendation

The everyday recipe is the global-plateau callback in fitting.md with a generous `epochs` ceiling; the per-node self-stopping variant is `tramdag.callbacks.PerNodePlateau`. One finding became a package default: `epochs` has no default, because Finding 6 is precisely that a fixed budget cannot be right for every workload — every in-repo caller states its recipe in its own YAML.

Per-observation scores & the effect-modifier scan

`flow.scores(df, node)` returns the score $\psi_i = \partial \ell_i / \partial \theta$ of each observation for the node's interpretable shift coefficients. These coefficients are every `LS` weight and every `VC` term's `beta0`. An `LS` weight gives one column per continuous parent and one column per one-hot level of an ordinal parent. The `beta0` column is named after the treatment.

The computation is **analytic and exact**, with no autograd. Shifts enter the latent additively. Thus $\partial \ell_i / \partial \beta = (\partial \ell_i / \partial s_i) \cdot x_i$, with $\partial \ell_i / \partial s$ in closed form (`src/tramdag/scores.py`). The function is a pure read-out. It does not touch the fitting or sampling code paths. At a fitted MLE, the sum of each column is ≈ 0. Tests pin this behavior and include a float64 finite-difference check.

Worked example: where does β(x) need to bend?

The score is the cheapest effect-modifier detector available. It uses the structural-change logic of model-based recursive partitioning (Zeileis & Hornik; Dandl et al. 2024). Before you fit an expensive model, fit the **cheap all-`LS` model**. This fit takes seconds and is deterministic. Then scan the treatment-coefficient scores:

```
flow = td.CausalFlowDAG(all_ls_spec, seed=0)
flow.fit_classical(df)  # seconds, exact MLE

scan = flow.effect_modifier_scan(df, node="Y", t="T")
#         stat   p_value  crit_5pct   flag
# X2      4.21    <0.001     1.3581   True     <- true modifier
# X3      3.05    <0.001     1.3581   True     <- true modifier
# X1      0.74     0.64      1.3581  False     <- prognostic only
```

For each candidate covariate, the scan orders the scores by that covariate and forms the scaled cumulative sum `B_j = Σ_{i≤j} ψ_(i) / (sd(ψ)·√n)`. Under a stable coefficient, `B` is a Brownian bridge. Thus `sup|B|` has the Kolmogorov distribution (5% critical value 1.358). A coefficient that truly varies with the covariate makes the ordered scores drift. The flagged covariates are the measured shortlist of `VC` modifiers (`docs/varying-coefficients.md`):

```
spec["Y"] = td.ContinuousNode(
    td.CS("X1", "X2", "X3") + td.VC("X2", "X3", t="T")
)  # scan-informed
```

Read it as screening, not as a test of effect modification. The statistic measures how unstable the *cheap* model is along a covariate, and instability has two sources: a coefficient that genuinely varies, and a **prognostic part the cheap model gets wrong**. In the example above `X1` is unflagged because its prognostic effect is linear; make it quadratic and `X1` flags as strongly as the true modifiers (0.56 → 4.14 on the DGP of `notebooks/varying_coefficients.py`, which demonstrates both cases side by side). A flag therefore means "look here", and what you find may be a modifier or a misspecification — both worth knowing.

Notes: For a binary ordinal `LS` treatment, `t` resolves to the identified level-1-vs-0 contrast. For a `VC` treatment, `t` resolves to `beta0`. Candidates default to every column of `df` except the node and `t`. Modifiers do not need to be parents. For few-level (heavily tied) candidates, the ordering is only partial. Then read the scan as a ranking diagnostic, not as an exact-size test. You can also use the scores later for influence-function analyses and robust standard errors.

Varying-coefficient treatment effects: `VC(*modifiers, t=, penalty=)`

A `VC` term gives a node a **treatment-effect head with its own bias-variance budget** (issue #28). The term contributes

```
beta(modifiers) * x_on        with    beta(x) = beta0 + b_theta(x)
```

to the node's shift. Here `b_theta` is a deliberately small (one 16-unit hidden layer), **penalized** network. The point is not expressiveness. The point is that the flow estimates the effect function *with care*, not as a by-product:

```python
import tramdag as td

spec = {
    "X1": td.ContinuousNode(),
    "X2": td.ContinuousNode(),
    "X3": td.ContinuousNode(),
    "T": td.OrdinalNode(2, td.LS("X1") + td.LS("X2")),
    "Y": td.ContinuousNode(
        td.CS("X1", "X2", "X3")  # prognostic part g(x): as flexible as you like
        + td.VC("X2", "X3", t="T", penalty=1.0)  # effect beta(X2, X3) * T
    ),
}
flow = td.CausalFlowDAG(spec, seed=0).fit(
    train, epochs=500, learning_rate=1e-2, batch_size=512
)  # best-validation weights: callbacks.EarlyStopping, see fitting.md

beta = flow.varying_coef(df_new, "Y")  # (n,) array beta(x) — deterministic, y-free
beta0 = float(flow.nodes["Y"].shifts["T"].beta0.detach())  # interpretable main effect
```

`X2` and `X3` appear **twice**: as prognostic parents through `CS` and as effect modifiers through `VC`. This pattern is intended. Only the treatment node named by `t=` owns its edge, and a second term that declares it raises an error. Modifiers can repeat.

Why not `CS("T", "X2", "X3")`? (anti-pattern)

For a binary treatment, the multi-parent `CS` is equivalent in *expressiveness*. Any shift decomposes exactly as `s(x,t) = s(x,0) + [s(x,1) − s(x,0)]·t`. But the `CS` form has **no effect-specific regularization**. The likelihood rewards a good average fit of `s(x,t)`, and nothing rewards a smooth difference between the arms. The read-out is thus the difference of two jointly fitted, unregularized networks, which amplifies noise.

On the heterogeneous-effect validation DGP (see below), the `CS` reduced form reaches corr ≈ 0.5 against the true effect function (tramdag-simu#18 / PR #21). This holds **even when the model is exactly in-class**. The `VC` term reaches corr ≈ 0.99 on the same protocol (`tests/test_vc_term.py`, acceptance bar 0.9). Causal forests and R-learners work not because they *target* the effect, but because they **regularize** it (Nie & Wager 2021, Athey–Tibshirani–Wager 2019). `VC` brings that ingredient into the TRAM framework.

Semantics

- **Scale**: `beta(x)` lives on the node's latent (log-odds) scale. The flow adds it for a continuous node (`u = h(y) + …`) and subtracts it from the cutpoints for an ordinal node, exactly like an `LS` weight. With no modifiers, `VC(t="T")` *is* `LS("T")` (identical model, testably bit-exact). Thus `VC` versus `LS` is a nested question.

- **Penalty**: the fitting objective is the penalized likelihood $\sum_i \mathrm{NLL}_i + \lambda \|b_\Theta\|^2$ (`penalty=` is λ) on the total-NLL scale. This is a fixed Gaussian prior whose shrinkage vanishes as n grows. The penalty never applies to `beta0`. `penalty → ∞` shrinks `b_theta` to the zero function and recovers the classical `LS` fit. The default is `penalty=1.0`. If the modifiers are many or n is small, raise the penalty.
- **Identification / centering**: a constant moves freely between `beta0` and `b_theta`. The head's output layer is zero-initialized (`beta(x) = beta0` at step 0). After `fit`, the flow re-centers the head to mean zero over the training data, which preserves the function. Thus `beta0` is the main effect in the training population, the `Colr` reading when `beta` is constant.
- **Warm start** (optional, two lines): fit the all-`ls` version of the node classically and copy the treatment weight into `flow.nodes[node].shifts[t].beta0` before `fit`, so training starts at the classical answer and only learns deviations. Measured on the `vc_hetero` DGP: `beta0` lands within 0.15 of the truth with it, 0.16 from the zero start; the recovery correlation is 0.99 either way.
- **Treatments**: the treatment `x_on` can be continuous or binary (2-level) ordinal. The term is linear in `x_on`. A binary ordinal enters as its 0/1 level, so `beta` is the identified level-1-vs-0 contrast. Multi-level ordinal treatments are a planned follow-up.
- **Read-out**: `flow.varying_coef(df, node, t=...)` evaluates `beta0 + b_theta(modifiers)` in closed form. The result is deterministic and y-free, and it needs no abduction. For a binary treatment, it equals the abduct-difference `u(x, t=1, y) − u(x, t=0, y)` identically (pinned by a test).

Propensity-centered VC: `center="col"` (R-learner orthogonalization)

```
td.VC("X2", "X3", t="T", penalty=1.0, center="ps")
# contributes  beta(x) * (t - e_hat(x))  to the shift; e_hat comes from you,
# out of fold, as the training-frame column named by center=
```

This is Robinson/R-learner centering inside the likelihood. Dandl et al. (2024) found treatment-centering to be *the* decisive ingredient for effect estimation under confounding in model-based forests. That finding reproduces here. The test DGP is strongly confounded, and the model deliberately under-specifies its prognostic part (true g(x) quadratic, model linear). On that DGP, the uncentered β absorbs the confounded misfit. The mean $|\beta - \tau|$ is ≈ 1.1–1.2, which effectively destroys the effect estimate. The centered β stays near truth (≈ 0.1–0.3), a **5–10× bias reduction** (`tests/test_vc_centered.py`). If the prognostic part is correctly specified, centering changes little. Centering is insurance against the misspecification that you do not know you have.

The naive implementations are wrong. Thus this is a **two-stage frozen** design:

- **Training** uses **out-of-fold** ê that *you* compute and pass as the frame column `VC(center=)` names (`df.assign(ps=e_oof)`), one value per training row: any propensity model, each fold predicted by a fit that never saw it. This is the DML cross-fitting requirement — in-sample ê reintroduces the own-observation bias and can be *worse* than no centering. Six lines with the flow's own classical fit:

  ```
  fold_id = np.random.default_rng(0).permutation(len(train)) % 5
  e_oof = np.empty(len(train))
  for j in range(5):
      proxy = td.CausalFlowDAG(t_spec, seed=0)          # the treatment node's spec
      proxy.fit_classical(train.ilocfold_id != j])
      e_oof[fold_id == j] = proxy.pmf(train.iloc[fold_id == j], "T")[:, 1]
  ```

  The OOF values enter the outcome loss as frozen data. Thus **no gradient reaches the treatment node** from the outcome node, and the per-node factorization stays intact (pinned by a gradient-isolation test). `fit` refuses a centered spec whose frame lacks the column, and one that does not match the spec.

- **Inference** (`log_prob` / `sample` / `abduct` / `pmf` / `scores`) recomputes ê from the flow's **own fitted**

treatment node, the full-data fit (the standard DML train/predict split). The computation is detached and uses the current parent values. Under `do(T=t)`, the flow re-derives the regressor as `t − ê(x)`. The flow caches nothing.

- **Interpretation**: with centering, `beta0` is the effect at the treatment margin (the observed propensities). `varying_coef` is unchanged, because centering moves the regressor, not β. The LS-nesting reading applies to the uncentered term only. Centering requires a binary ordinal treatment. Continuous-treatment centering with E[T|x] is a follow-up. `center=False` (the default) is bit-identical to the uncentered term.

Validation

The `vc_hetero` DGP in `tests/conftest.py` is a logistic-shift SCM with known `beta_true(x) = −1 + 0.8·X2 − 0.6·X3`. It has a nonlinear prognostic part and confounded assignment (X2 is confounder *and* modifier), which is the configuration where the `CS` reduced form fails hardest. Acceptance (`tests/test_vc_term.py`) requires three results: recovery corr ≥ 0.9 at n = 5000 (measured ≈ 0.99, min over 3 seeds 0.986), a fitted `beta0` that matches `fit_classical` under a large penalty, and the read-out identities. The centering claims are measured against the `confounded` DGP in the same file. Both former follow-ups have since shipped: propensity centering (#30) is `center="col"`, documented above, and the per-observation scores for effect-modifier scans (#29) are `flow.scores` and `flow.effect_modifier_scan` — see scores.md.

Paper replication — protocol, hyperparameters and results, per experiment

The eight variants under `experiments/paper/` replicate the TRAM-DAG paper (Sick & Dürr, CLeaR 2025, arXiv:2503.16206) against its own R code (`tensorchiefs/tram-dag`). This page lists, for each experiment, the DGP, the model, every hyperparameter with its source, what deviates from the paper and why, and the numbers: what the paper reports, what the previous protocol of this repository measured, and what the current 1:1 protocol measures.

Measured 2026-08-26 on `refactor/lean-fit` (seeds: DGP 42, init 7, shuffle 0) with the exact reference protocol — global plateau rule and Keras init. On 2026-09-01 the *optimization* was re-tuned for CI runtime (fewer epochs, lr scaled to match; data, model and init untouched); on **2026-09-02 VACA was restored to the reference protocol 1:1** (10000 @ 0.001 + plateau — every bound holds, do(x2=0) improves) after a visual pass against paper Fig. 5; on **2026-09-03 CAREFL became the reference run 1:1** — trained on CAREFL's own committed 2500 rows (`data/carefl-cf`, sd-standardized units) with `val = train`, 7000 @ 0.001 + plateau, and the min-max Bernstein domain (`range_q: 0`), which put the Fig. 6 curves on the paper's (the earlier 3000 @ 0.002 trade-off was an artifact of the fresh-draw data; the triangle cuts stay). The tuning campaign, with what was tried and what failed, is in the 2026-09-01 tuning round below. The 2026-08-25 numbers of `feat/followups` (per-node plateau, torch init; CI run 32878435001) are kept in the "R 1:1, torch init" column where they differ. "Previous" is the protocol before that branch: a 90/10 split of one draw, batch 512, the fit restarted in chunks (`fit_in_chunks`), and for VACA/CAREFL a second "polish" fit at a lower rate — a repository recipe, not the paper's. Its numbers come from the ground-truth files of that revision (`73f9f39`; `{max}` bounds were 2.5× the measurement).

The paper states four training numbers — n = 40000, 500 epochs, Adam, Bernstein order 20 — and shows results as figures. Where it gives no number, the "paper" column names the figure and what it shows.

What is common to all eight

item	reference	here
latent	standard logistic, shifts added on the continuous scale, subtracted for ordinal `P(Y<=k) = sigmoid(theta_k - shift)`	same (tests pin both signs)
Bernstein basis	`len_theta` unconstrained coefficients, `to_theta` softplus-cumsum, domain → [0, 1] with tangent-linear extrapolation outside; the domain comes from the train **5 %/95 % quantiles in the triangle scripts** (`quantile(..., c(0.05, 0.95))`, the min/max lines commented out) and from **min/max** in the comparison scripts (`scale_df`)	zuko Bernstein, `n_coeffs` unconstrained (zuko ties two control points on, so the free-parameter count matches), domain = train `range_q`/1−`range_q` quantiles → [−5, 5], linear extrapolation; `range_q` is an intercept option since 2026-09-03, default 0.05. Triangle: a match up to the reparametrization [0,1] vs [−5,5], the tail rule and order 21 vs 19. CAREFL: `range_q: 0` = the reference's min/max, a match. VACA: quantiles **kept deliberately — deviation D1**, measured (see VACA)
init	triangle scripts: `LinearMasked` layers with Keras `random_normal` (N(0, 0.05^2)) on weights and biases, the LS `beta` layer included; comparison scripts: `layer_dense` default, glorot-uniform weights and zero biases	`init: normal` (triangle) and `init: glorot` (VACA/CAREFL), `CausalFlowDAG(init=)`; torch's default init remains the framework default and was the decisive deviation under the full-batch protocol, see VACA
optimizer	Keras Adam, eps 1e-7	torch Adam, eps 1e-8 — measured: no effect (VACA identical to four digits)
seeds	triangle scripts unseeded (`SEED = -1`); comparison scripts `dgp(..., seed=42)` on R's RNG	not replayable in torch, so every seed is a repo choice (42 for the DGP is kept as a nod to the reference)
calibrated start	none	none — `calibrate` never touches the weights; `init_marginals` is a separate explicit step no paper script calls
intercept output layer	Keras dense with bias	bias-free — **D3** (the same function class; the bias adds a constant to all unconstrained coefficients)
plateau rule (VACA/CAREFL)	`update_learning_rate`: one optimizer, reduce when the summed validation NLL has not improved for 50 epochs (strict <), factor 0.1, min 1e-7	torch `ReduceLROnPlateau(patience=49, threshold=0, threshold_mode="abs", factor=0.1, min_lr=1e-7)` on the summed `history["val"]` entry (fit computes it) — the same rule, verified against torch's source; both keep it since 2026-09-02 (VACA's 2026-09-01 drop went with the reverted epoch cut)

D1 (VACA only since 2026-09-03) and D3 remain; both were measured (below). The per-node plateau approximation of 2026-08-25 (D4) is gone with the lean `fit`. CAREFL's two former repo choices — a fresh 2500-row draw with a separate validation draw, in raw units — are gone the same day: the benchmark now trains on CAREFL's own committed rows with `val = train`, in the reference's sd-standardized units, exactly as `carefl_fig5.r` does.

Triangle, continuous (`triangle.py`) — paper Sec. 6.1, App. C.3

DGP (`summerof24/triangle_structured_continous.R`): x1 ~ 0.5 N(0.25, 0.1^2) + 0.5 N(0.73, 0.05^2); h(x2 | x1) = 5 x2 + 2 x1 = u2; h(x3 | x1, x2) = 0.63 x3 − 0.2 x1 − f(x2) = u3, u ~ logistic. f: `linear` −0.3 x, `atan` 0.75 atan(5 (x + 0.12)), `sin` 2 sin(3x) + x. True weights in the flow's convention: β12 = +2, β13 = −0.2, β23 = +0.3 (linear only).

Model: `x1: SI`, `x2: SI + LS(x1)`, `x3: SI + LS(x1) + {LS(x2) | CS(x2)}`, Bernstein. CS net = reference `hidden_features_CS = c(2, 25, 25, 2)`, sigmoid (the ReLU line in `create_param_net` is commented out). **Corrected 2026-09-02**: the vector reads as in/out dims around the hidden stack **(25, 25)** — the earlier literal `[2, 25, 25, 2]` reading put a 2-sigmoid bottleneck on the input and could not reproduce paper Fig. 18 (sin) at any protocol; (25, 25) lands on Figs. 17/18 (linear cs err 0.13 → 0.025, sin 1.22 → 0.24 on the paper's [−1, 0] window). Mixed likewise: c(2, 2, 2, 2) → hidden (2, 2).

hyperparameter	paper / R code	previous	now
train / validation	`dgp(40000)` / `dgp(40000)`, two draws	36000 / 4000, one draw split	40000 / 40000, two draws
epochs	500	500 (in chunks of 10, Adam restarted each chunk)	**300** (`linear-cs`: 500), one continuous run — the CI deviation, floors below
lr	0.001 (`optimizer_adam()` default)	0.001	**0.004**, the CI deviation
batch	32 (Keras `fit()` default)	512	**256**, the CI deviation
`len_theta` / `n_coeffs`	20	20	20
schedule / early stop / init	none / none, final weights / random_normal	none / none / torch	none / none / `init: normal`
coefficient read-out	after every epoch (Keras loop)	at chunk boundaries	`fit(callbacks=)`, every epoch

Results

variant	metric	paper	previous	paper protocol, `init: normal` (batch 32 / lr 0.001 / 500 epochs)	CI config (batch 256 / lr 0.004, 300 epochs — `linear-cs` 500), the pinned ground truth
linear-ls	β12 / β13 / β23	Fig. 14 trajectories; App. C.3 text: 1.98 / −0.21 / 0.26	1.983 / −0.145 / 0.285	1.987 / −0.170 / 0.282	1.981 / −0.161 / 0.281
linear-cs	β13; max $\lvert\hat{g} - (-f)\rvert$ on [−1, 1]	Fig. 17: fitted CS is a straight line	−0.151; 0.507	−0.173; 0.122 (glorot init: 0.110, torch: 0.116)	−0.178; 0.088

variant	metric	paper	previous	paper protocol, `init: normal` (batch 32 / lr 0.001 / 500 epochs)	CI config (batch 256 / lr 0.004, 300 epochs — `linear-cs` 500), the pinned ground truth
atan-cs	β13; cs max err	Fig. 7 right / 15 / 16: CS on −f; text: β12 = 2.07, β13 = −0.203	−0.149; 0.229	−0.168; 0.059 (glorot: 0.080, torch: 0.085)	−0.168; 0.108 (hidden (25,25), 2026-09-02)
sin-cs	β13; cs max err	Fig. 18: CS follows the non-monotone f on x2 ∈ [−1, 0]	−0.185; 2.29	−0.195; 1.10 (glorot: 0.997, torch: 1.016)	−0.168; 0.240 on the paper's [−1, 0] window (hidden (25,25), 2026-09-02)
all	|E[x3 | do(x1 = −1)] flow − DGP|	Figs. 16/17: histograms overlap	—	0.09–0.14	0.08–0.11
all	val NLL x3	—	2.4603–2.4731	2.4607–2.4764	2.4606–2.4764

β13 at −0.16 to −0.18 instead of −0.2: x1 has sd 0.254, so SE(β13) ≈ 0.036 at n = 40000 — within ~1 SE. The old `sin-cs` error of ~1.0 was the misread 2-unit-bottleneck net, not "reference capacity" (retracted 2026-09-02): with hidden (25, 25) the fit lands on Fig. 18 including its small −1-endpoint deviation, and the paper's figure never plots beyond [−1, 0].

Triangle, mixed (`triangle_mixed.py`) — paper Sec. 6.2, App. C.4, App. B

DGP (`triangle_structured_mixed.R`): x1, x2 as above; x3 ordinal with four levels, cutpoints θ = (−2, 0.42, 1.02) (from the R code — the paper does not state them), level = #{k : u3 > θ_k + 0.2 x1 + f(x2)}; f: `linear` −0.3 x, `exp` 0.5 exp(x). The paper adds the ordinal shift, the flow subtracts it, so the fitted weights are β13 = −0.2, β23 = +0.3.

Model: `x1: SI`, `x2: SI + LS(x1)`, `x3: OrdinalNode(4, LS(x1) + {LS(x2) | CS(x2)})`. CS net = reference `c(2, 2, 2, 2)`, sigmoid.

hyperparameter	paper / R code	previous	now
train / validation	`dgp(40000)` / `dgp(10000)`	36000 / 4000 split	40000 / 10000, two draws
epochs, lr, batch, schedule, init	500, 0.001, 32, none, random_normal	500 (chunks of 10), 0.001, 512, torch	**350 (`exp-cs`) / 200 (`linear-ls`)** continuous, none, `init: normal`; **lr 0.004 (`linear-ls`: 0.002) / batch 256** — the CI deviations, floors below
`n_coeffs` (x1, x2)	20	20	20

hyperparameter	paper / R code	previous	now
odds-ratio check (App. C.4)	odds(x2 ≤ −1) under do(x1 += 1), theory $e^2 \approx 7.39$	40000 rows, seed 99	same
counterfactual PMF (App. B)	Fig. 10 explains why point counterfactuals fail for the ordinal node	2000 rows × 200 draws	same

Results

variant	metric	paper	previous	paper protocol, `init: normal` (500 epochs)	CI config (batch 256; `exp-cs` 350 epochs / lr 0.004, `linear-ls` 200 / 0.002), pinned
linear-ls	β12 / β13 / β23	Fig. 19: 2 / −0.2 / 0.3	1.983 / −0.268 / 0.322	1.987 / −0.246 / 0.319	1.978 / −0.258 / 0.333
linear-ls	odds ratio predicted / DGP	$e^2 \approx 7.39$; C.4 text: 7.74, CI [7.16, 8.38]	7.26 / 7.19	7.29 / 7.19	7.23 / 7.19
linear-ls	CF PMF TV vs analytic; P(true level) flow / analytic / mode bound	— (App. B qualitative)	0.084; 0.714 / 0.728 / 0.806	0.044; 0.718 / 0.728 / 0.806	0.050; 0.716 / 0.728 / 0.806
exp-cs	β13; cs max err	Fig. 20: distributions match	−0.227; 0.885	−0.207; 0.142 (glorot: 0.071, torch: 0.126)	−0.200; 0.156
exp-cs	CF PMF TV; P(true level)	—	0.075; 0.924 / 0.921	0.023; 0.917 / 0.921	0.024; 0.920 / 0.921

VACA / CNF benchmark (`vaca.py`) — paper Sec. 5.1–5.2, App. C.1

DGP (Sanchez-Martin et al. 2022, App. E.1): x1 ~ 0.5 N(−2, 1.5) + 0.5 N(1.5, 1); x2 = −x1 + N(0, 1); x3 = x1 + 0.25 x2 + N(0, 1). Gaussian noise — outside the logistic-latent family, so the all-`CI` flow has to fit it. Analytic target: E[x3 | do(x2 = a)] = −0.25 + 0.25 a. The paper's Sec. 5.2 text says a ∈ {−3, −2, 0}; its Fig. 5 panels and the R code (`vaca_triangle.r`) intervene at a ∈ {−3, −1, 0} — the code is followed (the frozen `data/vaca/truth.json` holds the same three).

Model: `x1: SI, x2: CI(x1), x3: CI(x1, x2)`, Bernstein `n_coeffs = 31` (reference M = 30, `len_theta = 31`), nets `dense(10, tanh) → dense(100, tanh) → dense(31)` (`comparison/utils.R::make_model`).

hyperparameter	R code	previous	now
train / validation	nTrain 2500 / `dgp(5000)`	18000 / 2000 split	2500 / 5000, two draws

hyperparameter	R code	previous	now
epochs	10000 (`Figure_Triangle_Linear_Bimodal.jpg`, the sourcing script — not in our copy of the R code, so EPOCHS/M/nTrain for VACA rest on that reading)	400 in chunks of 50, then 120 "polish"	**10000**, one run — the reference, 1:1 (the 2026-09-01 4800-epoch cut was reverted 2026-09-02)
lr	0.001	0.01, polish 0.001	**0.001** — the reference
batch	full batch (one `apply_gradients` per epoch)	512	2500 = n_train
schedule	`update_learning_rate`: ReduceLROnPlateau on the summed val NLL, factor 0.1, patience 50, min_lr 1e-7, strict <	none	**the same rule** (restored 2026-09-02 with the reference epochs; it fires ~epoch 9050 and freezes an all-bounds point)
input scaling	`scale_df`: everything min-max to [0, 1]	raw	`input_transform`: `minmax` on the CI terms (network inputs; targets D1)
Bernstein domain	train min/max (`scale_df`)	5%/95% quantiles	quantiles kept (`range_q: 0.05`) — **D1, now measured under the final protocol** (2026-09-03, seed 7): the reference's min/max domain scores 0.289 / 0.040 / 0.067 against the quantiles' 0.096 / 0.080 / 0.022 — worse at the off-manifold do(x2 = −3) and at do(x2 = 0). CAREFL, same nets, measures the opposite way and matches the reference (`range_q: 0`)
n_compare	—	50000	50000

Results — the check is the flow's error against the analytic mean, not a pinned flow value.

metric	paper	previous	R 1:1, torch init, per-node plateau (08-25)	R 1:1, glorot, global plateau (08-26)	now: the reference 10000 @ 0.001 + plateau — the pinned ground truth
|E[x3 | do(x2 = −3)] − (−1.0)|	Fig. 5: densities overlap	0.037	0.026	0.097	0.096
|E[x3 | do(x2 = −1)] − (−0.5)|	Fig. 5	0.012	— (do(−2) was scored: 0.034)	0.088	0.080
|E[x3 | do(x2 = 0)] − (−0.25)|	Fig. 5	0.010	0.077	0.019	0.022
sd(x1) flow vs analytic 2.0767	Fig. 4: bimodal x1 fitted (the default CNF fails)	2.074	error 0.0077	error 0.032	2.036, error 0.040
val NLL x3	—	1.4356	1.4351	1.4496	1.4427

The 08-25 and 08-26 columns are not the same three interventions: the do(x2) set moved from the paper text's {−3, −2, 0} to the R code's {−3, −1, 0} (see the DGP paragraph above), so only do(x2 = −3) and do(x2 = 0) compare directly. At the earlier grid and with glorot the errors were 0.098 / 0.159 / 0.026 at this seed and 0.268 / 0.119 / 0.005 and 0.217 / 0.031 / 0.021 at init seeds 8 and 9 — the spread quoted below.

How the gap was traced, one change at a time under the exact protocol — 10000 epochs at lr 0.001 with the plateau rule, before the 2026-09-01 epoch cut (inputs min-max scaled unless stated; do(x2) errors at the then-current grid −3 / −2 / 0):

variant	error	reading
raw parents into the nets	0.731 / 0.426 / 0.017	the tanh nets saturate: 40 % of the rows have |x1| > 2, 43 % |x2| > 2
per-node plateau, torch init	0.026 / 0.034 / 0.077	the 2026-08-25 approximation
global plateau (R's rule), torch init	0.523 / 0.334 / 0.129	the summed NLL keeps improving through x1 while x3 overfits
… without any plateau	0.562 / 0.354 / 0.142	the rule is what stops the full-batch overfit
… Adam eps 1e-7	0.523 / 0.334 / 0.129	no effect
… Bernstein on train min/max (D1 off)	0.154 / 0.012 / 0.062	helps, not the cause
… glorot init	0.035 / 0.006 / 0.007	the cause (a different random draw than the config's seed 7)
… glorot + min/max	0.035 / 0.056 / 0.057	

Under the exact protocol the result is **seed-sensitive at the off-manifold point** do(x2 = −3):

0.03–0.27 over four init draws, 0.005–0.026 at do(x2 = 0) (measured under the 10000-epoch protocol; the paper shows one run's densities). The committed bound is 2.5× the seed-7 measurement of the reference 10000-epoch run (the 2026-09-01 4800-epoch cut was reverted 2026-09-02), and this table is the honest picture.

CAREFL benchmark (`carefl.py`) — paper Sec. 5.3, App. C.2

Since 2026-09-03 this benchmark is the reference run 1:1, on the reference's own data. `carefl_fig5.r` sets `USE_EXTERNAL_DATA = TRUE`: it trains on CAREFL's own committed 2500 rows (`data/CAREFL_CF/X.csv`, x3/x4 sd-standardized by 6.0104/1.9114) with **`val = train`**, and its repository also commits the observation (`xObs.csv`), the analytic truth curves and CAREFL's own predictions on the grid `seq(-3, 2.9, 0.1)`. All of it is frozen here under `experiments/paper/data/carefl-cf` (external input, no generator — see its `truth.json`), so the Fig. 6 curves are comparable point by point and every metric is in the reference's standardized units.

DGP (Khemakhem et al. 2021): x1, x2 ~ Laplace(0, 1/√2); x3 = x1 + 0.5 x2³ + ε; x4 = −x2 + 0.5 x1² + ε, ε ~ Laplace(0, 1/√2); x3/x4 divided by their sample sds. Counterfactuals are analytic by noise abduction. Observation: `xObs.csv` = (2, 1.5, 0.8465, −0.2616); the paper prints (2, 1.5, 0.81, −0.28), slightly off it (CAREFL's own text). **Before `feat/followups` the repository used the printed values as raw coordinates**, a 4σ-off point. Queries: x3^cf | do(x2 = α) and x4^cf | do(x1 = α) on the committed grid.

Model: `x1, x2: SI`, `x3, x4: CI(x1, x2)`, Bernstein `n_coeffs = 31`, the same `make_model` nets as VACA; `range_q: 0` (the reference's `scale_df` min-max Bernstein domain — an intercept option since 2026-09-03).

hyperparameter	R code (`carefl_fig5.r`)	2026-09-02 era	now
train / validation	CAREFL's own `X.csv`, sd-standardized, `val = train`	2500 fresh SCM rows / a separate `dgp(5000)` draw, raw units	**the same committed `X.csv`, `val = train`** (a fresh standardized draw is scored, never trained or annealed on)
epochs, lr	7000 @ 0.001	3000 @ 0.002	**7000 @ 0.001**
batch, schedule, input scaling, init	full batch, plateau 0.1/50/1e-7, `scale_df`, glorot	same	same
Bernstein domain	train min/max (`scale_df`)	5%/95% quantiles (D1)	**train min/max** (`range_q: 0`)
scoring	the single `x_obs`, curves over α (Fig. 6)	+ 300 held-out rows at α ∈ {−1.5, 0, 1.5}	same, all in standardized units

Results (standardized units; the 09-02-era numbers were raw — divide x3 by 6.01 and x4 by 1.91 to compare):

metric	paper / CAREFL	09-02 era (fresh draw, 3000 @ 0.002, quantile domain), rescaled	now — the pinned ground truth
Fig. 6 max |x3^cf error|	Fig. 6: flow tracks the DGP curve; CAREFL's committed x3 preds err up to ~0.7 at α = −3	0.45 (at α = 3)	**0.066**
Fig. 6 max |x4^cf error|	CAREFL's committed x4 preds: max 0.174	0.40; ~0.29 near α = 0 (the "dip")	**0.204**, at the α = −3 grid edge; 0.074 at α = 0
held-out CF MAE x3, α = −1.5 / 0 / 1.5	—	0.035 / 0.010 / 0.021	0.015 / 0.009 / 0.013
held-out CF MAE x4, α = −1.5 / 0 / 1.5	—	0.065 / 0.051 / 0.064	0.080 / 0.027 / 0.048

Component attribution of the closed gap, each measured on 2026-09-03 with everything else held fixed: the fresh data draw → `X.csv` cut the Fig. 6 x4 max from 0.77 (raw ≈ 0.40 std) to 0.34 std; the reference optimization (7000 @ 0.001 with `val = train`) fixed the parabola bottom (err at α = 0 0.21 → 0.06); the min-max domain cut the grid edges (x4 0.37 → 0.20, x3 0.20 → 0.07). The earlier 3000-vs-7000 trade-off (3000 won held-out MAEs, 7000 the single-point curve) was an artifact of the fresh-draw data and is settled by using the reference's rows.

The former Fig. 6 x4 "dip", root-caused 2026-09-03 before the data was adopted: on fresh 2500-row draws the flow's parabola bottom sat well below the truth near α = 0 — finite-sample tail-quantile variance, not protocol or init (invariant to 3000 vs 7000 epochs, val choice, and init seeds 7/8/9 at 0.586/0.601/0.610 raw; the observed noise sits at the Laplace 12% quantile, whose fitted value errs +0.27 in the sparse observation region x1 = 2, ~190 nearby rows, against −0.19 in the dense middle); the magnitude moved with the data draw (0.54/0.31/0.43 raw at dgp seeds 42/43/44). CAREFL's own committed draw is a mild one — one more reason training on the committed rows is the right comparison.

Runtime and the epochs

The reference protocol's cost, the motivation for every deviation here — triangle: 500 epochs × 1250 steps of batch 32, 3.2 s per epoch single-process at 2–4 torch threads (5.5 s at 32 threads, the step is overhead-bound), 42–69 min per variant on the 2-core CI runners; VACA 355 s, CAREFL 338 s (2026-08-31); the workflow's ten jobs run in parallel, 70 min wall against a 300-min cap.

The first deviation taken for CI runtime (2026-08-26) was the triangle **batch size and learning rate**: batch 256 at lr 0.004 instead of the paper's batch 32 at lr 0.001, 8× fewer optimizer steps per epoch. This is the earlier selection grid, kept as the record of that choice — it ran at the paper's 500 epochs with glorot init, against the then-committed ground-truth bands (cs max err unless noted):

batch / lr / epochs	linear-ls β13	linear-cs	atan-cs	mixed exp-cs (TV)	steps vs paper
32 / 0.001 / 500 (paper)	−0.170	0.110	0.080	0.071 (0.019)	1×
128 / 0.001 / 500	−0.170	0.170	0.041	0.126 (0.023)	1/4
128 / 0.004 / 500	−0.178	0.160	0.094	0.070 (0.014)	1/4

batch / lr / epochs	linear-ls β13	linear-cs	atan-cs	mixed exp-cs (TV)	steps vs paper
128 / 0.004 / 250	−0.170	0.147	0.070	0.131 (0.035)	1/8
256 / 0.004 / 500 (CI)	−0.171	0.132	0.073	0.072 (0.017)	1/8
256 / 0.008 / 500	−0.181	0.163	0.113	0.073 (0.017)	1/8
512 / 0.010 / 500	−0.169	0.183	0.104	0.119 (0.015)	1/16

Batch 256 / lr 0.004 is the only row that keeps every cs error within ~0.02 (linear-cs: 0.022) of the paper protocol; larger batches or rates bend the misspecified linear-cs curve (0.16–0.18) and lr 0.008 hurts atan-cs. At 500 epochs and this batch/lr, CI run 32974872751 took 7–11 min per triangle job instead of 42–69, the workflow about 12 min wall instead of 70. The grid ran with glorot init; the committed configs use `init: normal` (the triangle scripts' initializer). The paper-protocol numbers stay in this document as the reference; the ground truth is pinned from the CI config, which since 2026-09-01 also cuts the epochs — the round below.

The 2026-09-01 tuning round

The frame (repo decision 2026-09-01): the *data, model and init* stay the reference's; allowed were dropping the network-input scaling, trying sigmoid/relu activations, and tuning lr / batch size / epochs for CI runtime. Outcome: the triangle scripts already ran raw parents + sigmoid (their reference does), so they comply by construction and only their epochs moved; VACA and CAREFL **keep tanh + min-max network inputs (`input_transform: minmax`)** because every alternative was tried and measurably fails. Every tuned variant's ground truth was re-pinned from its final run (centers to the run, `{max}` at 2.5x; `fit_seconds` waits for the next CI measurement) — **the bounds and pins quoted in this section are the pre-repin 2026-08-26 values the round was measured against**, and re-pinning loosened several. The full decision trail is in each config's YAML header.

experiment	was	now
triangle `linear-ls` / `atan-cs` / `sin-cs`	500 epochs	300 epochs
triangle `linear-cs`	500 epochs	500 epochs (kept)
triangle-mixed `exp-cs`	500 epochs	350 epochs
triangle-mixed `linear-ls`	500 epochs @ lr 0.004	200 epochs @ lr 0.002
vaca	10000 full-batch epochs @ lr 0.001, plateau	reverted 2026-09-02: back to the reference 10000 @ 0.001 + plateau
carefl	7000 full-batch epochs @ lr 0.001, plateau	reverted 2026-09-03: back to the reference 7000 @ 0.001 + plateau, on the reference's own committed rows (see the CAREFL section)
validate_ls `adam` (misc)	phases 4000/2000/1000 @ 1e-2/1e-3/1e-4	800/700/500, batch 256 kept

Triangle (batch 256 / lr 0.004 / sigmoid / `init: normal` / raw parents all unchanged): at 300 epochs every metric holds its bound; the floor is real — 150 epochs fails atan's do(x1) bound (0.235 > 0.206). `linear-cs` keeps 500: at 300 its fitted cs curve visibly flattens past |x2| > 0.5, max err 0.140 vs 0.088 — the DGP line spans only ±0.3 and the sigmoid net's slow tail convergence

is half of that. relu is ruled out: the reference's 2-unit input bottleneck dies (atan: β13 +0.53, cs err 1.42).

Triangle-mixed: the cs variant's slow parts are the 2-unit sigmoid CS net and the do(x1) mean — cs max err 0.344 / 0.177 / 0.156 and do err 0.033 / 0.060 / 0.022 at 100 / 250 / 350 epochs (0.107 / 0.016 at 500), so 100 failed the then-current cs bound (0.344 > 0.2686) and 250 the do bound (0.060 > 0.0398); `exp-cs` trains 350. `linear-ls` has no CS net to wait for and trains 200 at lr 0.002, which reads the weakly identified β13/β23 out slightly closer to the 500-epoch pins than lr 0.004 does (β23 0.333 vs 0.337, pin 0.306); at 100 epochs β13 sits 57% into its tolerance (−0.271 vs the pinned −0.243). relu dies at the sd-0.05 normal init (cs err 0.859, β13 −0.055).

VACA — the instrumented finding: probing the reference trajectory (10000 @ 0.001, plateau) shows the do(x2) errors descend until epoch ~6000–8500 (best 0.005–0.10), then creep back up; the plateau anneal happens to fire at ~9050 and freezes 0.101 / 0.099 / 0.030 (an instrumented re-run; the small offset from the 08-26 pins 0.097 / 0.088 / 0.019 is run-to-run sampling of the do-means) — the reference protocol's quality IS its anneal landing inside that window. At lr 0.002 the same smooth full-batch descent runs 2× compressed with an all-bounds window at epochs ~4350–5000 (binding edges: x1's marginal std enters its 2.0766 ± 0.08 band at ~4320, the do(x2=+0) error exits after ~5000); 4800 sits inside it, every margin ≥ 2×, at under half the reference's wall clock. The plateau rule is dropped because on the compressed trajectory the summed validation NLL keeps strictly improving past the window, so it cannot fire inside it; lr 0.004 is out because x1's std clock does not compress with lr (1.978 at its do-window, epochs ~2050–2350). Rejected, both measured: relu + raw parents wanders 0.17–0.45 on do(x2=−3), holding the bound only at isolated epochs (tanh on raw parents saturates outright, 0.731); minibatch stepping (batch 256, even with minmax + tanh) converges with a systematic −0.11 common-mode offset of all three do-means — 2.4× the do(x2=+0) bound — while the observational fit stays perfect.

CAREFL (full batch + plateau kept): the round's 3000 @ 0.002 pick was an artifact of the then-current fresh-draw data and was reverted on 2026-09-03 with the move to the reference's own committed rows (see the CAREFL section — the 3000-vs-7000 trade-off does not exist on `X.csv`). Still standing from the round: raw parents rejected — sigmoid saturates on the Laplace parents just like tanh, and relu underfits x3 (val NLL 1.46–1.47 vs the 1.403 ± 0.05 band) while growing a fragile Bernstein tail (one held-out row inverted to x4 ≈ 580, cf MAE 4.36 vs bound 0.33); minibatch — the fig6 grid ends live in the sparse x2 tail that minibatch gradients underweight (cf MAE x3 do(x2=+1.5) 0.40 — over the old 0.355 and the re-pinned 0.319 bound alike — and fig6 x3 err 5.9 against the full-batch run's 2.7).

validate_ls `adam` (misc, not a paper experiment but tuned in the same round): phases 800/700/500 instead of 4000/2000/1000 — 2000 epochs instead of 7000 — still lands on the classical MLE, named-coef gap to statsmodels 1.6e-5 vs the old 1.8e-5, ~3.5× less fit wall clock.

Local wall times under the tuned protocol (32-core box, 4 torch threads): triangle 162–281 s per variant, mixed 88–165 s, vaca 46.5 s, carefl 72–82 s. The CI timings quoted above and the `fit_seconds` pins in `ground_truth/` are still the 2026-08-31 pre-cut measurements; both will be re-measured on the next push and the pins updated then.

Repository choices the paper does not state

Seeds; the validation draw sizes for the triangle scripts (the R code's `test`); `n_compare`; `n_heldout = 300` and the α set {−1.5, 0, 1.5} scored next to the single `x_obs`; the mixed cutpoints from the R code; the odds-ratio sample (40000 rows) and the counterfactual-PMF sample (2000 rows × 200 draws).

Code map — every module of `src/tramdag/` — the public names and the private plumbing

One entry per name, with its role and its place in the pipeline. Names in parentheses are private machinery: useful to know, not part of the API. The last section lists every training hyperparameter and where it lives.

`spec.py` — declare the model

Every term is a `Term` subclass under its pythonic name; the paper's symbol is the same object, so `LS is LinearShift`.

Name	Role
`Term`	One additive term of a node's transformation, frozen data; each term is a subclass whose annotated attributes are its options (defaults dropped on serialization, so equality is canonical). `+` on terms builds plain lists. Carries the spec-level rules: `edge_parents`, `cells`, `classical`, `options()`, `from_serialized`. Subclass `Term` for a new effect; its module goes in `terms.py` and read as attributes (`term.penalty`, `term.units`, ...).
`SI()`	The parentless intercept — the paper's SI. Free transform parameters, the same for every row. Carries the transform choice (`transform=`, default `"bernstein"`); extra keyword arguments pass straight to the transform class.
`CI()`	The parent-conditioned intercept — the paper's CI: the parents reshape the monotone transform. Needs at least one parent. Also carries `units=` and `allow_interaction=` (joint vs. additive multi-parent intercept).
`Intercept` / `I`	The intercept term class: without parents the paper's SI, with parents the CI; `SI()`/`CI()` are the two spellings with their arity checked. The bare names `I` and `SI` in a term list both mean the simple intercept.
`LinearShift` / `LS`	Linear shift `beta * x` — the interpretable log-odds coefficient. Exactly one parent.
`ComplexShift` / `CS`	Complex shift: an NN `g(x)`, additive on the latent scale. Several parents form one joint network.
`VaryingCoefficient` / `VC`	Varying-coefficient shift `(beta0 + b_theta(mods)) * x_t` — the penalized treatment-effect head (issue #28). `center=` adds propensity centering (issue #30).
`ContinuousNode`	Continuous variable: monotone 1-D transform plus shifts. `terms` is the first positional argument.
`OrdinalNode`	Ordinal variable with `levels` classes: ordered logit (cutpoints) plus shifts.
`node_parents()`	Ordered de-duplicated parent names of a node (the canonical term list is `node.terms`).
`validate_and_sort()`	Edge-ownership validation plus Kahn topological sort. The returned order makes the flow triangular.
`spec_to_dict()` / `spec_from_dict()`	Checkpoint (de)serialization. A term serializes as `{term, parents, options}` and nothing else, since `options` is already canonical. No compatibility shims: `spec_from_dict` rejects a term without `options`, the node constructors normalize the formula, and `validate_and_sort` checks the DAG.

Name	Role
(_normalize_terms, _as_term, _term_class)	Formula flattening and per-entry validation (a + sum nested in a list is rejected), the one-parented-I rule plus transform hoisting in one pass, canonical option storage against each effect's option_defaults (a wrong-effect option errors).
(_check_node, _kahn_sort)	The stages behind validate_and_sort: parents exist, then each term's edge_parents (the VC treatment and centering rules), then edge ownership; a term's own shape (arity, option values) and a node's own (ordinal levels, the transform) are checked when they are built. Kahn's sort emits ready nodes in sorted batches, so the order is deterministic.

transforms.py — the monotone map h and the ordinal transform

Name	Role
StandardLogistic	The TRAM base distribution: log_prob, sample (generator-aware), icdf.
BernsteinUT	Bernstein-polynomial transform (the default, n_coeffs=20). Linear tail extrapolation follows the boundary derivative. marginal_init_theta(column) gives the marginal start init_marginals applies: the Bernstein approximation of logit(F_hat(y)), or the plain linear map onto the latent's range_q quantiles when called without a column.
SplineUT	Monotone rational-quadratic spline (bins=8). Tails extrapolate with a *fixed* slope — the structural reason spline trails Bernstein on tail-heavy data.
AffineUT	Monotone affine transform: the node-conditional is a logistic GLM.
make_univariate_transform()	Transform registry: name → transform instance.
ordinal_cutpoints()	Unconstrained (n, K-1) → increasing cutpoints with ±inf ends. Port of the original parametrization.
ordinal_log_prob()	log P(Y=y), computed in log-space. Load-bearing: the naive sigmoid difference saturates in float32 and freezes nodes at init. Do not simplify.
ordinal_pmf() / ordinal_sample() / ordinal_abduct()	Class probabilities / latent → level / truncated-logistic latent recovery (Pearl step 1) for ordinal nodes.
ordinal_marginal_init_theta()	Cutpoint start that matches the empirical class frequencies (init_marginals).
(_ScaledUT, _bounds, _log1mexp)	Quantile pre-scaling base class (the inverse is zuko's, with its closed-form tail); per-level cutpoint intervals; stable log(1-exp(x)).

conditioners.py — the networks behind the terms

Default architectures replicate the PyTorch reference this package grew out of (buehlpa/TramDag, tram_models.py), so a fitted model stays comparable to it. They are **not** the TRAM-DAG paper's nets: the paper's R code uses c(2, 25, 25, 2) with sigmoid for the triangle experiments and a 10-100 tanh net for its CAREFL/VACA comparisons, so each config in experiments/paper/ states units= and activation= itself.

Name	Term	Role
`SimpleIntercept`	bare `I`	Free parameter vector; no parents.
`ComplexIntercept`	`I(...)`	8-8 ReLU NN from parent features to the transform parameters.
`LinearShift`	`LS`	`Linear(n, 1, bias=False)`. `.weight` is the interpretable coefficient; no bias because the intercept slot owns the constant.
`ComplexShift`	`CS`	64-128-64 ReLU NN to one shift value.
`VaryingCoef`	`VC`	`beta0 + b_theta(mods)` with a zero-initialized output layer and the L2 hook `l2()`. `beta()` evaluates the effect, `recenter()` re-splits `beta0`/`b_theta` after training (function-preserving).
`(_nn)`	—	The one NN builder: a stack of the given `units` with the term's `activation` (relu by default, optional `batch_norm` before it), then a bias-free output layer.

`flow.py` — the model

Name	Role
`CausalFlowDAG`	The flow: one `_Node` per variable in topological order. Construction seeds the weights (`seed=` is the reproducibility knob).
`calibrate()`	Once, from the training rows, each term for itself: transform ranges (train `range_q` quantiles onto the domain), input-transform statistics. Never touches the weights. Called by the first fit; a checkpoint carries the flag.
`init_marginals()`	The calibrated start as an explicit step, callable any time: resets every simple intercept to its column's marginal (Bernstein map / ordinal class log-odds; spline and affine have no calibrated start). Not once-guarded — on a trained flow it restarts those intercepts. Calibrates a fresh flow's ranges itself.
`fit()`	Joint maximum likelihood: one minibatch Adam loop over all parameters (exact per node, because the NLL decomposes), final weights kept. Keras-shaped validation (`validation_data=`/`validation_split=` fill `history["val"]` per epoch, `validation_batch_size=` chunks the pass), the optimizer's rate after every epoch in `history["lr"]` (`{node: lr}` with `per_node_adam`), and progress (`verbose=`). Hooks: `optimizer=` (any torch optimizer, for schedulers) and `callbacks=` (one entry or a list; a `Callback` hooks `on_fit_begin`/`on_epoch_end`/`on_fit_end`, a bare callable is an `on_epoch_end` hook `cb(flow, epoch, opt)` — any `True` stops); the recipes in `callbacks.py` read `history["val"]`. A centered VC's out-of-fold propensities ride the training frame as the column its `center=` names. A second call continues training.
`fit_classical()`	Float64 full-batch L-BFGS for all-`ls` specs: deterministic, exact MLE, matches `statsmodels`/R `polr`. Refuses flexible specs.
`sample()`	Observational, interventional (`do=`, graph mutilation) and counterfactual (`u=`) sampling.
`abduct()`	Pearl step 1: recover the latents. Continuous exactly, ordinal by truncated draw.
`pmf()`	Analytic class probabilities of an ordinal node, with `do=` overrides.
`density()`	Analytic conditional density of a continuous node on a grid, with `do=` overrides — the continuous counterpart of `pmf`.

Name	Role
`log_prob()` / `nll()`	Joint per-row log-likelihood, or a `nodes=` subset (exact, and the log-space way to get one node's conditional likelihood per row) / mean per-node NLL diagnostic.
`node_log_prob()`	The per-node decomposition everything trains and evaluates through.
`varying_coef()`	Closed-form read-out `beta(x)` of a fitted VC term. Deterministic, y-free.
`scores()` / `effect_modifier_scan()`	Analytic per-observation scores and the CUSUM modifier scan (delegate to `scores.py`).
`intercept_contributions()`	Post-hoc GAM-style decomposition of a complex intercept into mean-centered per-term parts.
`ls_coefficients()`	The per-node linear-shift weights — the interpretable coefficients.
`design_matrix()`	Parent encoding as a DataFrame (`drop_first=` gives the classical statsmodels/`polr` design).
`to_matrix()`	The labeled meta-adjacency matrix of term tags.
`save()` / `load()`	Checkpoints with history and provenance (version, time, device). `load` requires a complete checkpoint and fails loudly otherwise.
(`_node`, `_encode_parent`, `_features`, `_tensorize`, `_generator`, `_dtype`, `_np_dtype`)	Node lookup with one shared error; parent encoding (continuous raw, ordinal one-hot); `_tensorize(df, cols=None)` for any column subset, checking every ordinal column against its levels on the way in; seeded-generator and dtype plumbing.
(`_check_side_columns`, `_binary_p1`, `_side_feats`, `_query_side_columns`, `_recenter_vc`)	The generic side-column plumbing (each term names/validates/recomputes its own columns via the `ShiftTerm` hooks) plus the binary propensity fit and the post-fit `finalize` loop.
(`_is_classical`)	Guard for `fit_classical`: every term's `classical` — `LS`, or a parentless `I()` transform carrier.

`terms.py` — the term modules (the 1.0 architecture's core)

One module class per term, declaring the `Term` subclass it builds (`data = CS`); see architecture.md for the contract diagram.

Name	Role
`module_for()`	The dispatch: a `TermDef` subclass declaring `data = <Term subclass>` stamps itself onto that class as `module` when defined, so subclassing is the registration; a term class no module declares fails by name.
`ShiftTerm` / `InterceptTerm`	The behavior hooks a term module owns: `build`, `shift_value`/`theta_value`, `post_init`, `regularizer`, post-fit `finalize`, `score_columns`, the side-input contract; `data` names the `Term` subclass it builds.
`LinearShiftTerm` / `ComplexShiftTerm` / `VaryingCoefficientTerm` / `FnShiftTerm`	The built-in shift terms, subclassing their conditioners (state-dict paths and the seeded RNG stream stay bit-stable). `VaryingCoefficientTerm.regressor` is both the forward regressor and the `beta0` score.
`SimpleInterceptTerm` / `ComplexInterceptTerm` / `AdditiveInterceptTerm`	The intercept slot: free theta, one joint net, or one net per parent summed in coefficient space.

`nodes.py` — the node model

Name	Role
(_Node)	One sub-model per variable: builds its intercept and shift terms through module_for; theta_shift() sums the terms' shift_values (plain shifts first, then VC); net_input() feeds every term network, input_transform applied.
(_InputTransform)	One term's frozen network-input transform (minmax / standardize / callable over frozen train columns).
(kind_log_prob, kind_sample, kind_abduct, kind_marginal_theta)	The ONLY continuous-vs-ordinal branches in the package, adjacent.
(_init_linear)	Keras' glorot/normal initializers on one linear layer.

fitting.py — _FitMixin

Name	Role
fit() / fit_classical() (_split_validation, _normalize_callbacks, _check_epoch_hook, _check_fit_sizes, _epoch_pass, _log_epoch, _val_nll, _fit_epoch, _FnCallback)	Defined here once, methods of the flow via the mixin. The loop plumbing: Keras-shaped validation split, callback normalization and pre-fit signature checks, the epoch/validation passes, verbose printing.

readouts.py — _ReadoutsMixin

Name	Role
shift_curve()	One fitted shift term on a 1-D grid, through the term's own shift_value — the public replacement for reaching into nd.shifts[..].
the read-out methods	varying_coef, ls_coefficients, to_matrix, intercept_contributions, design_matrix — defined here once, methods of the flow via the mixin.

scores.py — effect-modifier detection (issue #29)

Name	Role
node_scores()	Analytic, exact per-observation scores psi_i = d l_i / d theta for every LS weight and VC beta0. No autograd.
effect_modifier_scan()	Zeileis-Hornik fluctuation scan: order the treatment scores by each candidate, sup|CUSUM| against the Kolmogorov 5% value. A measured shortlist for VC modifiers from a seconds-long classical fit.
sup_bb_pvalue()	P(sup |Brownian bridge| > stat), the Kolmogorov series.
(_dl_ds, CRIT_5PCT)	Closed-form latent-scale derivative; the 5% critical value 1.3581. The per-term columns come from each term's score_columns hook.

`callbacks.py` — the shipped `fit` callbacks

Name	Role
`EarlyStopping`	Snapshots the weights of the best summed validation NLL (read from `history["val"]`) and restores them automatically at fit end (`restore_best=False` keeps the final weights), before the VC re-centering; an optional `patience` also stops the fit once the best is that many epochs old.
`PerNodePlateau`	Per-node lr decay and freezing on each node's own validation NLL (from `history["val"]`); stops the fit once every node froze, and records `frozen = {node: epoch}`. The pre-0.4 `fit(schedule="plateau")` recipe, opt-in. `step(nll, opt)` for a hand-computed NLL.
`per_node_adam()`	Adam with one `node`-tagged parameter group per node — the optimizer `PerNodePlateau` needs.

`plots.py` — the figures (matplotlib optional: `tramdag[plots]`)

Name	Role
`plot_dag()`	The labelled DAG of a spec or flow: layered left to right, ellipses for continuous and rounded boxes for ordinal nodes, every edge drawn by the term that owns it (LS / CS / CI / VC + modifiers / Fn, `joint` for a multi-parent net). Exported as `tramdag.plot_dag`.
`plot_marginals()`	Observed vs sampled marginal per node, one panel each.
`plot_training()`	Summed train/val NLL per epoch, with the freeze marks read off `history["lr"]` (or a given `frozen=`).
`(_layout, _edges)`	Longest-path layers with one barycenter sweep; the edge list with the VC treatment/modifier split. matplotlib is imported on the first call, never at package import.

What is *not* in the package

The SCM generators, the frozen datasets and the replication scripts are research code and live in `experiments/`, outside the installed package — see `experiments/README.md`. The framework's own tests do not depend on them: they measure against three inline DGPs in `tests/conftest.py`.

Where every training hyperparameter lives

Everything that shapes a fit is either a keyword you pass or a documented default you can read at the call site. Nothing numeric is buried.

Knob	Where	Default
learning rate, batch size	`fit()`	1e-2 / 512 (in-repo callers state them explicitly anyway)
validation, progress	`fit(validation_data=, validation_split=, valida-tion_batch_size=, verbose=)`	per-node val NLL into `history["val"]` each epoch; `verbose=N` prints every Nth + final epoch (default 0, silent)

Knob	Where	Default
schedules, early stopping	`fit(optimizer=, callbacks=)`	`tramdag.callbacks` ships `EarlyStopping`, `PerNodePlateau`; anything else is torch's `lr_scheduler` and a few lines of callback (fitting.md)
calibrated init	`init_marginals(train_df)`	never implicit — an always-explicit step (pure init, MLE unchanged; without it zuko's zero start)
VC stage-1 propensities	the training-frame column `VC(center=)` names	required for a centered VC term, out of fold, computed by the caller
VC penalty and centering	`VC(penalty=, center=)`	1.0 / False (`center="col"` names the propensity column)
L-BFGS budget	`fit_classical(max_iter=, tol=, history_size=)`	400 / 1e-9 (torch `tolerance_change`; `tolerance_grad` is off) / 50 — one full-batch run, no chunks
training budget	`fit(epochs=)`	**required** — a fixed default is wrong in both directions (training-speed)
network widths	`units=` on `I/CS/VC`	(8, 8) / (64, 128, 64) — parity with the PyTorch reference's default classes; VC's (16,) has no counterpart there and comes from the recovery measurement
activation	`activation=` on `I/CS/VC`	`"relu"` (the reference default classes); `"sigmoid"` and `"tanh"` are the paper's
batch norm	`batch_norm=` on `I/CS/VC`	`False` — neither reference uses it. `True` puts a `BatchNorm1d` between each hidden layer and its activation, so the fit needs more than one row per batch and inference needs `eval()` mode (`fit` and `load` leave the flow there)
transform class	`I(transform=, **kwargs)` (extra kwargs go to the transform class)	`"bernstein"`, `n_coeffs=20` unconstrained coefficients (zuko ties two more control points on, so order 21); spline `bins=8` = zuko's NSF default (the domain is fixed at [-5, 5], `transforms.BOUND`)
shuffling / weight init	`fit(seed=)` / `CausalFlowDAG(seed=)`	init happens at construction — the constructor seed is the reproducibility knob
weight init	`CausalFlowDAG(init=)`	`"torch"` (`nn.Linear` Kaiming-uniform); `"glorot"` = Keras `Dense` default, glorot-uniform weights and zero biases — the paper's reference; decisive under its full-batch protocol
network inputs	`CI/CS/VC(input_transform=)`	`None` (raw parents); `"minmax"` / `"standardize"` transform that term's continuous parents with statistics frozen at `calibrate`, and a callable `fn(x, train)` gets the frozen raw train column — LS and the VC treatment stay raw

Upstream candidates for zuko

What tramdag works around in zuko (1.6.0), ranked by value/effort as candidate upstream PRs/issues. tramdag uses zuko surgically — only `zuko.transforms` (`BernsteinTransform`, `MonotonicRQSTransform`, `MonotonicAffineTransform`) behind the three wrappers in `src/tramdag/transforms.py`; the flow machinery, base distribution and DAG structure are tramdag's own. Suggested order of attack: 4 → 1 → 3 → 2 → 5.

1. Analytic `call_and_ladj` for `BernsteinTransform`

`BernsteinTransform` inherits `MonotonicTransform.call_and_ladj`, which gets the Jacobian by `torch.autograd.grad` in the forward pass. That roughly doubles the training graph and makes the per-node loss un-`torch.compile`-able (double backward; see docs/research/REPORT.md). The derivative is closed form (`f'(x) = order·Δθ` against the order−1 basis — zuko already computes it in `_offset_and_slope` for the tail slopes), so an override is ~25 lines with no API change, consistent with `MonotonicRQSTransform` which already has one. tramdag gets a free speedup and the `torch.compile` axis back.

2. Learnable/linear tails for `MonotonicRQSTransform`

zuko pads the knot derivatives with `exp(0)=1` and extrapolates as the identity outside `[-B, B]`, so the tail slope is fixed at 1 regardless of θ — the structural reason `spline` consistently trails `bernstein` here (~10% of data sits beyond the 5%/95% pre-scaling range; CLAUDE.md, `spec.py`, demo notebook section 6). Upstream: accept boundary derivatives (shape `(*, K+1)`) or an opt-in `tails="linear"`; identity tails are deliberate in the NSF design, so the framing must be back-compatible. Would delete the caveats and make `spline` a first-class transform choice.

3. Public inverse of `_constrain_theta`

`BernsteinUT.marginal_init_theta` closed-form-inverts zuko's *private* `_constrain_theta` (cumsum-of-softplus), hard-coding both its internal `log(2)·n/2` centering and its tying of the first two and the last two diffs — any upstream reparametrization silently breaks the calibrated start. A public `unconstrain_theta(theta)` taking the target control points is ~15 lines upstream and removes the private-detail coupling. Framing: "identity/linear initialization support", a standard flow trick.

4. Docstring fix: the Bernstein θ-shape off-by-one

zuko's docstring says θ has shape `(*, M-2)` for a degree-M polynomial; `n` unconstrained coefficients become `n+2` control points = degree `n+1`, so the correct claim is `(*, M-1)`. This off-by-one is a real replication trap (order 21 vs the paper's `len_theta=20` = order 19) that costs tramdag prose in three docs. Trivial PR, near-certain accept.

5. `Logistic` in `zuko.distributions`

Neither zuko nor torch ships a Logistic distribution; tramdag hand-rolls `StandardLogistic` (~50 lines: `log_prob`, `sample`, `icdf`). Logistic latents are standard in transformation models and discrete flows, and zuko has precedent (`GeneralizedNormal`) — but zuko may point at `TransformedUniform(SigmoidTransform().inv)`, and tramdag would keep its `_U_EPS` clamp locally either way. Lowest value of the five.

Anti-candidates (look upstreamable, are not)

- **Quantile pre-scaling / `_ScaledUT`**: a modeling choice; zuko's `ComposedTransform` with `MonotonicAffineTransform` already composes it.
- **The ordinal ordered-logit transform and log-space `ordinal_log_prob`**: not a bijection, outside zuko's flow scope (torch.distributions material, if anywhere).
- **LS/CS/CI/VC terms, marginal-init policy, propensity centering, scores**: TRAM-DAG semantics. zuko's `MaskedMLP` already supports arbitrary adjacency; tramdag doesn't use it because it needs per-term interpretability.
- **Spline `slope` / Bernstein `eps` knobs**: already exposed upstream.

Additive vs joint complex intercept — interpreting per-parent effects

A node whose transform parameters depend on its parents (a **complex intercept**, `CI`) can group those parents two ways:

- **joint** — `CI("x1", "x2")`: *one* network over both parents. It can represent **interactions** (the effect of `x1` may depend on `x2`), but the two parents are entangled in one black box. This is the default.
- **additive** — `CI("x1", "x2", allow_interaction=False)`: *one network per parent*, summed in unconstrained parameter space, `theta(pa) = net_1(x1) + net_2(x2)`. Each parent reshapes the transform **independently** — a separable, GAM-like structure.

The grouping is said with the flag, not by writing two terms: a node takes at most **one** intercept term with parents, so `[CI("x1"), CI("x2")]` is an error. That keeps a term list purely additive on the latent scale.

The additive form is the interpretable one: you can ask "what does `x1` *alone* do?". The catch (issue #20) is that the additive sum is identified only up to a constant moving between the nets, so the **raw** per-parent outputs are not comparable. `flow.intercept_contributions(data, node)` resolves this with a sum-to-zero (mean-centering) constraint and returns each parent's centered contribution.

This notebook fits both models and shows the interpretability difference. The punchline is perhaps surprising: the two models fit the **data** almost identically — the difference is **structural**, visible only in parameter space, and that is precisely what `intercept_contributions` surfaces.

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

from tramdag import CI, CausalFlowDAG, ContinuousNode
from tramdag.callbacks import EarlyStopping
```

The data

`x1, x2 ~ U(-2, 2)` and $x_3 = x_1 + x_2 + \gamma\, x_1 x_2 + \frac{1}{2}\varepsilon$, $\varepsilon \sim \text{Logistic}(0,1)$ — a genuine `x1·x2` interaction.

```
GAMMA = 0.6


def simulate(n, seed):
    rng = np.random.default_rng(seed)
    x1 = rng.uniform(-2, 2, n)
    x2 = rng.uniform(-2, 2, n)
    eps = rng.logistic(0, 1, n)
    x3 = x1 + x2 + GAMMA * x1 * x2 + 0.5 * eps
    return pd.DataFrame({"x1": x1, "x2": x2, "x3": x3})


train = simulate(4000, seed=1)
val = simulate(1000, seed=2)
train.head()
```

x1

x2

x3

0

0.047286

0.755745

1.239427

1

1.801855

0.126365

0.853817

2

-1.423362

0.970406

-0.294179

3

1.794598

1.590401

4.880817

4

-0.752674

-0.168717

-0.689205

Fit both models

Same data, same node — only the grouping of the two `I` parents differs.

```
def make_flow(joint: bool):
    spec = {
        "x1": ContinuousNode(),
        "x2": ContinuousNode(),
        "x3": ContinuousNode(
            [CI("x1", "x2")] if joint else [CI("x1", "x2", allow_interaction=False)]
        ),
    }
    return CausalFlowDAG(spec, seed=0)


flow_joint = make_flow(joint=True)
flow_add = make_flow(joint=False)

for f in (flow_joint, flow_add):
    f.fit(
        train,
        epochs=1200,
        learning_rate=1e-2,
        validation_data=val,
```

```
        callbacks=EarlyStopping(),  # keep the best-validation weights
    )

# Both fit the data well; the joint model is only marginally better on held-out
# likelihood. A flexible additive Bernstein intercept can *mimic* a lot of
# apparent interaction through its nonlinear transform, so you generally **cannot
# tell the two apart from the fitted distribution** — see the note below.
print("val NLL  joint   :", round(sum(flow_joint.nll(val).values()), 4))
print("val NLL  additive:", round(sum(flow_add.nll(val).values()), 4))
```

```
val NLL  joint   : 4.1711
val NLL  additive: 4.202
```

`intercept_contributions` — the exact, centered decomposition

It returns each `I`-term's mean-centered (sum-to-zero) contribution to the transform parameters `theta`, plus the absorbed `baseline`. The centering is over the rows of the `df` you pass, and the decomposition is exact.

```
res_add = flow_add.intercept_contributions(train, "x3")
res_joint = flow_joint.intercept_contributions(train, "x3")

print("additive terms :", list(res_add["contributions"]))  # 'x1', 'x2' — separable
print("joint terms    :", list(res_joint["contributions"]))  # 'x1+x2' — inseparable

# the decomposition is exact (baseline + contributions == theta, pinned by the
# test suite); the centering is easy to see here: every column mean is ~0
print(
    "per-term column means (≈0)   :",
    {k: float(np.abs(v.mean(0)).max()) for k, v in res_add["contributions"].items()},
)
```

```
additive terms : ['x1', 'x2']
joint terms    : ['x1+x2']
per-term column means (≈0)   : {'x1': 3.5154818078808603e-07, 'x2': 6.728172365910723e-
07}
```

The structural difference: separable curves vs an entangled cloud

Pick the transform coefficient that varies most across the data and plot each term's centered contribution to it against a parent value.

- **Additive** (`net_1` depends only on `x1`): its contribution to any coefficient is a deterministic 1-D function of `x1` — the points collapse to a **clean curve** (same for `x2`). These curves *are* the per-parent partial effects: well-defined because the structure is separable.
- **Joint** (one network over both parents): the single `x1+x2` component depends on *both*, so plotted against `x1` alone it is a **cloud** whose height is explained by the hidden `x2` (color). There is nothing to read off per parent.

```
c1 = res_add"contributions"
c2 = res_add"contributions"
cj = res_joint"contributions"
k = int((c1.var(0) + c2.var(0)).argmax())  # most-varying coefficient
x1v, x2v = train["x1"].values, train["x2"].values

fig, axes = plt.subplots(1, 2, figsize=(11, 4.2), sharey=True)
o1, o2 = np.argsort(x1v), np.argsort(x2v)
axes[0].plot(x1v[o1], c1[o1, k], color="#1b9e77", lw=2.5, label="net(x1)  vs x1")
```

```
axes[0].plot(x2v[o2], c2[o2, k], color="#d95f02", lw=2.5, label="net(x2)  vs x2")
axes[0].axhline(0, color="0.7", lw=0.8)
axes[0].set_title("additive — separable per-parent curves")
axes[0].set_xlabel("parent value")
axes[0].set_ylabel(f"centered contribution to theta[{k}]")
axes[0].legend()

sc = axes[1].scatter(x1v, cj[:, k], c=x2v, s=8, alpha=0.6, cmap="coolwarm")
axes[1].axhline(0, color="0.7", lw=0.8)
axes[1].set_title("joint — vs x1 is a cloud (height set by x2)")
axes[1].set_xlabel("x1")
fig.colorbar(sc, ax=axes[1], label="x2")
fig.tight_layout()
plt.show()
```

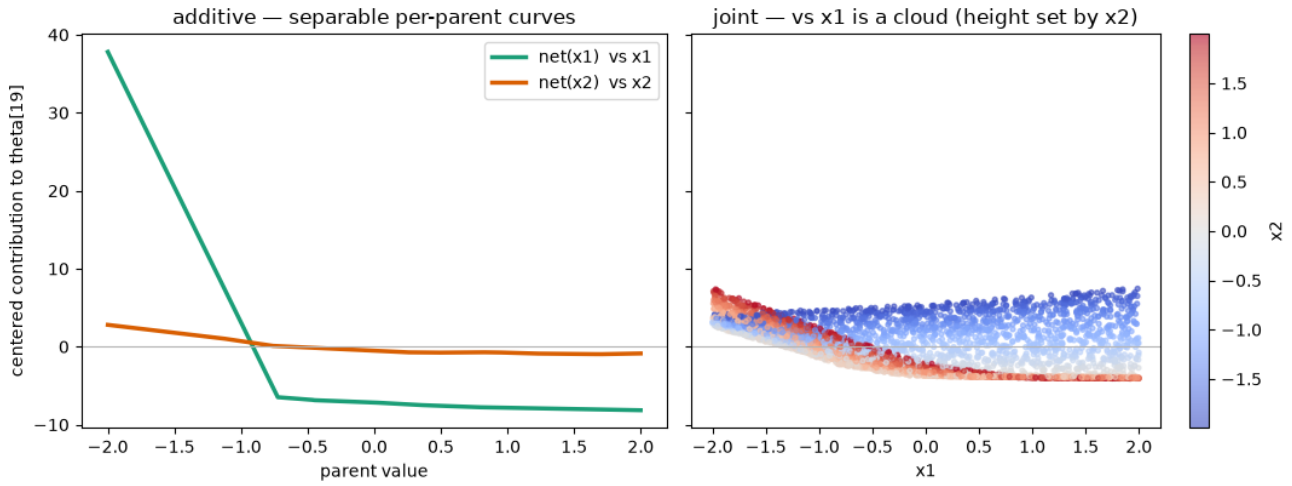


Figure 1: png

Takeaway

you want…	use	what you get
a per-parent partial-effect plot (“what does x1 do?”)	**additive** `CI("x1", "x2", allow_interaction=False)`	`intercept_contributions` → exact, mean-centered, **separable** components
interactions between parents in the transform	**joint** `CI("x1", "x2")`	one entangled network — flexible, **not** separable

Both give correct likelihoods and L1/L2/L3 causal queries — the choice is about *interpretability*, not correctness, and (as the near-equal NLLs show) you usually can’t distinguish them from the fitted distribution alone. The separability that makes per-parent effects well-defined is a property of the **parameter space**, and `intercept_contributions` is what surfaces it. It is post-hoc: it reads the fitted weights and changes nothing about the model.

Caveat: contributions live in the transform’s **unconstrained** parameter space (where the additive terms are summed, before the monotonicity constraint), so they are exact partial effects on the parameters but not, in general, an additive split of the curve `h` itself.

Classical fitting of all-`ls` TRAM-DAGs (`fit_classical`)

When **every edge of a TRAM-DAG is a linear shift (`LS`)**, each node-conditional is a *classical transformation model*

- an ordered-logit / proportional-odds model for ordinal nodes
- a continuous-outcome logistic transformation model (R's `tram::Colr`) for continuous ones.

For such a model, the classical optimizer `flow.fit_classical()` is best suited: - **full-batch, float64, L-BFGS** with a strong-Wolfe line search, - **deterministic** — no minibatching, so the same start gives bit-identical results, - lands on the **exact maximum-likelihood estimate**, matching `statsmodels` and R to ~1e-3 on the well-identified coefficients.

Every R reference printed below is hard-coded from a real fit, and every one of those fits lives in `notebooks/classical_fit_tram_dag.R`. Run `Rscript notebooks/classical_fit_tram_dag.R` from the repo root to re-check them; it needs `MASS` (ships with R) and `tram`.

```
import time
import warnings
from pathlib import Path

import numpy as np
import pandas as pd
import torch
from statsmodels.miscmodels.ordinal_model import OrderedModel

from tramdag import LS, SI, CausalFlowDAG, ContinuousNode, OrdinalNode
from tramdag.callbacks import PerNodePlateau, per_node_adam

warnings.filterwarnings("ignore")

# repo-relative data, whether the notebook runs from the repo root or notebooks/
HERE = [Path.cwd(), *Path.cwd().parents]
REPO = next(p for p in HERE if (p / "pyproject.toml").exists())
DATA = REPO / "experiments" / "misc" / "data"
NB_DATA = REPO / "notebooks" / "data"
```

0. The zeroth example — logistic regression on `MASS::birthwt`

Before any DAG, take the model everyone already knows, on a dataset every R user already has. A **two-level** `OrdinalNode` whose terms are all `LS` *is* logistic regression — not an analogue of it, the same model. The ordinal transform is

$$P(Y \le 0 \mid x) = \sigma\big(\theta_0 - \textstyle\sum_p w_p x_p\big)\,,$$

and with two levels there is a single cutpoint θ_0, so

$$\operatorname{logit} P(Y = 1 \mid x) = -\theta_0 + \textstyle\sum_p w_p x_p.$$

The shift weights **are** the logistic-regression coefficients; the intercept is **minus** the cutpoint, because an ordinal node *subtracts* its shift. That sign convention is the one thing worth internalizing here — Section 2 is this same picture with $K-1$ cutpoints instead of one.

The data is `birthwt` from **MASS**, the low-birth-weight study of Hosmer & Lemeshow: 189 births, outcome `low` (birth weight under 2.5 kg), predictors `age` (mother's age), `lwt` (mother's weight at

last period, lbs) and `smoke` (smoking during pregnancy). `notebooks/data/birthwt.csv` is those four columns exported verbatim from MASS. Please note that this is not a **causal** model.

The R you can copy-paste

`birthwt` ships with MASS, so nothing needs downloading — paste this into any R session and you have the reference fit:

```
library(MASS)
m <- glm(low ~ age + lwt + smoke, data = birthwt, family = binomial)
coef(m)
logLik(m)
```

which prints

```
(Intercept)         age         lwt       smoke
 1.36822527 -0.03899458 -0.01213854  0.67076374
'log Lik.' -111.4396765 (df=4)
```

(R 4.2.3, MASS 7.3-58.2.) Those four numbers are hard-coded below as `R_GLM`, so the notebook can show the three-way comparison without an R installation.

```
R_GLM = {  # coef(glm(low ~ age + lwt + smoke, birthwt, family = binomial))
    "intercept": 1.3682252685,
    "age": -0.0389945827,
    "lwt": -0.0121385423,
    "smoke": 0.6707637407,
}
R_LOGLIK = -111.43967649

bw = pd.read_csv(NB_DATA / "birthwt.csv")
print(
    bw.head(3).to_string(index=False),
    f"\n\n{len(bw)} births, {bw['low'].sum()} of them low birth weight",
)
```

```
 low  age  lwt  smoke  bwt
   0   19  182      0 2523
   0   33  155      0 2551
   0   20  105      1 2557

189 births, 59 of them low birth weight
```

The spec. `low` is the outcome; `smoke` is a binary **parent**, so it is an ordinal node too. `age` and `lwt` are source nodes — the flow models their marginals as well, which a plain `glm` does not. That is a nuisance here, not a feature, so they get the cheapest transform (`SI(transform="affine")`), and it cannot disturb the outcome node in any case: the joint NLL decomposes per node, so the `low` node's gradient never sees the covariate marginals.

```
spec_bw = {
    "age": ContinuousNode([SI(transform="affine")]),
    "lwt": ContinuousNode([SI(transform="affine")]),
    "smoke": OrdinalNode(2),
    "low": OrdinalNode(2, [LS("age"), LS("lwt"), LS("smoke")]),
}

flow_bw = CausalFlowDAG(spec_bw, seed=0)
flow_bw.fit_classical(bw)
```

```
{'converged': True,
```

```
 'n_iter': 64,
 'final_nll': 9.137612929338468,
 'grad_norm': 0.0001517562203257858,
 'seconds': 1.1383557839999412,
 'coefficients': {'low': {'age': array([-0.03899488]),
   'lwt': array([-0.01213841]),
   'smoke': array([-0.13497243,  0.53579548])}}}
```

Now read the coefficients back out. `age` and `lwt` are continuous parents, so they enter raw and their weights compare directly. `smoke` is ordinal, so it enters **one-hot over both levels**.

> **Careful — different coding from R.** R gives a binary predictor one column; we give it two, w_0 and w_1. Only the difference $w_1 - w_0$ is determined by the data, and it is what R's single `smoke` coefficient means. Each level alone is fixed by the weight initialization: a different `seed` shifts both by the same constant and moves θ_0 to compensate, leaving every fitted probability unchanged. Section 2 uses the same rule for `T`.

The second column also moves the intercept. With $s \in \{0, 1\}$ we have $\mathbf{1}[s = 0] = 1 - s$, so the one-hot pair collapses to

$$w_0\,\mathbf{1}[s = 0] + w_1\,\mathbf{1}[s = 1] \;=\; w_0 + (w_1 - w_0)\,s,$$

and the node reads

$$\operatorname{logit} P(\texttt{low} = 1) \;=\; (-\theta_0 + w_0) \;+\; w_{\text{age}}\,\texttt{age} \;+\; w_{\text{lwt}}\,\texttt{lwt} \;+\; (w_1 - w_0)\,s.$$

The first bracket is R's `(Intercept)` — it is $-\theta_0 + w_0$, not $-\theta_0$ — and the last is R's `smoke`.

```python
import statsmodels.formula.api as smf  # noqa: E402

from tramdag.transforms import ordinal_cutpoints  # noqa: E402

# the same formula as the R call above, and the same names in the output
res_bw = smf.logit("low ~ age + lwt + smoke", data=bw).fit(disp=False)

w = flow_bw.ls_coefficients()["low"]
with torch.no_grad():  # cutpoints are [-inf, theta_0, +inf]; take the finite one
    theta_0 = float(ordinal_cutpoints(flow_bw.nodes["low"].intercept(1))[0, 1])

rows_bw = [
    # -theta_0 + w_smoke[0]: the one-hot level-0 column is part of the intercept
    (
        "intercept",
        -theta_0 + float(w"smoke"),
        res_bw.params["Intercept"],
        R_GLM["intercept"],
    ),
    ("age", float(w"age"), res_bw.params["age"], R_GLM["age"]),
    ("lwt", float(w"lwt"), res_bw.params["lwt"], R_GLM["lwt"]),
    (
        "smoke",
        float(w"smoke" - w"smoke"),
        res_bw.params["smoke"],
        R_GLM["smoke"],
    ),
]
```

```
print(f"{'':<11}{'fit_classical':>14}{'sm.Logit':>10}{'R glm':>10}{'max|diff|':>11}")
for name, flow_value, sm_value, r_value in rows_bw:
    spread = max(abs(flow_value - sm_value), abs(flow_value - r_value))
    print(
        f"{name:<11}{flow_value:>14.4f}{sm_value:>10.4f}{r_value:>10.4f}{spread:>11.2e}"
    )
```

```
            fit_classical  sm.Logit     R glm  max|diff|
intercept          1.3682    1.3682    1.3682   6.46e-06
age               -0.0390   -0.0390   -0.0390   2.96e-07
lwt               -0.0121   -0.0121   -0.0121   1.32e-07
smoke              0.6708    0.6708    0.6708   4.16e-06
```

Three optimizers — L-BFGS on a transformation model, `statsmodels`' Newton solver, R's IRLS — land on the same maximum likelihood, because there is only one. The agreement is not only in the parameters: the log-likelihood of the `low` node and R's `logLik(m)` are the same number, and `flow.pmf` reproduces `glm`'s fitted probabilities row by row.

```
# nll() is the *mean* NLL per node; times n and negated it is the log-likelihood
ll_flow = -flow_bw.nll(bw)["low"] * len(bw)
p_flow = flow_bw.pmf(bw, node="low")[:, 1]
p_classical = res_bw.predict(bw).values
print(f"log-likelihood   flow {ll_flow:.6f}   R glm {R_LOGLIK:.6f}")
print(
    f"max |P_flow(low=1) - P_glm(low=1)| over {len(bw)} rows: "
    f"{np.abs(p_flow - p_classical).max():.2e}"
)
```

```
log-likelihood   flow -111.439677   R glm -111.439676
max |P_flow(low=1) - P_glm(low=1)| over 189 rows: 3.32e-06
```

1. Continuous case

Section 0 modelled `low`, the *dichotomized* birth weight. `birthwt` also carries the number it was cut from — `bwt`, the weight in grams — so the same three predictors can be fitted against a continuous outcome. That is a continuous outcome logistic transformation model, R's `tram::Colr` bwt ~ age + lwt + smoke No way to model this with glm.

```
R_COLR_BWT = {  # Colr(bwt ~ age + lwt + smoke, order = 21); R 4.2.3 / tram 1.0.4
    "age": -0.021971,
    "lwt": -0.010233,
    "smoke": 0.668698,
    "loglik": -1499.6678,
}

spec_bwt = {
    "age": ContinuousNode([SI(transform="affine")]),  # nuisance marginals
    "lwt": ContinuousNode([SI(transform="affine")]),
    "smoke": OrdinalNode(2),
    "bwt": ContinuousNode(
        [SI(n_coeffs=20), LS("age"), LS("lwt"), LS("smoke")]  # degree 21
    ),
}

flow_bwt = CausalFlowDAG(spec_bwt, seed=0)
flow_bwt.fit_classical(bw, max_iter=800)

w = flow_bwt.ls_coefficients()["bwt"]
```

```
fitted = {
    "age": w"age",
    "lwt": w"lwt",
    "smoke": w"smoke" - w"smoke",  # one-hot: read the difference
    "loglik": -flow_bwt.nll(bw)["bwt"] * len(bw),
}

print(f"{'':<9}{'age':>9}{'lwt':>9}{'smoke':>9}{'logLik':>11}")
for name, r in [("flow", fitted), ("R Colr", R_COLR_BWT)]:
    print(
        f"{name:<9}{r['age']:>9.4f}{r['lwt']:>9.4f}{r['smoke']:>9.4f}"
        f"{r['loglik']:>11.2f}"
    )
diff = {k: abs(fitted[k] - R_COLR_BWT[k]) for k in fitted}
print(
    f"{'|diff|':<9}{diff['age']:>9.4f}{diff['lwt']:>9.4f}{diff['smoke']:>9.4f}"
    f"{diff['loglik']:>11.2f}"
)
```

```
               age      lwt    smoke     logLik
flow       -0.0223  -0.0103   0.6641   -1500.55
R Colr     -0.0220  -0.0102   0.6687   -1499.67
|diff|      0.0004   0.0000   0.0046       0.88
```

The largest *absolute* difference is in `smoke`, but that only reflects its coefficient being some thirty times bigger than the others; as a fraction of the estimate it is the smallest of the three. Neither comparison is the useful one. A difference matters when it is large relative to the **sampling uncertainty** of the estimate — and that is a quantity we have not computed yet.

Standard errors and confidence intervals

The sampling distribution of the maximum likelihood estimator is

$$\hat{\theta} \;\dot{\sim}\; N(\theta, I^{-1})$$

where I is the Fisher information matrix. Since θ is unknown we evaluate I at $\hat{\theta}$, and read the standard errors off the diagonal of I^{-1}. For a contrast c, we read off $\sqrt{c^\top I^{-1} c}$.

The **observed** Fisher information is the Hessian of the *negative* log-likelihood at the MLE,

$$I_{ab} = \left.\frac{\partial^2(-\ell)}{\partial\theta_a\,\partial\theta_b}\right|_{\hat{\theta}},$$

which autograd gives directly: one backward pass produces the gradient with the graph retained, then one further backward pass per component produces a row of I.

Here I is singular, so I^{-1} does not exist. It is symmetric, so it has an orthonormal eigendecomposition, and the **pseudo-inverse** inverts only the directions that carry curvature:

$$I = \sum_{k=1}^{P} \lambda_k\, e_k e_k^\top \qquad\Longrightarrow\qquad I^{+} = \sum_{k:\;\lambda_k > \tau} \frac{1}{\lambda_k}\, e_k e_k^\top, \qquad \tau = 10^{-7}\,\lambda_{\max}.$$

Terms with $\lambda_k \le \tau$ are dropped rather than inverted. That is right for a contrast which avoids the flat subspace, and the `leak` column measures exactly how far it fails to:

$$\operatorname{leak}(c) = \max_{k\colon \lambda_k \le \tau} \left| c^\top e_k \right| .$$

Only interpret rows whose leak is close to zero. The choice of τ is not delicate here: the eigenvalues split into 13 below 4.4×10^{-8} and 11 above 0.27, a gap of seven orders of magnitude, so any threshold in between gives the same answer.

```
from scipy.stats import norm  # noqa: E402


def observed_information(flow, node, data):
    """Give one node's NLL Hessian, the gradient norm, and parameter offsets."""
    flow.double()  # second derivatives need float64
    names, params = zip(*flow.nodes[node].named_parameters(), strict=True)
    nll = -flow.node_log_prob(flow._tensorize(data))[node].sum()
    grad = torch.cat(
        [g.reshape(-1) for g in torch.autograd.grad(nll, params, create_graph=True)]
    )
    hess = torch.stack(
        [
            torch.cat(
                [
                    h.reshape(-1)
                    for h in torch.autograd.grad(grad[i], params, retain_graph=True)
                ]
            )
            for i in range(grad.numel())
        ]
    )
    offsets, at = {}, 0
    for name, p in zip(names, params, strict=True):
        offsets[name] = at
        at += p.numel()
    result = hess.detach().numpy(), float(grad.detach().norm()), offsets
    flow.float()  # back to the stored precision
    return result


def ls_conf_int(flow, node, data, level=0.95):
    """Give Wald intervals for the linear-shift coefficients of one node."""
    hess, grad_norm, offsets = observed_information(flow, node, data)
    evals, evecs = np.linalg.eigh(hess)
    rcond = 1e-7
    tau = rcond * np.abs(evals).max()  # one threshold, used for both
    flat = evecs[:, np.abs(evals) <= tau]  # the directions without curvature
    cov = np.linalg.pinv(hess, rcond=rcond)  # invert only the rest
    z = norm.ppf(0.5 + level / 2)

    rows = []
    for parent, w in flow.ls_coefficients()[node].items():
        c = np.zeros(hess.shape[0])
        key = next(n for n in offsets if n.startswith(f"shifts.{parent}"))
        if len(w) == 1:  # continuous parent: one weight
            c[offsets[key]], estimate, term = 1.0, w[0], parent
        else:  # ordinal parent: the contrast w[1] - w[0]
```

```
            c[offsets[key]], c[offsets[key] + 1] = -1.0, 1.0
            estimate, term = w[1] - w[0], f"{parent} (1 vs 0)"
        se = float(np.sqrt(c @ cov @ c))
        rows.append(
            {
                "term": term,
                "estimate": estimate,
                "se": se,
                "lower": estimate - z * se,
                "upper": estimate + z * se,
                # 0 = the contrast avoids the flat subspace, so the SE is real
                "leak": float(np.abs(c @ flat).max()) if flat.size else 0.0,
            }
        )
    diagnostics = {
        "n_params": hess.shape[0],
        "n_flat": flat.shape[1],
        "grad_norm": grad_norm,
    }
    return pd.DataFrame(rows).set_index("term"), diagnostics

table, diag = ls_conf_int(flow_bwt, "bwt", bw)
print(table.round(6).to_string())
print(
    f"\n{diag['n_params']} parameters, {diag['n_flat']} of them without curvature"
    f"   |grad| = {diag['grad_norm']:.1e}"
)
print("\nR: sqrt(diag(vcov(m))) -> age 0.025305   lwt 0.004116   smoke 0.260131")
print("   confint(m)           -> age [-0.07157, +0.02763]")
print("                           lwt [-0.01830, -0.00217]")
print("                           smoke [+0.15885, +1.17854]")
```

```
                estimate        se     lower     upper  leak
term
age            -0.022343  0.025264 -0.071861  0.027174   0.0
lwt            -0.010276  0.004113 -0.018337 -0.002216   0.0
smoke (1 vs 0)  0.664074  0.259577  0.155312  1.172835   0.0

24 parameters, 13 of them without curvature   |grad| = 3.4e-02

R: sqrt(diag(vcov(m))) -> age 0.025305   lwt 0.004116   smoke 0.260131
   confint(m)           -> age [-0.07157, +0.02763]
                           lwt [-0.01830, -0.00217]
                           smoke [+0.15885, +1.17854]
```

The standard errors reproduce `Colr`'s to three decimals. R's `confint()` on a `Colr` fit is Wald too.

Before adding the Confidence Intervals to the core code. We have to think harder on 1. $|\nabla \ell|$ is 1.7e-02 rather than 0, because `fit_classical` stops on flatness of the NLL rather than on the gradient norm, so the Hessian is taken a hair off the exact stationary point.

2. Since CI make sense only for estimable parameters, a `conf_int` returning one row per parameter would be mostly nonsense. We have to think harder on this.

1b. A bimodal DAG — the demo data

The VACA benchmark triangle (`x1 → x2 → x3 ← x1`, the demo notebook's bimodal SCM): `x1` is a two-component Gaussian mixture, `x2 = -x1 + N(0,1)`, `x3 = x1 + 0.25 x2 + N(0,1)`. We fit an

all-ls model: each node is a continuous logistic transformation model with a Bernstein baseline and linear shifts.

Note this is an *honest misspecification*: the DGP noise is Gaussian while the TRAM latent is logistic, so the all-ls model is not the true generator — but `fit_classical` still finds its exact MLE, which is the point here.

```
if False:

    def sample_vaca(n, seed):
        """Draw n rows from the VACA bimodal triangle SCM."""
        rng = np.random.default_rng(seed)
        mix, a, b = rng.uniform(size=n), rng.normal(size=n), rng.normal(size=n)
        x1 = np.where(mix < 0.5, -2.0 + np.sqrt(1.5) * a, 1.5 + b)
        x2 = -x1 + rng.normal(size=n)
        x3 = x1 + 0.25 * x2 + rng.normal(size=n)
        return pd.DataFrame({"x1": x1, "x2": x2, "x3": x3})

    df = sample_vaca(1000, seed=42)
    df.to_csv(NB_DATA / "vaca.csv", index=False)
```

The R you can copy-paste

`tram::Colr` fits the same model family — a continuous outcome logistic transformation model with a Bernstein baseline. The cell above already wrote `notebooks/data/vaca.csv`, so R reads the identical rows the flow is fitted on.

The two libraries **count the basis differently**, and the comparison is only meaningful at the same polynomial degree. The flow's `n_coeffs` unconstrained coefficients become `n_coeffs + 2` control points, that is a Bernstein polynomial of degree `n_coeffs + 1` (`order = n + 1` in `transforms.py`). So `N_COEFFS = 20` below is tram's `order = 21`, **not** `order = 19`.

```
library(tram)
d <- read.csv("notebooks/data/vaca.csv")

m2 <- Colr(x2 ~ x1, data = d, order = 21)
coef(m2)     # -> x1 1.800042
logLik(m2)   # -> -1401.213542

m3 <- Colr(x3 ~ x1 + x2, data = d, order = 21)
coef(m3)     # -> x1 -1.777289   x2 -0.455282
logLik(m3)   # -> -1411.901958
```

The coefficients need **no sign flip**: `Colr` reports the shift on the same side as the flow.

```
R_COLR = {  # Colr(..., order = 21) on vaca.csv; R 4.2.3 / tram 1.0.4
    "x2": {"coef": {"x1": 1.800042}, "loglik": -1401.213542},
    "x3": {"coef": {"x1": -1.777289, "x2": -0.455282}, "loglik": -1411.901958},
}

df = pd.read_csv(NB_DATA / "vaca.csv")
# n_coeffs is stated rather than left to the default, because Section 1 compares
# against R and the two libraries count the basis differently: the flow's
# n_coeffs unconstrained coefficients become n_coeffs + 2 control points, i.e. a
# Bernstein polynomial of degree n_coeffs + 1 (see `order = n + 1` in
# transforms.py). So n_coeffs=20 here is tram's `order = 21`, not `order = 19`.
N_COEFFS = 20  # -> Bernstein degree 21

spec_vaca = {
```

```
    "x1": ContinuousNode([SI(n_coeffs=N_COEFFS)]),
    "x2": ContinuousNode([SI(n_coeffs=N_COEFFS), LS("x1")]),
    "x3": ContinuousNode([SI(n_coeffs=N_COEFFS), LS("x1"), LS("x2")]),
}

flow_c = CausalFlowDAG(spec_vaca, seed=0)
rep = flow_c.fit_classical(df)  # logs iters / NLL / time
print(f"\n{'':<12}{'fit_classical':>14}{'R Colr':>12}{'|diff|':>10}")
loglik = {k: -v * len(df) for k, v in flow_c.nll(df).items()}  # nll() is a mean
for node, parents in flow_c.ls_coefficients().items():
    for p, w in parents.items():
        c, r = w[0], R_COLRnode[p]
        print(f"{p + ' -> ' + node:<12}{c:>14.4f}{r:>12.4f}{abs(c - r):>10.4f}")
    c, r = loglik[node], R_COLRnode
    print(f"{'logLik ' + node:<12}{c:>14.4f}{r:>12.4f}{abs(c - r):>10.4f}")
```

```
             fit_classical      R Colr    |diff|
x1 -> x2            1.7998      1.8000    0.0002
logLik x2       -1401.8717  -1401.2135    0.6581
x1 -> x3            -1.7806     -1.7773    0.0033
x2 -> x3            -0.4550     -0.4553    0.0003
logLik x3       -1410.9678  -1411.9020    0.9341
```

We note that the coefficients are very close to the R Colr coefficients. ### Why this one is not an exact-MLE check

The coefficients agree to about 0.002, and not to 1e-8 as in Sections 0 and 2. The two libraries implement `h` differently, so neither result is the more correct one. The largest difference is where the basis sits: the flow puts it on the 5%/95% quantiles, and `h` is a straight line outside them. This leaves 10% of the rows in the tails, while `Colr` uses the full range of the data.

The shift coefficients agree to about 0.1% regardless, which is why the ordered logit above finds the same two numbers with no Bernstein basis.

Classical vs Adam — same optimum. A converged Adam fit (no early stopping) reaches the same coefficients to ~1e-2. The classical fit's guarantees are *determinism* and the *exact MLE*; raw speed is model-dependent — it wins clearly on ordinal-outcome models (Section 2), but on this Bernstein-heavy continuous DAG L-BFGS has to grind through the flat polynomial valleys, so wall-clock here is comparable to Adam rather than faster:

```
flow_a = CausalFlowDAG(spec_vaca, seed=0)
t0 = time.perf_counter()
# the pre-0.4 fit(schedule="plateau", freeze_patience=) recipe, now a callback
sched = PerNodePlateau(patience=15, freeze=60)
flow_a.fit(
    df,
    epochs=2000,
    batch_size=4096,
    validation_data=df,
    optimizer=per_node_adam(flow_a, lr=1e-1),
    callbacks=sched,
)
t_adam = time.perf_counter() - t0

coef_c, coef_a = flow_c.ls_coefficients(), flow_a.ls_coefficients()
print(f"{'coef':<10}{'classical':>12}{'adam':>12}{'|diff|':>10}")
for node, p in [("x2", "x1"), ("x3", "x1"), ("x3", "x2")]:
    c, aw = coef_cnode[0], coef_anode[0]
```

```
    print(f"{p}->{node:<6}{c:>12.4f}{aw:>12.4f}{abs(c - aw):>10.4f}")
print(f"\nclassical {rep['seconds']:.2f}s  vs  adam {t_adam:.1f}s")
```

```
coef         classical        adam    |diff|
x1->x2          1.7998      1.7944    0.0055
x1->x3         -1.7806     -1.7663    0.0143
x2->x3         -0.4550     -0.4474    0.0076

classical 4.58s  vs  adam 4.2s
```

2. Ordinal case (treatment effect)

For an **ordinal** outcome the classical model is the ordered-logit, which `statsmodels.OrderedModel` fits — so here the equivalence is checkable in Python. The all-`ls` stroke DAG (the synthetic `magic-mrclean/ls` cohort):

```
obs = pd.read_csv(DATA / "magic-mrclean" / "ls" / "obs.csv")

spec_stroke = {
    "Age": ContinuousNode(),
    "mRS_pre": OrdinalNode(6, [LS("Age")]),
    "NIHSSa": ContinuousNode([LS("Age"), LS("mRS_pre")]),
    "T": OrdinalNode(2, [LS("Age"), LS("mRS_pre"), LS("NIHSSa")]),
    "mRS_3m": OrdinalNode(7, [LS("Age"), LS("mRS_pre"), LS("NIHSSa"), LS("T")]),
}

flow_s = CausalFlowDAG(spec_stroke, seed=0)  # the seed validate_ls.py checks
flow_s.fit_classical(obs)
```

```
{'converged': False,
 'n_iter': 400,
 'final_nll': 10.306837499958505,
 'grad_norm': 0.017277114988926635,
 'seconds': 5.38655338600006,
 'coefficients': {'mRS_pre': {'Age': array([0.05768183])},
  'NIHSSa': {'Age': array([-0.05582796]),
   'mRS_pre': array([ 1.73456272,  1.32279731,  1.14820555,  1.40077438,  0.31263827,
          -0.23347872])},
  'T': {'Age': array([-0.03928215]),
   'mRS_pre': array([0.67340049, 0.79612263, 0.45463112, 0.78673051, 0.15841777,
          0.26415468]),
   'NIHSSa': array([-0.0137474])},
  'mRS_3m': {'Age': array([0.05374448]),
   'mRS_pre': array([-1.362594  , -0.95564466, -0.77616902, -0.57278162,  1.14801943,
          0.41579485]),
   'NIHSSa': array([0.16336759]),
   'T': array([-0.70514631, -1.63106338])}}}
```

Using a classical fit

This can be done quite easily with statsmodels just have to hand over the design matrix `flow.design_matrix(..., drop_first=True)`

```
design = flow_s.design_matrix(obs, "mRS_3m", drop_first=True)
res = OrderedModel(obs["mRS_3m"].astype(int), design, distr="logit").fit(
    method="bfgs", disp=False
```

```
)

fitted = flow_s.ls_coefficients()["mRS_3m"]
rows = [
    ("Age", float(fitted"Age"), res.params["Age"]),
    ("NIHSSa", float(fitted"NIHSSa"), res.params["NIHSSa"]),
    ("T (1 vs 0)", float(fitted"T" - fitted"T"), res.params["T[1]"]),
]
print(f"{'coefficient':<14}{'fit_classical':>14}{'statsmodels':>13}{'|diff|':>9}")
for name, a_, b_ in rows:
    print(f"{name:<14}{a_:>14.4f}{b_:>13.4f}{abs(a_ - b_):>9.4f}")
```

```
coefficient    fit_classical  statsmodels   |diff|
Age                   0.0537       0.0526   0.0012
NIHSSa                0.1634       0.1630   0.0004
T (1 vs 0)           -0.9259      -0.9424   0.0165
```

Standard errors, and what is actually weakly identified

The helper from Section 1 needs no change: give it the node, and it builds the contrast for each ordinal parent. `statsmodels` reports the same quantities, so every number below has an external check.

```
table_s, diag_s = ls_conf_int(flow_s, "mRS_3m", obs)
ci_sm = res.conf_int()
print(
    f"{'':<12}{'flow SE':>9}{'sm SE':>8}{'|est|/SE':>10}   "
    f"{'flow 95%':>18}{'statsmodels 95%':>20}"
)
for term, key in [("Age", "Age"), ("NIHSSa", "NIHSSa"), ("T (1 vs 0)", "T[1]")]:
    r = table_s.loc[term]
    print(
        f"{term:<12}{r['se']:>9.4f}{res.bse[key]:>8.4f}"
        f"{abs(r['estimate']) / r['se']:>10.2f}   "
        f"[{r['lower']:+.3f}, {r['upper']:+.3f}]  "
        f"[{ci_sm.loc[key, 0]:+.3f}, {ci_sm.loc[key, 1]:+.3f}]"
    )
print(
    f"\n{diag_s['n_params']} parameters, {diag_s['n_flat']} without curvature"
    " — one per one-hot parent, mRS_pre and T"
)
```

```
              flow SE   sm SE  |est|/SE             flow 95%     statsmodels 95%
Age            0.0050  0.0050     10.83   [+0.044, +0.063]  [+0.043, +0.062]
NIHSSa         0.0090  0.0090     18.08   [+0.146, +0.181]  [+0.145, +0.181]
T (1 vs 0)     0.1408  0.1407      6.58   [-1.202, -0.650]  [-1.218, -0.667]

16 parameters, 2 without curvature — one per one-hot parent, mRS_pre and T
```

The standard errors agree with `statsmodels` to four decimals, and the treatment effect is **not** the uncertain one: at 6.6 standard errors from zero it is the best determined coefficient in the table.

To find the weak one, look at every `mRS_pre` level instead of only the first, and print how many rows carry it.

```
hess_s, _, offsets_s = observed_information(flow_s, "mRS_3m", obs)
cov_s = np.linalg.pinv(hess_s, rcond=1e-7)
at = offsets_s[next(n for n in offsets_s if n.startswith("shifts.mRS_pre"))]
```

```
w_pre = flow_s.ls_coefficients()"mRS_3m"
counts = obs["mRS_pre"].value_counts()

print(f"{'level':>6}{'rows':>7}{'estimate':>10}{'SE':>9}{'|est|/SE':>10}   95% CI")
for level in range(1, 6):
    c = np.zeros(hess_s.shape[0])
    c[at], c[at + level] = -1.0, 1.0  # w[level] - w[0]
    est = w_pre[level] - w_pre[0]
    se = float(np.sqrt(c @ cov_s @ c))
    print(
        f"{level:>6}{counts.get(level, 0):>7}{est:>10.4f}{se:>9.4f}"
        f"{abs(est) / se:>10.2f}   [{est - 1.96 * se:+.3f}, {est + 1.96 * se:+.3f}]"
    )
```

level	rows	estimate	SE	|est|/SE	95% CI
1	273	0.4069	0.1340	3.04	[+0.144, +0.670]
2	166	0.5864	0.1632	3.59	[+0.267, +0.906]
3	107	0.7898	0.1922	4.11	[+0.413, +1.166]
4	41	2.5106	0.4158	6.04	[+1.696, +3.326]
5	7	1.7784	0.8762	2.03	[+0.061, +3.496]

There it is. Level 5 is carried by **7 of 1275 rows**, and its standard error is 0.88 — six times that of level 1, with an interval from +0.10 to +3.55 that only just excludes zero. That is what a weakly identified coefficient looks like, and the cause is visible in the second column: too few rows.

This also explains the `|diff|` column above. The gap between the flow and `statsmodels` is not evidence about identification, it is the optimizer stopping while the likelihood is still flat. The displacement it causes, $c^\top I^{+} \nabla \ell$, is +1.4e-03 for `Age` and +7.4e-03 for `T` — which is exactly the difference each column shows. `Age` looks worse only because its standard error is 0.005, so the same numerical slack is 28% of an SE for `Age` and 5% for `T`.

`experiments/misc/validate_ls.py` runs this comparison as a checked experiment, and adds R (`MASS::polr` / `tram`) and the treatment effect. Its docstring records the same finding about `mRS_pre` level 5.

3. Warm-start handoff: classical fit → further training

`fit_classical` leaves the model at the MLE in float32, ready for any normal operation. Two things to verify:

1. the float64→float32 round-trip didn't move the coefficients, and
2. continuing with `fit()` from the classical solution *stays put* — confirming it really is the optimum (and showing the classical fit as a fast, principled initialization for further or richer training).

```
before = {k: v.copy() for k, v in flow_s.ls_coefficients()["mRS_3m"].items()}

# continue training from the classical solution with a gentle Adam phase
flow_s.fit(obs, epochs=300, learning_rate=1e-3, batch_size=256)
after = flow_s.ls_coefficients()["mRS_3m"]

print("coefficient drift after 300 more Adam epochs from the classical MLE:")
for p in ["Age", "NIHSSa", "T"]:
    d = float(np.abs(after[p] - before[p]).max())
    print(f"  {p:<8} max|Δ| = {d:.4f}")
print("\n-> small drift = the classical fit was already at the optimum;")
print("   fit_classical is a valid warm start for continued / flexible training.")
```

```
coefficient drift after 300 more Adam epochs from the classical MLE:
```

```
  Age      max|Δ| = 0.0002
  NIHSSa   max|Δ| = 0.0003
  T        max|Δ| = 0.0238

-> small drift = the classical fit was already at the optimum;
   fit_classical is a valid warm start for continued / flexible training.
```

TRAM-DAG in 5 minutes — one causal model, all three rungs of Pearl's ladder

TRAM-DAGs (Sick & Dürr, CLeaR 2025) are *interpretable neural causal models*: one normalizing flow, wired exactly like the **adjacency matrix of your causal DAG**. The flow transforms each variable conditional on its parents. Fit it **once** on observational data. Then you can

1. **L1** sample and score the observational distribution,
2. **L2** answer interventional queries (`do(...)`) by graph mutilation,
3. **L3** compute **individual counterfactuals** ("what would have happened to *this* unit?") with Pearl's abduction–action–prediction.

This demo uses the first benchmark of the paper: a 3-variable SCM with a **bimodal** source. On this example, a default causal normalizing flow visibly fails (paper, Fig. 4) and TRAM-DAG does not. The ground truth is known analytically. Thus every claim below is *checked*, not asserted.

The demo runs on CPU; a Colab **GPU runtime** (Runtime → Change runtime type → any GPU, for example the free T4) speeds up training.

```
import importlib.util
import subprocess
import sys
import time

if importlib.util.find_spec("tramdag") is None:  # Colab: install from PyPI
    subprocess.check_call([sys.executable, "-m", "pip", "install", "-q", "tramdag"])
    # to track active development instead of the latest release, use:
    #   pip install "git+https://github.com/tensorchiefs/tramdag.git@main"

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import scipy.stats as st  # for KDE plots only (preinstalled on Colab)
import torch

from tramdag import CausalFlowDAG, ContinuousNode, I
from tramdag.callbacks import PerNodePlateau, per_node_adam

DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
plt.rcParams["figure.dpi"] = 110
print(f"torch {torch.__version__}  device: {DEVICE}")
```

```
torch 2.13.0+cu130  device: cpu
```

1. The challenge: a bimodal structural causal model

The DGP (Sánchez-Martín et al. 2022, App. E.1, and TRAM-DAG paper App. C.1):

$$
\begin{aligned}
x_1 &\sim \tfrac{1}{2}\,\mathcal{N}(-2,\,1.5) + \tfrac{1}{2}\,\mathcal{N}(1.5,\,1) \quad \text{(bimodal source)} \\
x_2 &= -x_1 + \mathcal{N}(0,1) \\
x_3 &= x_1 + 0.25\,x_2 + \mathcal{N}(0,1)
\end{aligned}
$$

The DAG is $x_1 \to x_2$, $x_1 \to x_3$, $x_2 \to x_3$.

```
# The DGP, written out here so this notebook needs nothing but tramdag.
# draw_latents/simulate are separate on purpose: keeping the noise lets us
# intervene on the SAME individuals later, which is what makes the
# counterfactual check in section 5 possible.


def draw_latents(n, rng):
    """Draw every noise variable of the SCM, n rows each."""
    return {
        "x1_mix": rng.uniform(size=n),  # which mixture component
        "x1_a": rng.normal(size=n),  # N(-2, 1.5) branch
        "x1_b": rng.normal(size=n),  # N(1.5, 1) branch
        "x2": rng.normal(size=n),
        "x3": rng.normal(size=n),
    }


def simulate(latents, do=None):
    """Run the SCM forward; a variable named in `do` is clamped instead."""
    do = do or {}
    n = len(latents["x2"])

    if "x1" in do:
        x1 = np.full(n, float(do["x1"]))
    else:
        x1 = np.where(
            latents["x1_mix"] < 0.5,
            -2.0 + np.sqrt(1.5) * latents["x1_a"],
            1.5 + latents["x1_b"],
        )

    if "x2" in do:
        x2 = np.full(n, float(do["x2"]))
    else:
        x2 = -x1 + latents["x2"]

    if "x3" in do:
        x3 = np.full(n, float(do["x3"]))
    else:
        x3 = x1 + 0.25 * x2 + latents["x3"]

    return pd.DataFrame({"x1": x1, "x2": x2, "x3": x3})


def sample_dgp(n, seed, do=None):
    """Draw n fresh rows from the SCM, optionally under an intervention."""
    return simulate(draw_latents(n, np.random.default_rng(seed)), do)
```

```
df = sample_dgp(50_000, seed=43)
train, val = df.iloc[:45_000], df.iloc[45_000:]

fig, axes = plt.subplots(1, 3, figsize=(11, 3))
for ax, c in zip(axes, df.columns, strict=True):
    ax.hist(df[c], bins=80, density=True, alpha=0.7)
    ax.set_title(f"observed ${c[0]}_{c[1]}$")
fig.suptitle("50,000 observational samples — note the bimodal $x_1$")
fig.tight_layout()
plt.show()
```

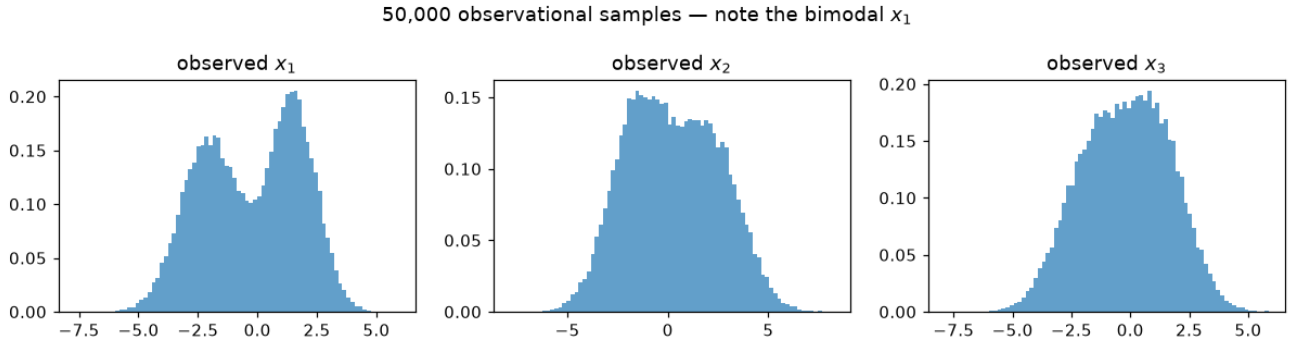


Figure 2: png

2. Fit the TRAM-DAG — the spec *is* the DAG

Each node gets a monotone Bernstein transform. The terms say how parents enter (`I(...)` = the parents control the transform parameters, for maximal flexibility). Training maximizes the exact joint likelihood with one Adam. `fit` is a plain loop with Keras-shaped extras: `validation_data=` gives a per-epoch validation NLL in `flow.history["val"]`, `verbose=` prints progress. Strategy attaches through `callbacks=` — here the shipped self-stopping recipe: `per_node_adam` gives every node its own parameter group, and `PerNodePlateau` decays each node's rate on its own validation NLL and freezes it once flat; the fit ends when the last node froze. (Section 6 shows the hand-rolled alternative: torch's `ReduceLROnPlateau` through the same hook.)

```
spec = {
    "x1": ContinuousNode(),  # source
    "x2": ContinuousNode(I("x1")),
    "x3": ContinuousNode(I("x1", "x2")),
}


def early_stopping(patience):
    """ReduceLROnPlateau on fit's own validation NLL, stop after `patience` flat epochs.

    The stop half ships as `tramdag.callbacks.EarlyStopping`; hand-rolled here
    to show the bare-callable hook carrying a torch scheduler.
    """
    log = {"val": [], "sched": None}

    def cb(f, epoch, opt):
        if log["sched"] is None:
            # built lazily: the optimizer only exists once fit() runs
```

```
            log["sched"] = torch.optim.lr_scheduler.ReduceLROnPlateau(
                opt, factor=0.3, patience=patience // 3
            )
        nll = sum(f.history"val".values())  # fit computed it
        log["val"].append(nll)
        log["sched"].step(nll)
        return len(log["val"]) - int(np.argmin(log["val"])) > patience

    return cb


torch.manual_seed(0)
flow = CausalFlowDAG(spec, device=DEVICE)
lr_trace = []  # x1's learning rate per epoch, for the plot below

sched = PerNodePlateau(patience=10, freeze=40)
t0 = time.perf_counter()
flow.fit(
    train,
    epochs=400,
    batch_size=4096,
    validation_data=val,
    verbose=25,
    optimizer=per_node_adam(flow, lr=1e-1),
    callbacks=[
        lambda f, e, opt: lr_trace.append(opt.param_groups0),
        sched,
    ],
)
t_fit = time.perf_counter() - t0
print(
    f"\nfitted on {DEVICE} in {t_fit:.1f}s "
    f"({len(flow.history['val'])} epochs, then every node had frozen)"
)
```

```
epoch 25/400  train 4.9168  val 4.9191


epoch 50/400  train 4.9132  val 4.9178


epoch 75/400  train 4.9115  val 4.9149


epoch 100/400  train 4.9114  val 4.9149

fitted on cpu in 28.4s (100 epochs, then every node had frozen)
```

`fit` records the per-node train AND validation NLL (`history["val"]` — `validation_data=` did that); the one-line callback kept x1's learning rate. Watch the per-node plateau rule step that rate down until the node freezes (rate 0):

```
ep = np.arange(1, len(flow.history["val"]) + 1)
tot_tr = np.array([sum(d.values()) for d in flow.history["train"]])
tot_va = np.array([sum(d.values()) for d in flow.history["val"]])
fig, ax = plt.subplots(figsize=(7.5, 3.6))
ax.plot(ep, tot_tr, label="train NLL (total)")
```

```
ax.plot(ep, tot_va, label="val NLL (total)")
for e in np.nonzero(np.diff(lr_trace) < 0)[0] + 1:
    ax.axvline(e, ls="--", lw=1, color="gray")
ax.set_ylim(tot_va.min() - 0.02, tot_va.min() + 0.6)  # zoom past the initial drop
ax.set_xlabel("epoch"), ax.set_ylabel("NLL"), ax.legend()
ax.set_title("training curve — dashed: the plateau rule lowered the learning rate")
fig.tight_layout()
plt.show()
```

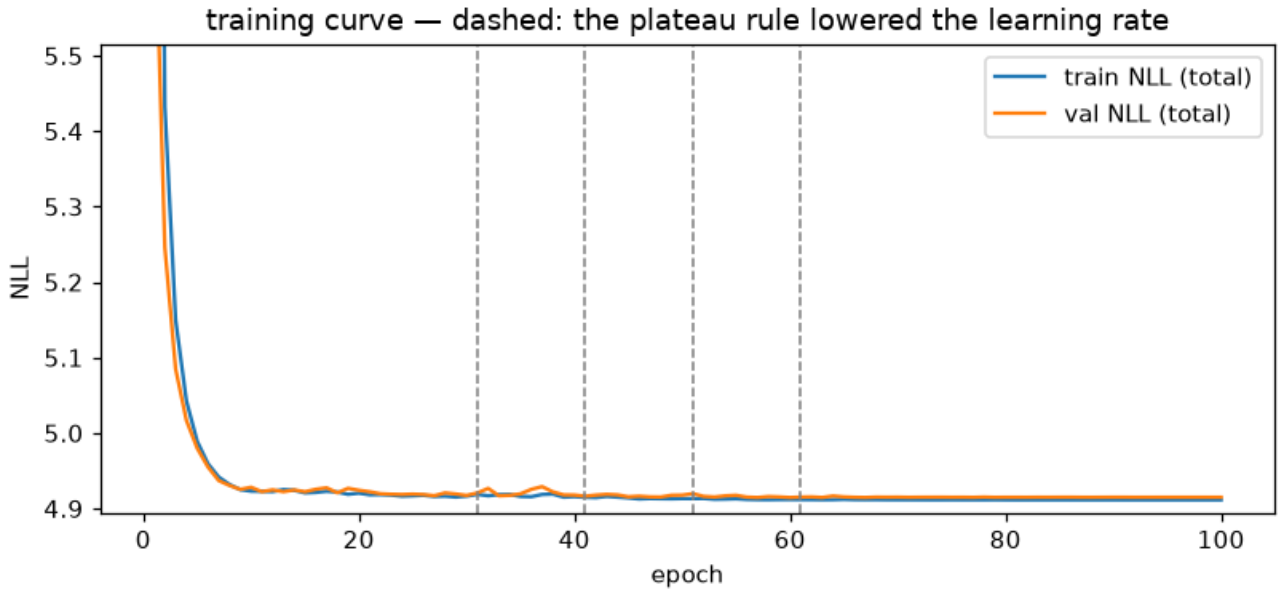


Figure 3: png

3. Rung 1 — does it actually fit? (the plot the CNF baseline fails)

```
samp = flow.sample(len(df), seed=1)
cols = list(df.columns)
fig, axes = plt.subplots(3, 3, figsize=(9, 8.5))
for i, ci in enumerate(cols):
    for j, cj in enumerate(cols):
        ax = axesi
        if i == j:
            bins = np.linspace(df[ci].quantile(0.001), df[ci].quantile(0.999), 70)
            # Plot DGP histogram
            ax.hist(df[ci], bins=bins, density=True, alpha=0.5, label="DGP")
            # KDE for TRAM-DAG samples
            kde = st.gaussian_kde(samp[ci])
            x_eval = np.linspace(bins[0], bins[-1], 300)
            ax.plot(x_eval, kde(x_eval), color="C3", lw=1.8, label="TRAM-DAG KDE")
        else:
            ax.scatter(dfcj, dfci, s=2, alpha=0.3)
            ax.scatter(sampcj, sampci, s=2, alpha=0.3, color="C3")
        if i == 2:
            ax.set_xlabel(cj)
        if j == 0:
            ax.set_ylabel(ci)
axes0.legend(fontsize=8)
```

```
fig.suptitle("L1: observational joint — DGP (blue) vs fitted TRAM-DAG (red, KDE)")
fig.tight_layout()
plt.show()
```

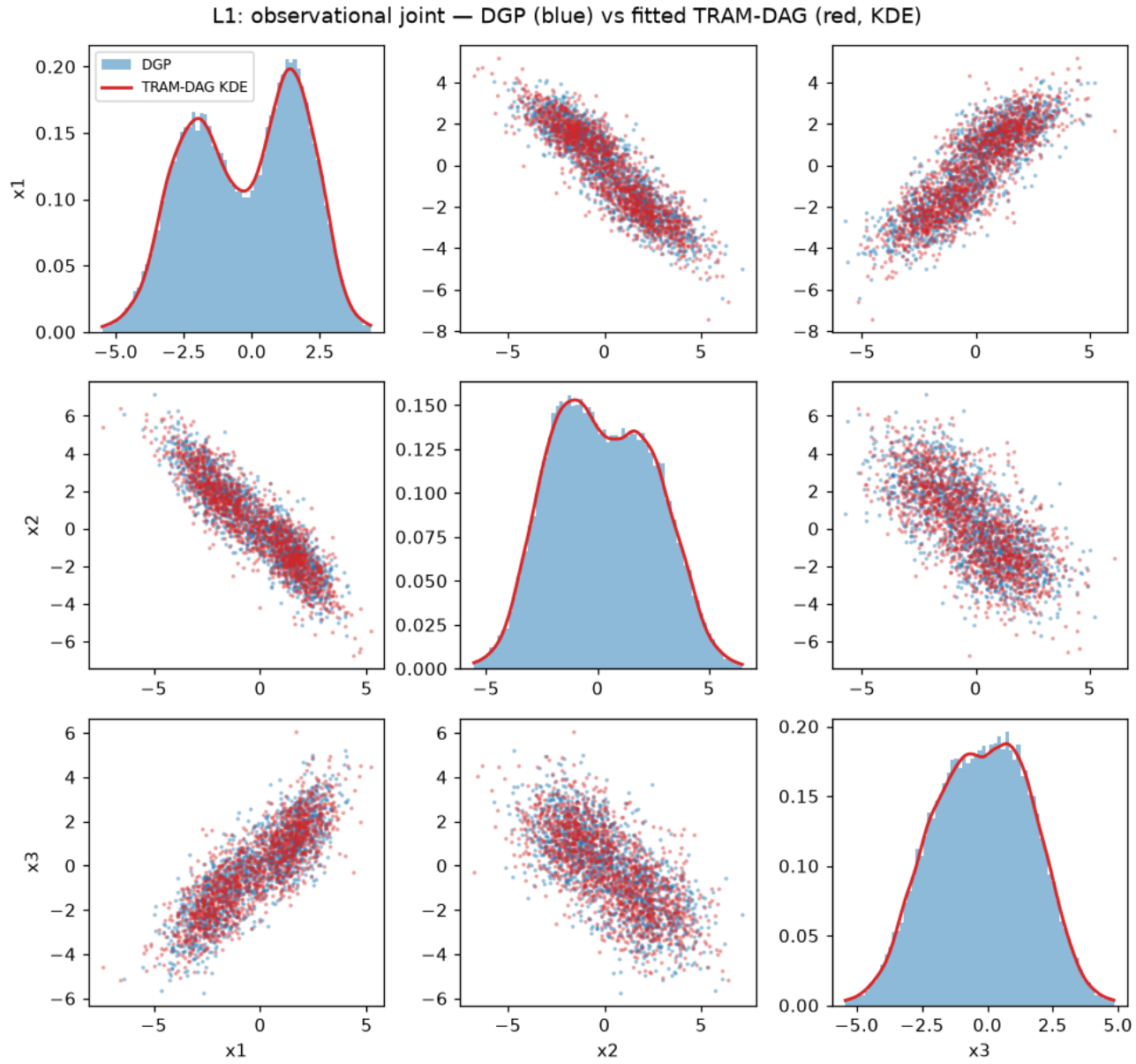


Figure 4: png

4. Rung 2 — interventions: `do(x2 = a)`

Graph mutilation: clamp x_2, cut its incoming edge, resample. Under the DGP, $x_3 \,|\, do(x_2{=}a) = x_1 + 0.25a + \mathcal{N}(0,1)$. Thus $\mathbb{E}[x_3] = -0.25 + 0.25a$ **analytically**. This exact target makes an error easy to see.

```
fig, axes = plt.subplots(1, 3, figsize=(11, 3.2), sharey=True)
print("E[x3 | do(x2=a)]:   analytic    TRAM-DAG")
for ax, a in zip(axes, (-3.0, -1.0, 0.0), strict=True):
    truth = sample_dgp(50_000, seed=543, do={"x2": a})
```

```
    fl = flow.sample(50_000, do={"x2": a}, seed=2)
    bins = np.linspace(truth["x3"].quantile(0.001), truth["x3"].quantile(0.999), 70)
    # DGP histogram
    ax.hist(truth["x3"], bins=bins, density=True, alpha=0.5, label="DGP")
    # Flow density: KDE for a smoother estimate
    kde = st.gaussian_kde(fl["x3"])
    x_eval = np.linspace(bins[0], bins[-1], 300)
    ax.plot(x_eval, kde(x_eval), color="C3", lw=1.8, label="TRAM-DAG density")
    ax.set_title(f"$p(x_3 \\mid do(x_2={a:+.0f}))$")
    print(
        f"   a = {a:+.0f}:          {-0.25 + 0.25 * a:+.3f}      {fl['x3'].mean():+.3f}"
    )
axes[0].legend()
fig.tight_layout()
plt.show()
```

```
E[x3 | do(x2=a)]:   analytic    TRAM-DAG

   a = -3:          -1.000      -1.013

   a = -1:          -0.500      -0.505

   a = +0:          -0.250      -0.263
```

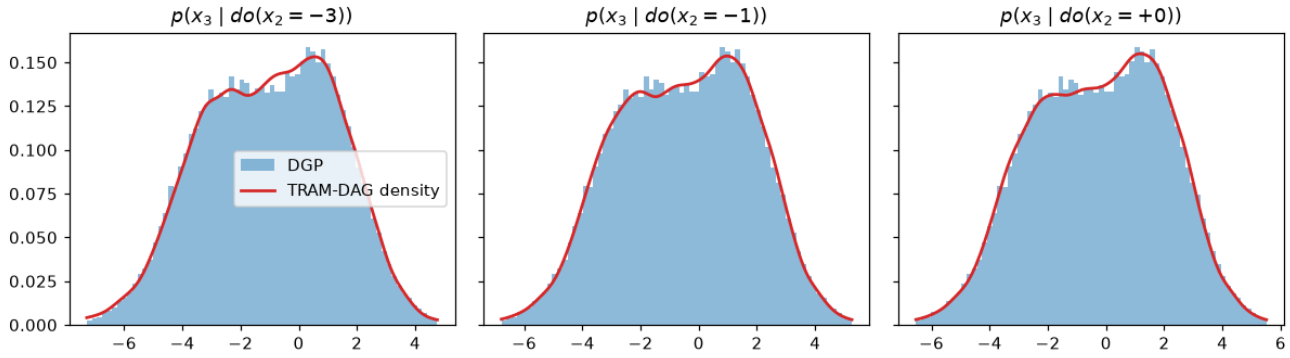


Figure 5: png

5. Rung 3 — counterfactuals for held-out individuals

Take 1,000 **held-out** individuals. Step 1 (*abduction*): invert the flow to recover the latent noise u of each individual. This noise is everything about them that the model does not attribute to their parents. Sanity check: when we push u back through the flow, it must reproduce the observed data *exactly*.

Step 2+3 (*action* + *prediction*): rerun history with $do(x_1 = 0)$, that is, the same u on a mutilated graph. Because the DGP is fully continuous, the **true** individual counterfactuals are known (the simulator keeps its noise). Thus we can score the flow *per individual*. This is the strictest test in causal inference, and it is impossible with real data.

```
lat = draw_latents(1_000, np.random.default_rng(7))
factual = simulate(lat)  # the same noise on both sides ...
cf_true = simulate(lat, do={"x1": 0.0})  # ... so these are true counterfactuals
```

```
u = flow.abduct(factual)
recon = flow.sample(u=u)
err = np.abs(recon.to_numpy() - factual.to_numpy()).max()
print(f"abduction -> reconstruction: max |error| = {err:.2e}  (exact recovery)")

cf_flow = flow.sample(do={"x1": 0.0}, u=u)
fig, axes = plt.subplots(1, 2, figsize=(9, 3.6))
for ax, c in zip(axes, ["x2", "x3"], strict=True):
    ax.scatter(cf_true[c], cf_flow[c], s=4, alpha=0.4)
    lims = [cf_true[c].min(), cf_true[c].max()]
    ax.plot(lims, lims, "k--", lw=1)
    r = np.corrcoef(cf_true[c], cf_flow[c])[0, 1]
    ax.set_title(
        f"counterfactual ${c[0]}_{c[1]}$ under $do(x_1{{=}}0)$   (r = {r:.4f})"
    )
    ax.set_xlabel("true (DGP, shared noise)"), ax.set_ylabel("TRAM-DAG")
fig.suptitle("L3: individual counterfactuals, scored unit by unit")
fig.tight_layout()
plt.show()
```

```
abduction -> reconstruction: max |error| = 5.02e-06  (exact recovery)
```

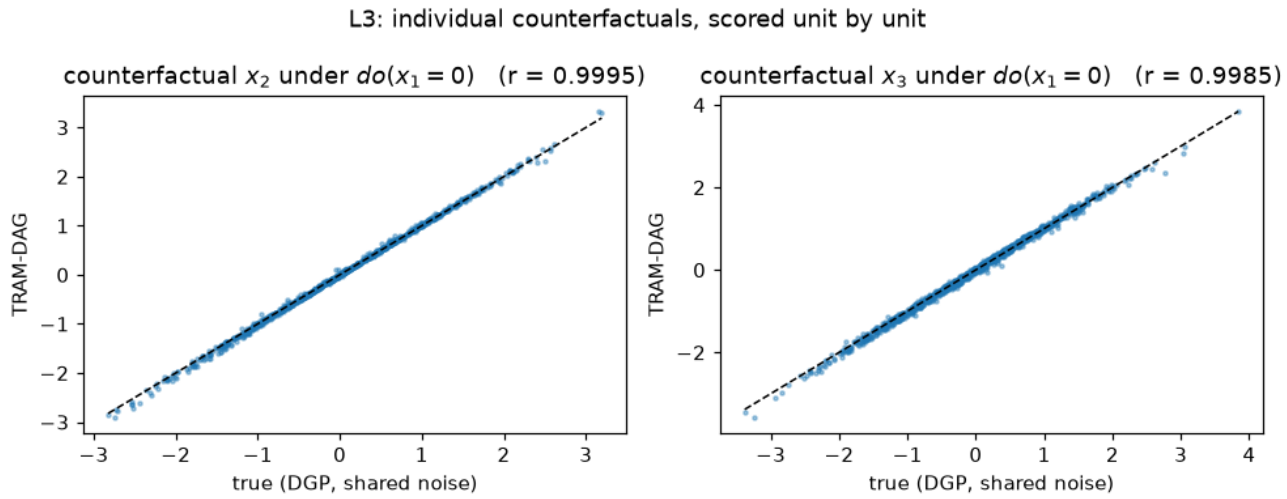


Figure 6: png

6. Swapping the transform: Bernstein vs spline vs affine

Each continuous node owns a **monotone 1-D transform**. That transform holds all of the distributional flexibility. One constructor argument switches it: `"bernstein"` (default, TRAM-faithful polynomial), `"spline"` (monotone rational-quadratic, the neural-spline-flow building block), or `"affine"` (location–scale only, which forces every node-conditional to be a logistic, in effect a classical GLM). Same DAG, same protocol (the learning rate is tuned per family), three model families:

```
def make_spec(transform):
    return {
        "x1": ContinuousNode([I(transform=transform)]),
        "x2": ContinuousNode(I("x1", transform=transform)),
        "x3": ContinuousNode(I("x1", "x2", transform=transform)),
```

```
    }


fits = {"bernstein": flow}  # already trained above
for tr in ["spline", "affine"]:
    torch.manual_seed(0)
    f = CausalFlowDAG(make_spec(tr), device=DEVICE)
    cb = early_stopping(patience=30)
    f.fit(
        train,
        epochs=400,
        learning_rate=1e-2,
        batch_size=4096,
        validation_data=val,
        callbacks=cb,
    )
    fits[tr] = f
print("held-out NLL (lower is better):")
for tr, f in fits.items():
    print(f"  {tr:9s}: {sum(f.nll(val).values()):.4f}")
```

```
held-out NLL (lower is better):
  bernstein: 4.9149
  spline   : 5.6335
  affine   : 5.0393
```

```
bins = np.linspace(df["x1"].quantile(0.001), df["x1"].quantile(0.999), 70)
fig, ax = plt.subplots(figsize=(7, 3.4))
ax.hist(df["x1"], bins=bins, density=True, alpha=0.4, color="gray", label="data")
for tr, color in [("bernstein", "C3"), ("spline", "C0"), ("affine", "C2")]:
    ax.hist(
        fits[tr].sample(len(df), seed=3)["x1"],
        bins=bins,
        density=True,
        histtype="step",
        lw=1.8,
        color=color,
        label=tr,
    )
ax.set_title("the affine (GLM-like) transform cannot bend into two modes")
ax.set_xlabel("$x_1$"), ax.legend()
fig.tight_layout()
plt.show()
```

Reading the table: **affine** pays exactly where you expect it to pay. A location–scale transform *cannot* produce a bimodal x_1 (the same failure mode as the inflexible CNF in Fig. 4 of the paper). The **RQ-spline** is expressive enough *in principle*, and its gap is **structural, not an optimization failure** (same result for 8–32 bins, lr 0.01–0.1, up to 2000 epochs): zuko's spline extrapolates outside its `[-5, 5]` domain with a *fixed* slope regardless of the fitted parameters, so the ~10% of data beyond the 5%/95% pre-scaling range is misweighted whenever the true tail slope differs. Bernstein's linear extrapolation follows the boundary derivative instead — which is why it is the TRAM-faithful default. You can swap the transform per node at any time with `I(..., transform="spline", bins=16)`.

What you just saw

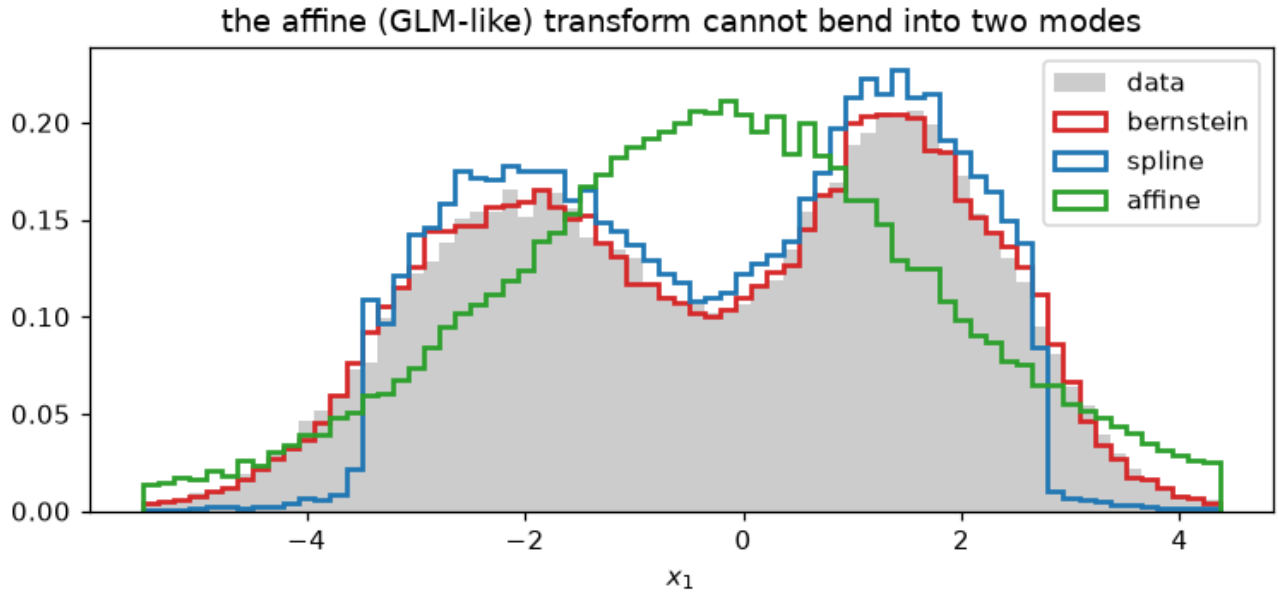


Figure 7: png

rung	query	call	checked against
L1	observational joint	`flow.sample(n)`	50k DGP samples (bimodal x_1 □)
L2	$p(x_3 \mid do(x_2{=}a))$	`flow.sample(n, do=...)`	analytic $\mathbb{E}[x_3] = -0.25 + 0.25a$
L3	individual counterfactuals	`flow.abduct(df)` + `flow.sample(do=..., u=u)`	per-unit DGP truth, r ≈ 0.99+

And the same model family stays **interpretable** when you want it to be. Write an edge as `LS("parent")` instead of `I("parent")`. Then its coefficient is a log-odds ratio that you can read after training (this is the actual point of the paper).

More: repo · paper · didactic walkthrough: `notebooks/intro_tram_dag.py`

TRAM-DAG — a didactic introduction with `tramdag`

TRAM-DAGs (paper, original R/Keras code) are causal models. They use **structured transformation functions** to map a latent representation U to the observed data X. For continuous variables they are **bijective causal models**. Thus one trained model answers all three rungs of Pearl's causal hierarchy:

rung	query	`tramdag` call
L1 association	$p(x)$, sampling	`flow.log_prob(df)`, `flow.sample(n)`
L2 intervention	$p(x \mid do(x_j{=}a))$	`flow.sample(n, do={...})`, `flow.pmf(df, node, do={...})`

rung	query	`tramdag` call
L3 counterfactual	"what would x_i have been, had x_j been a?"	`u=flow.abduct(df); flow.sample(do={...}, u=u)`

This notebook shows the model exactly in the paper notation. It builds a small data-generating process (DGP) **inside the model family**. It fits a `CausalFlowDAG` and verifies each claim against the known ground truth.

(Notebook format and how to run it: see `notebooks/README.md`.)

1. The model

We assume a causal ordering of the variables. For each variable, a TRAM-DAG fits a monotone **transformation function** h (bijective, monotone increasing). This function maps the observed value to a latent scale, conditional on the variable's parents:

$$
\begin{aligned}
u_1 &= h(x_1) \\
u_2 &= h(x_2 \mid x_1) \\
u_3 &= h(x_3 \mid x_1, x_2) \\
&\dots \\
u_p &= h(x_p \mid x_1, x_2, \dots, x_{p-1})
\end{aligned}
$$

This **observed → latent** map is the convention of the paper (Eq. 2, $F_{X|\text{pa}}(x) = F_U(h(x \mid \text{pa}))$) and of the code (in code: `u = h(x) + shift`). It is also the **training direction**: the model evaluates h *directly* to score the likelihood, which is cheap. **Sampling** runs the inverse $x_i = h^{-1}(u_i \mid \text{pa})$. This inverse has no closed form, so bracketed bisection solves it. That is the costlier direction.

Together the h's form one **triangular** flow. Each variable can depend only on a *subset* of its predecessors, its causal parents $\text{pa}(x_i)$. Thus the Jacobian sparsity of the flow *is* the DAG.

For the latents $u_1, \dots, u_p$ we assume a **standard logistic** distribution. This choice makes the fitted parameters interpretable: shifts on the latent scale are **log-odds ratios** (Section 6).

The four components

To keep a valid interpretation, we decompose the transformation **additively on the latent scale**. Each node's h (observed → latent, as above) is

$$
u_i \;=\; h(x_i \mid \text{pa}(x_i)) \;=\; \underbrace{h_\vartheta(x_i)}_{\text{intercept}} \;+\; \underbrace{\textstyle\sum_j \beta_{ij}\, x_j}_{\text{linear shifts (LS)}} \;+\; \underbrace{\textstyle\sum_k g_{ik}(x_k)}_{\text{complex shifts (CS)}} ,
$$

with every causal parent in exactly one term. We use x_5 with parents $\text{pa}(x_5) = \{x_1, x_2, x_4\}$ as the running example:

- **Simple intercept (SI):** $h_\vartheta(x_5)$ has *constant* parameters ϑ. It is a flexible monotone baseline transformation (here: a Bernstein polynomial), the same for every observation.
- **Complex intercept (CI):** the parameters ϑ of $h_\vartheta(x_5)$ are themselves a function of (a subset of) the parents. The whole transformation bends with the parent, which permits interactions beyond additive shifts.
- **Linear shift (LS):** $\beta_{51}x_1 + \beta_{52}x_2$. This gives one interpretable number per parent.
- **Complex shift (CS):** $g(x_4)$, an unrestricted (NN) function of the parent. It stays *additive* on the latent scale.

so that **sampling** (the inverse direction) must invert only the intercept. The shifts move to the other side:

$$x_5 = h^{-1}(u_5 \mid x_1, x_2, x_4) = h_\vartheta^{-1}\Big(u_5 - \underbrace{\beta_{51}x_1 + \beta_{52}x_2}_{\text{LS}} - \underbrace{g(x_4)}_{\text{CS}}\Big).$$

In `tramdag` each node declares its transformation as an **additive formula of terms**. The formula is the node's first argument, written as a list `[...]` or as a `+` sum. The constructors `I` (intercept), `LS` (linear shift), and `CS` (complex shift) build it. Each constructor names the parent(s) that its term depends on:

paper component	`tramdag`
SI — baseline $h_\vartheta(x_i)$, constant ϑ	automatic: every node owns a monotone transform (`bernstein` / `spline` / `affine`); with no intercept term its ϑ is a free parameter vector
CI — ϑ depends on parents	`I("X1")` (several `I(...)` parents feed **one joint** network → interactions)
LS — $\beta_{ij}x_j$	`LS("X1")` (a single weight, no bias)
CS — $g_{ik}(x_k)$	`CS("X1")` (64-128-64 NN, additive)

`I(...)` dispatches on its arguments: no parents → a simple intercept (`SI`), parents → a complex intercept (`CI`). `SI()`/`CI()` spell that out.

Gallery: a transformation *is* an additive decomposition

Read every spec line as a recipe for $h(x_i \mid \text{pa})$. Each parent lands in **exactly one** term: the intercept (the *shape*) or one shift. The table shows the resulting decomposition for a single continuous target X_3:

formula (`terms`)	$u_3 = h(x_3 \mid \text{pa})$	what carries each parent
`None` / `[SI()]` (source)	$h_\vartheta(x_3)$	`SimpleIntercept` — ϑ a free vector
`[CI("X1")]`	$h_{\vartheta(x_1)}(x_3)$	`ComplexIntercept` — **no shift term**; ϑ (the whole shape) bends with x_1
`[LS("X1")]`	$h_\vartheta(x_3) + \beta\, x_1$	`LinearShift` — **one number** β
`[CS("X1")]`	$h_\vartheta(x_3) + g_1(x_1)$	`ComplexShift` — additive NN g_1
`LS("X1") + CS("X2")`	$h_\vartheta(x_3) + \beta x_1 + g_2(x_2)$	one `LinearShift` + one `ComplexShift` (the model fitted below)
`[CS("X1", "X2")]`	$h_\vartheta(x_3) + g_{1,2}(x_1, x_2)$	**one joint** `ComplexShift` — an interaction in the shift
`CS("X1") + CS("X2")`	$h_\vartheta(x_3) + g_1(x_1) + g_2(x_2)$	two **additive** `ComplexShifts`
`[I("X1", "X2", allow_interaction=False)]`	$h_{\vartheta(x_1)+\vartheta(x_2)}(x_3)$	**additive** CI — one network per parent, coefficient vectors summed; each parent reshapes the transform independently

formula (`terms`)	$u_3 = h(x_3 \mid \text{pa})$	what carries each parent
`[I("X1", "X2")]`	$h_{\vartheta(x_1,x_2)}(x_3)$	**one joint** `ComplexIntercept` over both parents (they interact)

A list and a `+` sum are interchangeable: `[LS("X1"), CS("X2")] == LS("X1") + CS("X2")`. You select the class of the monotone transform on the intercept term: `I("X1", transform="spline")`.

For an **ordinal** target the intercept is not a Bernstein curve but the vector of ordered cutpoints $\vartheta_k(\text{pa})$. The model **subtracts** the shift: $P(Y \le k \mid \text{pa}) = \sigma(\vartheta_k - \text{shift})$. `LS` and `CS` terms enter that shift exactly as above.

The odd one out is **`I(...)`** (a complex intercept). It is the only term that is *not* an additive shift. It moves the parent into the intercept, so there is no separate summand and no single-number coefficient to read off (§6). That is the interpretability price when the transformation's *shape* depends on the parent.

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import torch

from tramdag import CS, LS, CausalFlowDAG, ContinuousNode, OrdinalNode

plt.rcParams["figure.dpi"] = 110
```

2. A hand-built DGP — *inside* the model family

To verify everything against ground truth, we now build a small structural causal model **by hand**. It has logistic latents and has exactly the form above. This mirrors the construction in `triangle_structured_continous.R`. The DAG is $X_1 \to X_2 \to X_3 \leftarrow X_1$ plus an ordinal outcome $X_3 \to Y$:

$$
\begin{aligned}
u_1 &= h_1(x_1) = 1.2\,x_1 - 0.4 && \Rightarrow\ x_1 = (u_1 + 0.4)/1.2 \\
u_2 &= h_2(x_2) + \beta_{21} x_1, \quad h_2(x) = 2x + 1,\ \beta_{21} = 1.5 && \Rightarrow\ x_2 = (u_2 - 1.5\,x_1 - 1) \\
u_3 &= h_3(x_3) + \beta_{31} x_1 + g(x_2), \quad h_3(x) = \sinh(x),\ \beta_{31} = 0.8,\ g(x) = \tfrac{1}{2}x^2 && \Rightarrow\ x_3 = \operatorname{asinh}(u_3 - 0.8\,x \\
P(Y \le k) &= \sigma(\vartheta_k - \beta_Y x_3), \quad \vartheta = (-2, 0, 1.5),\ \beta_Y = 1 && \Rightarrow\ y = \#\{k : u_4 > \vartheta_k -
\end{aligned}
$$

These conventions are the TRAM conventions, and tests in this repo pin them:

- continuous nodes: the model **adds** the shift on the latent scale, $u = h(x) + \text{shift}$.
- ordinal nodes: the model **subtracts** the shift inside the sigmoid, $P(Y \le k) = \sigma(\vartheta_k - \text{shift})$, with increasing cutpoints ϑ_k (an *ordered logit*). Because of this flip, a positive β pushes Y toward *higher* categories.

X_2 enters X_3 through a quadratic **complex shift** $g(x_2) = \frac{1}{2}x_2^2$. A linear shift cannot represent this function. We will return to this later.

The figure shows the wiring at a glance. The label on each edge gives the term through which its parent enters:

```
fig, ax = plt.subplots(figsize=(7, 3.2))
pos = {"X1": (0, 1.0), "X2": (0, 0.0), "X3": (1.4, 0.5), "Y": (2.6, 0.5)}
edges = [
```

```
    ("X1", "X2", r"LS: $\beta_{21}=1.5$", (-0.32, 0.5)),
    ("X1", "X3", r"LS: $\beta_{31}=0.8$", (0.62, 0.93)),
    ("X2", "X3", r"CS: $g(x)=\frac{1}{2}x^2$", (0.62, 0.07)),
    ("X3", "Y", r"LS: $\beta_{Y}=1$", (2.0, 0.62)),
]
for src, dst, label, (lx, ly) in edges:
    ax.annotate(
        "",
        xy=pos[dst],
        xytext=pos[src],
        arrowprops=dict(
            arrowstyle="-|>", color="#333333", lw=1.4, shrinkA=16, shrinkB=16
        ),
    )
    ax.text(lx, ly, label, fontsize=10, ha="center", va="center")
for name, (x, y) in pos.items():
    ordinal = name == "Y"
    ax.scatter(
        x,
        y,
        s=900,
        facecolor="white" if not ordinal else "#dce8f4",
        edgecolor="#333333",
        zorder=3,
    )
    ax.text(
        x,
        y,
        f"${name[0]}_{name[1]}$" if len(name) > 1 else f"${name}$",
        ha="center",
        va="center",
        fontsize=13,
        zorder=4,
    )
ax.text(2.6, 0.24, "ordinal (4 levels)", fontsize=9, ha="center", color="#555555")
ax.set_xlim(-0.7, 3.1)
ax.set_ylim(-0.35, 1.35)
ax.axis("off")
ax.set_title("The DGP's DAG", fontsize=11)
plt.show()
```

```
TRUE = dict(b21=1.5, b31=0.8, bY=1.0, theta_Y=np.array([-2.0, 0.0, 1.5]))


def rlogis(rng, n):
    """Standard logistic draws."""
    u = rng.uniform(1e-9, 1 - 1e-9, size=n)
    return np.log(u) - np.log1p(-u)


def g_cs(x2):
    """The true complex shift of X2 on X3."""
    return 0.5 * x2**2


def simulate(n, rng, x1=None, u=None):
```

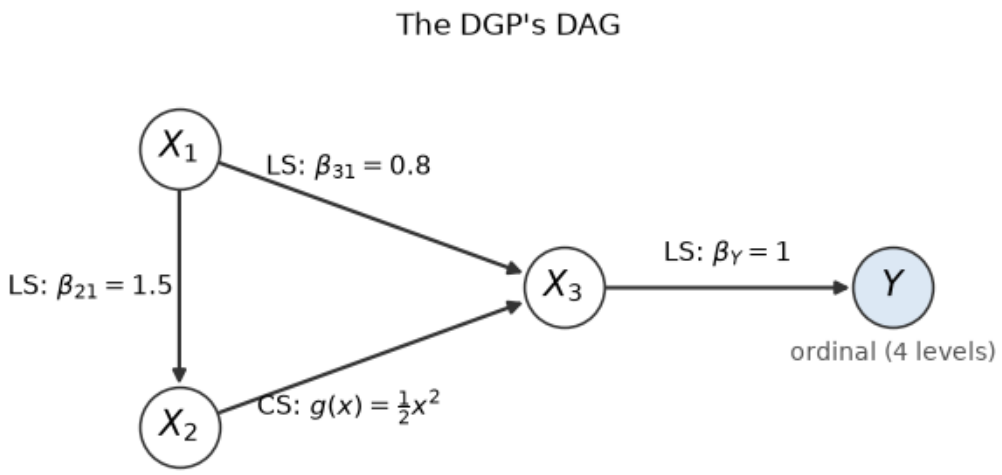


Figure 8: png

```
    """Sample from the SCM. `x1` overrides the source node (= do(X1)),
    `u` reuses given latents (= counterfactuals).
    """
    if u is None:
        u = {k: rlogis(rng, n) for k in ["u1", "u2", "u3", "u4"]}
    if x1 is None:
        x1 = (u["u1"] + 0.4) / 1.2
    else:
        x1 = np.full(n, float(x1))
    x2 = (u["u2"] - TRUE["b21"] * x1 - 1.0) / 2.0
    x3 = np.arcsinh(u["u3"] - TRUE["b31"] * x1 - g_cs(x2))
    cut = TRUE"theta_Y" - TRUE["bY"] * x3[:, None]
    y = (u"u4" > cut).sum(axis=1)
    df = pd.DataFrame({"X1": x1, "X2": x2, "X3": x3, "Y": y.astype(float)})
    return df, u


rng = np.random.default_rng(1)
df, u_obs = simulate(6000, rng)  # keep the latents -> true counterfactuals
train_df, val_df = df.iloc[:5000], df.iloc[5000:]
df.describe().round(2)
```

X1

X2

X3

Y

count

6000.00

6000.00

6000.00

6000.00

mean
0.34
-0.75
-0.73
1.24
std
1.51
1.44
1.41
1.03
min
-7.38
-7.34
-4.18
0.00
25%
-0.57
-1.68
-1.86
0.00
50%
0.34
-0.74
-0.99
1.00
75%
1.26
0.16
0.43
2.00
max
7.70
5.01
2.85
3.00

```
fig, axes = plt.subplots(1, 3, figsize=(11, 3.2))
for ax, (a, b) in zip(axes, [("X1", "X2"), ("X1", "X3"), ("X2", "X3")], strict=True):
    ax.scatter(df[a], df[b], s=3, alpha=0.25)
    ax.set_xlabel(a), ax.set_ylabel(b)
axes[2].set_title("the U-shape of the complex shift", fontsize=9)
fig.suptitle("Observational data from the hand-built SCM")
fig.tight_layout()
plt.show()
```

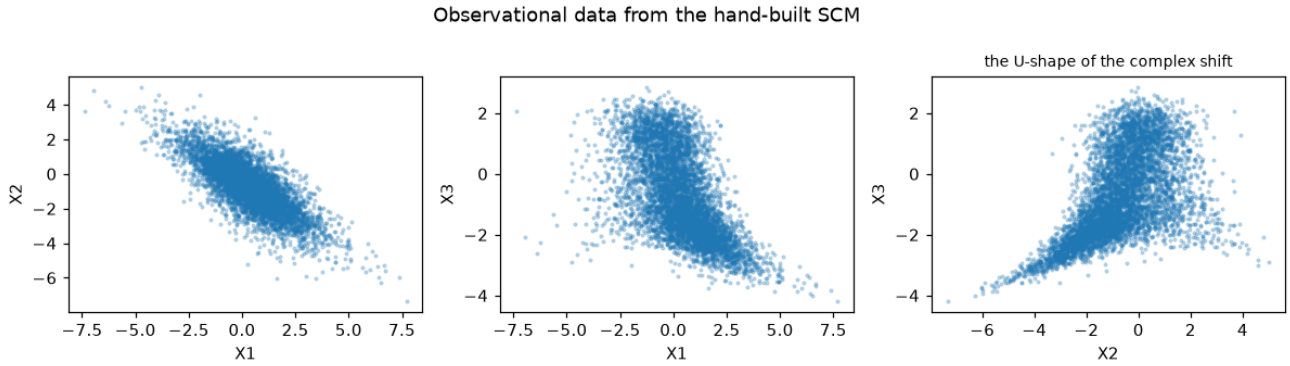


Figure 9: png

3. Specifying the DAG and fitting the flow

The model spec *is* the labeled adjacency matrix, written per node. Each continuous node automatically gets its monotone baseline transformation (default: Bernstein polynomial with 20 coefficients, the TRAM-faithful choice). The edges declare how each parent enters.

Fitting maximizes the joint likelihood. Because the flow is triangular, the negative log-likelihood **decomposes per node** ($\log p(x) = \sum_i \log p(x_i \mid \mathrm{pa}(x_i))$). Thus one Adam optimizer trains all nodes at once. `fit` keeps the final weights — for this unconfounded DGP the MLE is the right target. Progress printing is `fit`'s own `verbose=` (Keras-style; 0 = silent, N = every Nth epoch).

```
spec = {
    "X1": ContinuousNode(),
    "X2": ContinuousNode(LS("X1")),
    "X3": ContinuousNode(LS("X1") + CS("X2")),
    "Y": OrdinalNode(4, LS("X3")),
}

flow = CausalFlowDAG(
    spec, seed=1
)  # seed here too, for the Bernsteins' initial uniform knots
flow.fit(train_df, epochs=800, learning_rate=1e-2, batch_size=20000, verbose=200)
flow.fit(train_df, epochs=300, learning_rate=1e-3, batch_size=512)  # polish
flow.nll(val_df)
```

```
epoch 200/800  train 5.5673


epoch 400/800  train 5.4506


epoch 600/800  train 5.4299
```

```
epoch 800/800  train 5.4265
```

```
{'X1': 1.8364977836608887,
 'X2': 1.3018457889556885,
 'X3': 1.174437403678894,
 'Y': 1.1122666597366333}
```

4. Anatomy: the spec *is* the additive decomposition

Before the per-node detail, the whole model at a glance. `flow.to_matrix()` labels every edge with the term that carries it — this is the paper's **meta-adjacency matrix** (Fig. 3), and it is generated from the fitted object rather than drawn by hand, so it cannot disagree with the model:

```
print(flow.to_matrix())
```

```
   X1  X2  X3   Y
X1     LS  LS
X2         CS
X3             LS
Y
```

Rows are parents, columns children; an empty cell means no edge. Now read the same structure node by node, directly from the **fitted** flow. Two small helpers do the job. `describe_node` reports which network carries each parent (the structural view). `decompose_row` prints the actual numbers for one observation and verifies that they rebuild the per-node log-likelihood **exactly**. The equation $u = h_\vartheta(x) + \sum$ shifts is an identity, not a picture. We run both on `X2` (an `LS` edge), `X3` (`LS` + `CS`), and the ordinal `Y` (shift **subtracted**).

```
from tramdag.transforms import (  # noqa: E402
    StandardLogistic,
    ordinal_cutpoints,
    ordinal_log_prob,
)
```

```
def describe_node(flow, name):
    """Structural view: the intercept module and each parent's term + network."""
    node = flow.nodes[name]
    n_params = node.ut.n_params if node.ut is not None else node.levels - 1
    print(f"{name}  ({node.kind})")
    if node.intercept.ci_parents:
        print(
            f"  intercept: ComplexIntercept({node.intercept.ci_parents}"
            f" -> {n_params} params)"
        )
    else:
        print(f"  intercept: SimpleIntercept({n_params} params)")
    for parent in node.intercept.ci_parents:
        print(f"    {parent:>3} -> I    (feeds the joint intercept above)")
    for parent, mod in node.shifts.items():
        eff = "LS" if type(mod).__name__ == "LinearShift" else "CS"
```

```
        print(f"    {parent:>3} -> {eff}   ({type(mod).__name__})")
    if not node.parents:
        print("    (source node — no parents)")


def decompose_row(flow, name, row_df):
    """Numeric view: print u = intercept + sum(shifts) for one row and check it
    reproduces flow.node_log_prob exactly.

    Anatomy on purpose: ``_tensorize``/``_features`` are library internals,
    used here to open the model up — user code never needs them.
    """
    node = flow.nodes[name]
    vals = flow._tensorize(row_df)
    feats = flow._features(vals)
    theta, shift = node.theta_shift(feats, len(row_df))
    parts = {p: node.shifts[p](feats[p]) for p in node.shifts}  # per-parent shift
    print(f"{name} = {float(valsname):+.3f}  ({node.kind})")
    if node.kind == "continuous":
        h0, ladj = node.ut.forward(theta, vals[name])
        terms = "  +  ".join(
            [f"h_theta(x)={float(h0[0]):+.3f}"]
            + [f"{p}={float(v[0]):+.3f}" for p, v in parts.items()]
        )
        u = h0 + shift
        print(f"  u = {terms}  =  {float(u[0]):+.3f}   (standard-logistic latent)")
        lp = StandardLogistic.log_prob(u) + ladj
    else:  # ordinal: cutpoints minus a subtracted shift
        cuts = ordinal_cutpoints(theta)[0, 1:-1].detach().numpy().round(3)
        terms = "  +  ".join(f"{p}={float(v[0]):+.3f}" for p, v in parts.items()) or "0"
        print(f"  cutpoints theta_k = {cuts}")
        print(
            f"  shift (SUBTRACTED) = {terms}   ->   P(Y<=k) = sigmoid(theta_k - shift)"
        )
        lp = ordinal_log_prob(theta, shift, vals[name])
    check = flow.node_log_prob(vals)[name]
    print(
        f"  log p(row) rebuilt = {float(lp[0]):+.4f}   node_log_prob = "
        f"{float(check[0]):+.4f}   match={bool(torch.allclose(lp, check))}\n"
    )


for nm in ["X2", "X3", "Y"]:
    describe_node(flow, nm)
print()
row0 = val_df.iloc[[0]]
for nm in ["X2", "X3", "Y"]:
    decompose_row(flow, nm, row0)
```

```
X2  (continuous)
  intercept: SimpleIntercept(20 params)
     X1 -> CS   (LinearShiftTerm)
X3  (continuous)
  intercept: SimpleIntercept(20 params)
     X1 -> CS   (LinearShiftTerm)
     X2 -> CS   (ComplexShiftTerm)
```

```
Y  (ordinal)
  intercept: SimpleIntercept(3 params)
     X3 -> CS   (LinearShiftTerm)

X2 = +2.579  (continuous)
  u = h_theta(x)=+6.186  +  X1=-3.390  =  +2.797   (standard-logistic latent)
  log p(row) rebuilt = -2.2075   node_log_prob = -2.2075   match=True

X3 = -1.570  (continuous)
  u = h_theta(x)=-1.460  +  X1=-1.707  +  X2=+2.341  =  -0.825  (standard-logistic latent)
  log p(row) rebuilt = -0.6135   node_log_prob = -0.6135   match=True

Y = +1.000  (ordinal)
  cutpoints theta_k = [-2.011  0.019  1.587]
  shift (SUBTRACTED) = X3=-1.553   ->   P(Y<=k) = sigmoid(theta_k - shift)
  log p(row) rebuilt = -0.8199   node_log_prob = -0.8199   match=True



/tmp/ipykernel_3643/575676898.py:45: UserWarning: Converting a tensor with requires_grad=True to a s
Consider using tensor.detach() first. (Triggered internally at /__w/pytorch/pytorch/torch/csrc/auto
  [f"h_theta(x)={float(h0[0]):+.3f}"]
```

5. Rung 1 — the observational distribution

First sanity check: samples from the fitted flow must reproduce the joint observational distribution (including the ordinal outcome's marginal).

```python
samp = flow.sample(len(df), seed=0)

fig, axes = plt.subplots(1, 4, figsize=(13, 3))
for ax, col in zip(axes[:3], ["X1", "X2", "X3"], strict=True):
    bins = np.linspace(df[col].min(), df[col].max(), 60)
    ax.hist(df[col], bins=bins, density=True, alpha=0.45, label="data")
    ax.hist(
        samp[col],
        bins=bins,
        density=True,
        histtype="step",
        lw=1.8,
        color="C3",
        label="flow",
    )
    ax.set_title(col)
levels = np.arange(4)
w = 0.35
axes[3].bar(
    levels - w / 2,
    df["Y"].value_counts(normalize=True).sort_index(),
    width=w,
    alpha=0.6,
    label="data",
)
axes[3].bar(
    levels + w / 2,
    samp["Y"].value_counts(normalize=True).sort_index(),
```

```
    width=w,
    color="C3",
    alpha=0.8,
    label="flow",
)
axes[3].set_title("Y"), axes[3].set_xticks(levels)
axes[0].legend()
fig.suptitle("L1: observational marginals, data vs. flow samples")
fig.tight_layout()
plt.show()
```

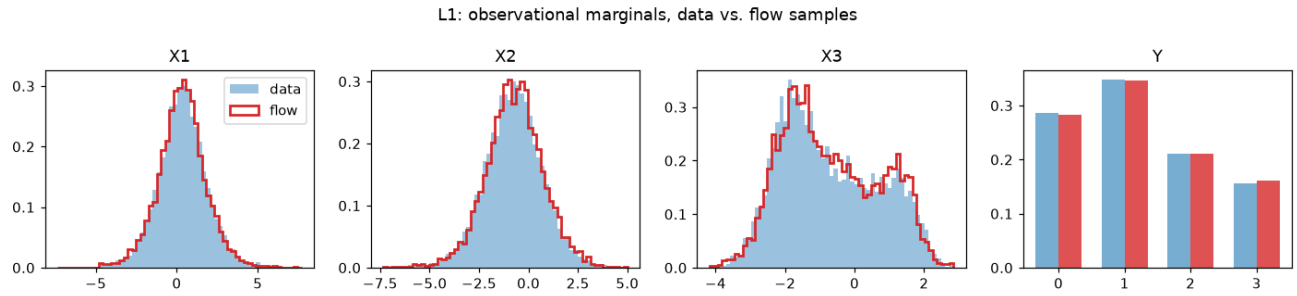


Figure 10: png

6. Single-number interpretable statistics

Because the latents are standard logistic, every linear-shift weight is a **log-odds ratio**. For a continuous node ($u = h(x) + \beta\, x_{\text{pa}}$), a unit increase of the parent multiplies the odds of $\{X \le x\}$ by e^{β}, uniformly in x (a proportional-odds / Colr-type effect). For the ordinal node the sign convention flips ($\sigma(\vartheta_k - \text{shift})$). Thus a positive β moves Y toward higher categories: e^{β} multiplies the odds of $\{Y > k\}$.

These are *parameters of the fitted flow*. We can read them directly and compare them with the DGP constants:

```
# raw parameter access, to make the point; flow.ls_coefficients() is the
# one-call read-out for all of them
b21_hat = float(flow.nodes["X2"].shifts["X1"].weight.detach())
b31_hat = float(flow.nodes["X3"].shifts["X1"].weight.detach())
bY_hat = float(flow.nodes["Y"].shifts["X3"].weight.detach())


with torch.no_grad():
    theta_hat = ordinal_cutpoints(flow.nodes["Y"].intercept(1))[0, 1:-1].numpy()

print(f"beta_21 (X1 -> X2):  true {TRUE['b21']:+.3f}   fitted {b21_hat:+.3f}")
print(f"beta_31 (X1 -> X3):  true {TRUE['b31']:+.3f}   fitted {b31_hat:+.3f}")
print(f"beta_Y  (X3 -> Y):   true {TRUE['bY']:+.3f}   fitted {bY_hat:+.3f}")
print(f"cutpoints theta_Y:   true {TRUE['theta_Y']}   fitted {theta_hat.round(3)}")
```

```
beta_21 (X1 -> X2):  true +1.500   fitted +1.486
beta_31 (X1 -> X3):  true +0.800   fitted +0.748
beta_Y  (X3 -> Y):   true +1.000   fitted +0.989
cutpoints theta_Y:   true [-2.   0.   1.5]   fitted [-2.011  0.019  1.587]
```

The fit also recovers the flexible parts. The baseline transformation $\hat{h}_3$ must match sinh, and the complex shift $\hat{g}$ must match $\frac{1}{2}x_2^2$. Each match holds **up to an additive constant**, because a

constant can move freely between the intercept and a complex shift (only their sum is identified). Therefore we center both curves before the comparison.

```
def fitted_baseline(flow, name, grid):
    """h_hat(x) for a continuous node with constant (simple) intercept."""
    node = flow.nodes[name]
    x = torch.as_tensor(grid, dtype=torch.float32)
    with torch.no_grad():
        h0, _ = node.ut.forward(node.intercept(len(grid)), x)
    return h0.detach().numpy()


def fitted_cs(flow, name, parent, grid):
    """g_hat(parent) for a 'cs' edge."""
    x = torch.as_tensor(grid, dtype=torch.float32).view(-1, 1)
    with torch.no_grad():
        return flow.nodes[name].shifts[parent](x).detach().numpy()


x3_grid = np.linspace(*df["X3"].quantile([0.01, 0.99]), 200)
x2_grid = np.linspace(*df["X2"].quantile([0.01, 0.99]), 200)
h3_hat, h3_true = fitted_baseline(flow, "X3", x3_grid), np.sinh(x3_grid)
g_hat, g_true = fitted_cs(flow, "X3", "X2", x2_grid), g_cs(x2_grid)

fig, axes = plt.subplots(1, 2, figsize=(9, 3.4))
axes[0].plot(x3_grid, h3_true - h3_true.mean(), lw=2, label=r"true $\sinh(x)$")
axes[0].plot(x3_grid, h3_hat - h3_hat.mean(), "--", lw=2, label=r"fitted $\hat h_3$")
axes[0].set_title("baseline transformation of $X_3$"), axes[0].set_xlabel("$x_3$")
axes[1].plot(x2_grid, g_true - g_true.mean(), lw=2, label=r"true $\frac{1}{2}x^2$")
axes[1].plot(x2_grid, g_hat - g_hat.mean(), "--", lw=2, label=r"fitted $\hat g$")
axes[1].set_title("complex shift $X_2 \\to X_3$"), axes[1].set_xlabel("$x_2$")
for ax in axes:
    ax.legend()
fig.suptitle("Recovered transformation functions (centered)")
fig.tight_layout()
plt.show()
```

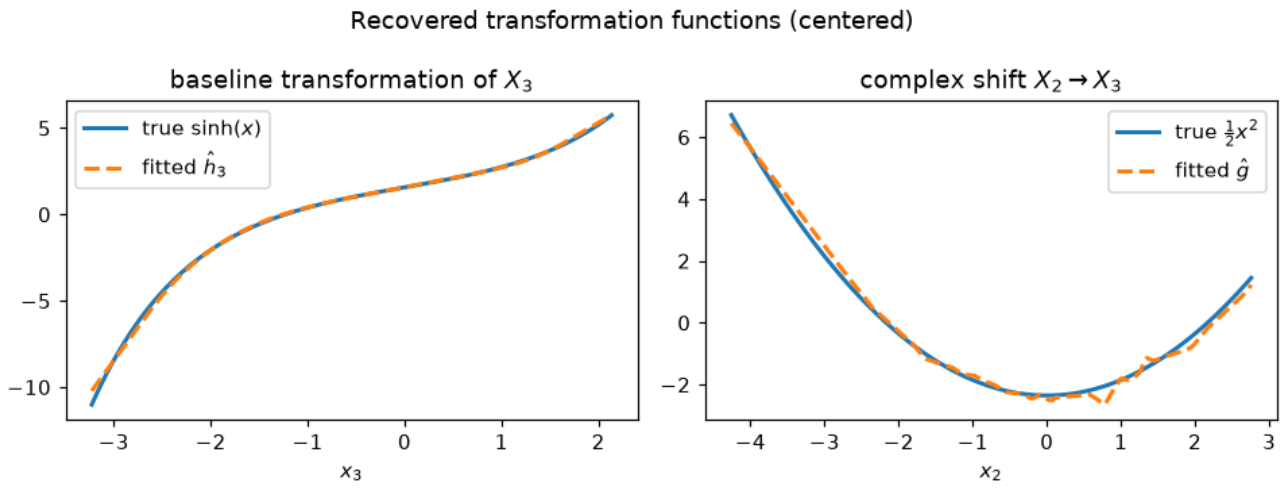


Figure 11: png

Why the term choice matters: a deliberately misspecified model

What happens if we declare the $X_2 \to X_3$ edge as a *linear* shift? The best a linear shift can do is the **average local slope** of g. The data mass sits around $E[x_2] \approx -0.75$, so the `LS` model finds $\hat{\beta}_{32} \approx E[g'(x_2)] = E[x_2] \approx -0.7$. That is the tangent of the U-shape, not the U-shape. The model loses the curvature. The per-node validation NLL makes the misfit measurable, and the interventional distributions in the next section are visibly wrong.

```
spec_ls = {
    "X1": ContinuousNode(),
    "X2": ContinuousNode(LS("X1")),
    "X3": ContinuousNode(LS("X1") + LS("X2")),  # <- CS replaced by LS
    "Y": OrdinalNode(4, LS("X3")),
}
flow_ls = CausalFlowDAG(spec_ls, seed=7)
# the same schedule as the cs model above, so the NLL comparison is fair
flow_ls.fit(train_df, epochs=800, learning_rate=1e-2, batch_size=20000)
flow_ls.fit(train_df, epochs=300, learning_rate=1e-3, batch_size=512)

print(
    f"misspecified beta_32 (X2 -> X3): "
    f"{float(flow_ls.nodes['X3'].shifts['X2'].weight.detach()):+.3f}"
)
print(
    f"val NLL of node X3:  cs model {flow.nll(val_df)['X3']:.4f}"
    f"   ls model {flow_ls.nll(val_df)['X3']:.4f}"
)
```

```
misspecified beta_32 (X2 -> X3): -0.609
val NLL of node X3:  cs model 1.1744   ls model 1.3798
```

7. Rung 2 — interventions: the do-operator

`flow.sample(n, do={"X1": a})` performs **graph mutilation**: it clamps X_1 to a and discards the node's own mechanism (and latent). All downstream nodes react. Because we own the DGP, we can simulate the *true* interventional distribution and compare. We also show the misspecified all-`LS` model. It gets the interventional distribution of X_3 visibly wrong.

```
rng_iv = np.random.default_rng(123)
fig, axes = plt.subplots(1, 2, figsize=(10, 3.4), sharey=True)
for ax, a in zip(axes, [-1.0, 1.0], strict=True):
    truth, _ = simulate(20000, rng_iv, x1=a)
    fl = flow.sample(20000, do={"X1": a}, seed=5)
    fls = flow_ls.sample(20000, do={"X1": a}, seed=5)
    bins = np.linspace(truth["X3"].min(), truth["X3"].max(), 60)
    ax.hist(truth["X3"], bins=bins, density=True, alpha=0.4, label="DGP truth")
    ax.hist(
        fl["X3"],
        bins=bins,
        density=True,
        histtype="step",
        lw=2,
        color="C3",
        label="flow (cs)",
    )
    ax.hist(
        fls["X3"],
```

```
        bins=bins,
        density=True,
        histtype="step",
        lw=1.5,
        color="C7",
        ls=":",
        label="flow (all-ls)",
    )
    ax.set_title(f"$p(x_3 \\mid do(X_1 = {a:+.0f}))$"), ax.set_xlabel("$x_3$")
    print(
        f"E[X3 | do(X1={a:+.0f})]:  truth {truth['X3'].mean():+.3f}   "
        f"flow(cs) {fl['X3'].mean():+.3f}   flow(all-ls) {fls['X3'].mean():+.3f}"
    )
axes[0].legend()
fig.suptitle("L2: interventional distributions")
fig.tight_layout()
plt.show()
```

```
E[X3 | do(X1=-1)]:  truth +0.243   flow(cs) +0.239   flow(all-ls) +0.169


E[X3 | do(X1=+1)]:  truth -1.101   flow(cs) -1.112   flow(all-ls) -1.227
```

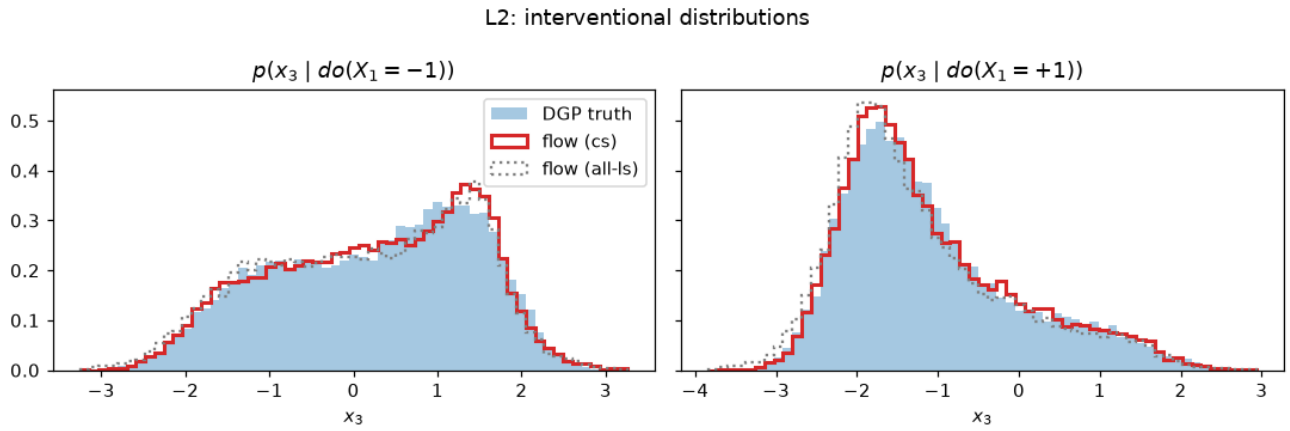


Figure 12: png

For the **ordinal** outcome, Monte Carlo is not necessary. The ordered-logit head gives the interventional PMF **analytically**, $P(Y = k \mid do(X_3 = a)) = \sigma(\vartheta_k - \beta_Y a) - \sigma(\vartheta_{k-1} - \beta_Y a)$.

```
a = 1.0
pmf_flow = flow.pmf(pd.DataFrame({"X3": [a]}), "Y")[0]
cdf_true = 1 / (1 + np.exp(-(np.r_[-np.inf, TRUE["theta_Y"], np.inf] - TRUE["bY"] * a)))
pmf_true = np.diff(cdf_true)
print("P(Y = k | do(X3 = 1)):")
print("  true:", pmf_true.round(4), "\n  flow:", pmf_flow.round(4))
```

```
P(Y = k | do(X3 = 1)):
  true: [0.0474 0.2215 0.3535 0.3775]
  flow: [0.0474 0.2274 0.3703 0.3549]
```

For a **continuous** node the same closed form exists: `flow.density` gives $p(x_3 \mid do(X_1 = a), x_2)$ on a grid from the fitted transform, no sampling. The DGP tells us the truth: $u_3 = \sinh(x_3) + 0.8\,x_1 + \frac{1}{2}x_2^2$, so $p(x_3) = f_U(u_3)\cosh(x_3)$ with f_U the standard-logistic density.

```
grid = np.linspace(-3.5, 3.5, 301)
row = pd.DataFrame({"X2": [0.0]})  # the parent we keep; X1 is set by do=
fig, ax = plt.subplots(figsize=(7, 3.2))
for a, color in ((-1.0, "C0"), (1.0, "C3")):
    dens = flow.density(row, "X3", grid, do={"X1": a})[0]
    u3 = np.sinh(grid) + 0.8 * a + 0.5 * 0.0**2
    truth_pdf = np.exp(-u3) / (1 + np.exp(-u3)) ** 2 * np.cosh(grid)
    ax.plot(
        grid, truth_pdf, color=color, lw=2.5, alpha=0.4, label=f"truth, $a={a:+.0f}$"
    )
    ax.plot(grid, dens, color=color, lw=1.2, label=f"flow.density, $a={a:+.0f}$")
ax.set_xlabel("$x_3$"), ax.set_ylabel("$p(x_3 \\mid do(X_1=a), x_2=0)$")
ax.legend()
fig.tight_layout()
plt.show()
```

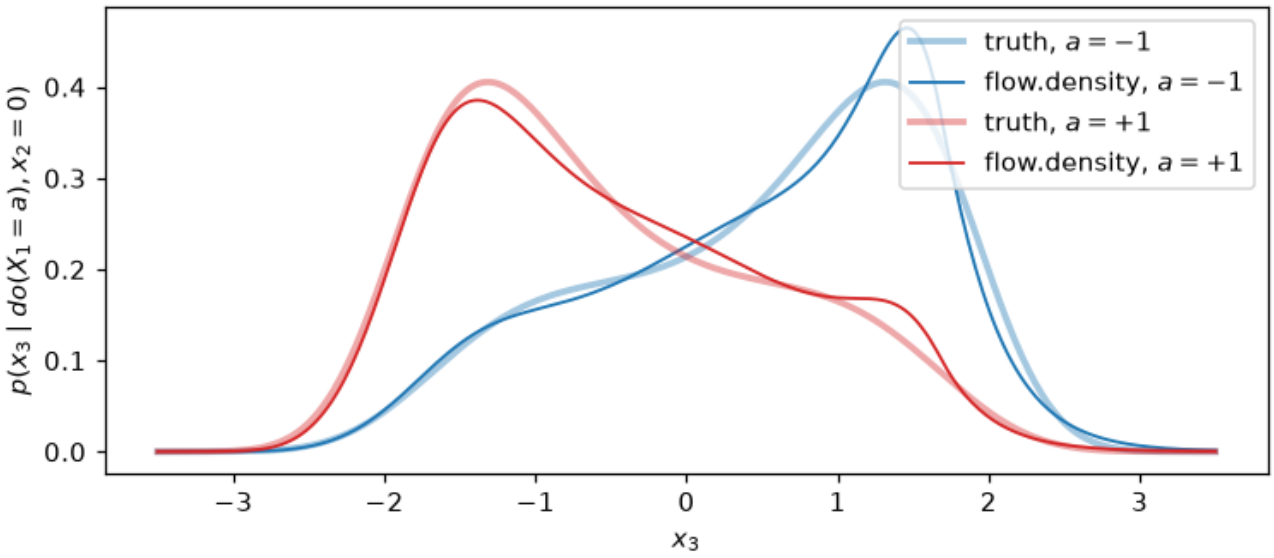


Figure 13: png

8. Rung 3 — counterfactuals: abduction → action → prediction

Because the flow is **bijective in the continuous variables**, Pearl's three steps are exact:

1. **Abduction** — map the *factual* data to its latent (the training direction): $u = h(x \mid \text{pa})$ per node (`flow.abduct(df)`). Each row's u is its individual "noise". It is everything about the unit that the model does not attribute to the parents. (For ordinal nodes u is only interval-identified. Thus the flow draws it from the logistic truncated to the observed level's interval.)
2. **Action** — mutilate the graph: `do={"X1": 0.0}`.
3. **Prediction** — push the *same* u back through the mutilated flow: `flow.sample(do=..., u=u)`.

First check: we push the abducted latents through the flow with no action. This must reproduce the factual data exactly (level-exactly for Y).

```
u = flow.abduct(val_df, seed=11)
recon = flow.sample(u=u)
err = recon[["X1", "X2", "X3"]].to_numpy() - val_df[["X1", "X2", "X3"]].to_numpy()
print(f"max |reconstruction error| (continuous): {np.abs(err).max():.2e}")
print(f"Y level-exact: {(recon['Y'].to_numpy() == val_df['Y'].to_numpy()).mean():.1%}")
```

```
max |reconstruction error| (continuous): 1.51e-05
Y level-exact: 100.0%
```

Now we ask the counterfactual: *"what would X_2, X_3 have been for **this** unit, had X_1 been 0?"*. The DGP kept every unit's true latents `u_obs`. Thus we can compute the **true individual counterfactuals** and compare them unit by unit. This is the strongest test on the ladder.

```
cf_flow = flow.sample(do={"X1": 0.0}, u=u)
u_val = {k: v[5000:] for k, v in u_obs.items()}
cf_true, _ = simulate(len(val_df), rng, x1=0.0, u=u_val)

fig, axes = plt.subplots(1, 2, figsize=(9, 3.4))
for ax, col in zip(axes, ["X2", "X3"], strict=True):
    ax.scatter(cf_true[col], cf_flow[col], s=5, alpha=0.4)
    lims = [cf_true[col].min(), cf_true[col].max()]
    ax.plot(lims, lims, "k--", lw=1)
    r = np.corrcoef(cf_true[col], cf_flow[col])[0, 1]
    rmse = float(np.sqrt(np.mean((cf_true[col] - cf_flow[col]) ** 2)))
    print(f"counterfactual {col}:  corr(truth, flow) = {r:.4f}   RMSE = {rmse:.3f}")
    ax.set_title(f"counterfactual {col}   (r = {r:.4f})")
    ax.set_xlabel("DGP truth"), ax.set_ylabel("flow")
fig.suptitle("L3: individual counterfactuals under $do(X_1 = 0)$, unit by unit")
fig.tight_layout()
plt.show()
```

```
counterfactual X2:  corr(truth, flow) = 0.9999   RMSE = 0.013
counterfactual X3:  corr(truth, flow) = 0.9954   RMSE = 0.117
```

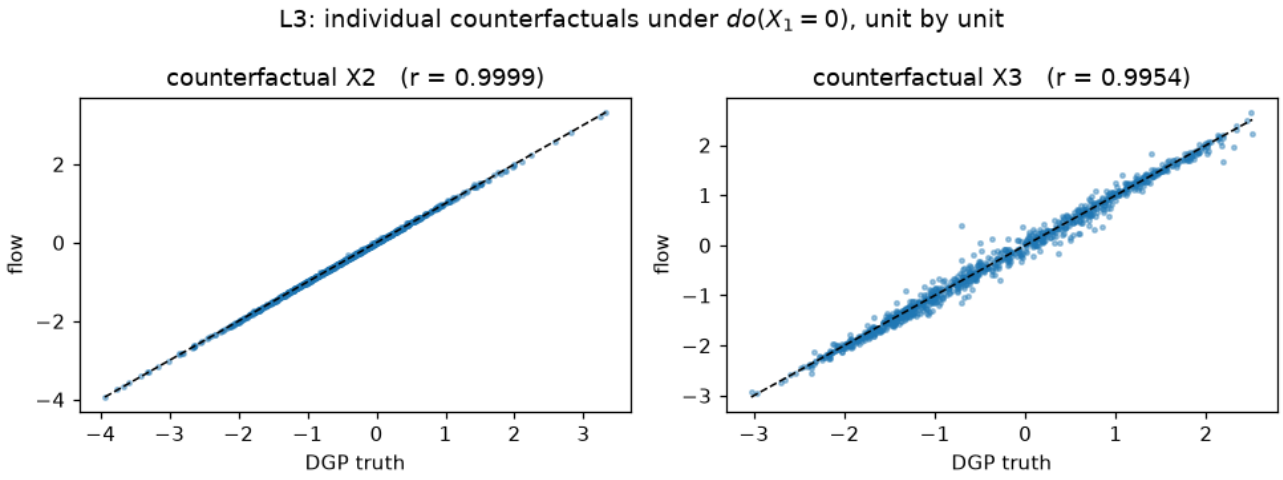


Figure 14: png

9. Where to go from here

- **Complex intercepts (`I(...)`)** — this is the one component not exercised here. Declare `ContinuousNode(I("Age"))`, and the *parameters* of the Bernstein transform become a function of the parent. Several `I(...)` parents feed one joint network, that is, they can interact. The paper replications in `experiments/` use `I(...)` heavily. Run `uv run python -m paper.vaca flexible` (from `experiments/`) for an all-intercept flow on a bimodal benchmark with analytic interventional truth.
- **Early stopping vs. exact MLE** — this notebook's DGP has no unobserved confounding, so the MLE (the final weights `fit` keeps) is the right target. Under observational con-

founding, flexible (`I`/`CS`) models can *overfit the confounding* at the MLE and need best-validation weights (`tramdag.callbacks.EarlyStopping`) to recover the causal effect. See `docs/fitting.md`.
- **Validation against classical models** — an all-`LS` flow trained to convergence *is* the classical proportional-odds MLE (`experiments/misc/validate_ls.py` pins flow ≡ `statsmodels` ≡ R `polr`, and the framework tests check the same identity on an inline DGP).
- **Joint terms** — write several parents inside one term to model an *interaction*. `CS("x1", "x2")` is a single shift network $g(x_1, x_2)$, and `I("x1", "x2")` is a single intercept network over both parents. Separate terms (`CS("x1") + CS("x2")`) stay additive. The grouping *is* the joint/additive choice.
- **Current limitations** (vs. the general formulation): the latent is fixed to the standard logistic.

This file is a jupytext percent notebook. If you prefer `.ipynb`, pair the file with `uvx jupytext --to ipynb notebooks/intro_tram_dag.py`. Keep the `.ipynb` out of git (see `.gitignore`).

Heterogeneous treatment effects: the `VC` term

Most causal questions are not "does the treatment help?" but "**whom** does it help, and by how much?". A TRAM-DAG answers that with a **varying-coefficient** term, which gives one edge of the DAG a treatment effect that depends on covariates:

$$\text{shift} \;+\!=\; \beta(\text{modifiers}) \cdot x_t, \qquad \beta(x) = \beta_0 + b_\Theta(x)$$

`beta0` is the constant part — an interpretable log-odds ratio, exactly as a linear shift would give — and `b_Theta` is a deliberately small, **penalized** network that bends it per individual.

This notebook builds a DGP whose effect function we know exactly, fits the model, and scores the recovered `beta(x)` against the truth. It then shows the two things the term exists for that a plain flexible shift does not give: a read-out you can plot, and a **propensity-centered** variant that survives confounding.

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

from tramdag import CS, LS, VC, CausalFlowDAG, ContinuousNode, I, OrdinalNode
from tramdag.callbacks import EarlyStopping

plt.rcParams["figure.dpi"] = 110
```

1. A DGP with a known effect function

Three covariates, a **confounded** treatment (`X1` and `X2` push assignment), and an outcome whose conditional is exactly a transformation model:

$$h(y) + g(x) + \beta(x)\, t = U, \qquad U \sim \text{Logistic}(0, 1)$$

with $h(y) = 2y$, a nonlinear prognostic part $g(x) = \frac{1}{2}X_1^2 + X_2 - \frac{1}{2}X_3$, and the effect function

$$\beta(x) = -1 + 0.8\, X_2 - 0.6\, X_3.$$

`X2` is deliberately *both* a confounder and an effect modifier — the case where a naive estimate fails hardest.

```
B0, B2, B3 = -1.0, 0.8, -0.6


def true_beta(df):
    """The effect function the model has to recover, on the latent scale."""
    return B0 + B2 * df["X2"].to_numpy() + B3 * df["X3"].to_numpy()


def simulate(n, seed):
    rng = np.random.default_rng(seed)
    x1, x2, x3 = (rng.normal(size=n) for _ in range(3))
    t = (rng.logistic(size=n) > -(0.4 * x1 + 0.4 * x2)).astype(float)
    g = 0.5 * x1**2 + x2 - 0.5 * x3
    beta = B0 + B2 * x2 + B3 * x3  # the same expression true_beta reports
    y = (rng.logistic(size=n) - g - beta * t) / 2.0
    return pd.DataFrame({"X1": x1, "X2": x2, "X3": x3, "T": t, "Y": y})


train, val, test = simulate(4500, 1), simulate(500, 2), simulate(2000, 3)
print(f"treated fraction: {train['T'].mean():.2f}")
print(
    f"true effect ranges from {true_beta(test).min():+.2f} to {true_beta(test).max():+.2f}"
)
```

```
treated fraction: 0.48
true effect ranges from -4.41 to +2.29
```

2. Which covariates modify the effect? (`effect_modifier_scan`)

Before committing to a set of modifiers, ask the data. Fit the **cheap all-`ls` model** — seconds, deterministic — and look at the per-observation scores of the treatment coefficient. If the effect really were constant, those scores would carry no structure in any covariate; a CUSUM statistic per covariate turns that into a ranking.

```
def cheap_all_ls():
    return {
        "X1": ContinuousNode(),
        "X2": ContinuousNode(),
        "X3": ContinuousNode(),
        "T": OrdinalNode(2, [LS("X1"), LS("X2")]),
        "Y": ContinuousNode([LS("X1"), LS("X2"), LS("X3"), LS("T")]),
    }


screen = CausalFlowDAG(cheap_all_ls(), seed=0)
screen.fit_classical(train)
print(screen.effect_modifier_scan(train, "Y", t="T"))
```

```
        stat       p_value  crit_5pct  flag
X2  4.163334  1.759830e-15     1.3581  True
X1  4.139814  2.600764e-15     1.3581  True
X3  3.162057  4.133820e-09     1.3581  True
```

`X2` and `X3` are the true modifiers and both flag — but so does `X1`, which does **not** enter $\beta(x)$ at all. That is not a bug, and it is worth understanding before you trust the shortlist: the statistic

measures how unstable the *cheap model* is along a covariate, and instability has two sources — a coefficient that truly varies, and a **prognostic part the cheap model gets wrong**. Here g contains $\frac{1}{2}X_1^2$, which a linear term cannot represent.

The demonstration: re-generate with a *linear* prognostic X_1, change nothing else, and `X1` drops out of the shortlist.

```
def simulate_linear_x1(n, seed):
    rng = np.random.default_rng(seed)
    x1, x2, x3 = (rng.normal(size=n) for _ in range(3))
    t = (rng.logistic(size=n) > -(0.4 * x1 + 0.4 * x2)).astype(float)
    g = 0.5 * x1 + x2 - 0.5 * x3  # linear, so the cheap model fits it exactly
    y = (rng.logistic(size=n) - g - (B0 + B2 * x2 + B3 * x3) * t) / 2.0
    return pd.DataFrame({"X1": x1, "X2": x2, "X3": x3, "T": t, "Y": y})


well_specified = simulate_linear_x1(4500, 1)
screen2 = CausalFlowDAG(cheap_all_ls(), seed=0)
screen2.fit_classical(well_specified)
print(screen2.effect_modifier_scan(well_specified, "Y", t="T"))
```

```
        stat       p_value  crit_5pct   flag
X2  4.609058  7.066926e-19     1.3581   True
X3  3.476365  6.368441e-11     1.3581   True
X1  0.552602  9.201793e-01     1.3581  False
```

Read the scan as a **screening** step, then: it shortlists covariates worth giving to the effect head, and a flag can also mean "your prognostic part is wrong here". Both are things you want to know.

3. The spec: prognostic part and effect head are separate

`CS("X1", "X2", "X3")` absorbs the prognostic signal — as flexible as you like — while `VC("X2", "X3", t="T")` carries the effect. `X2` and `X3` appear twice on purpose: prognostically through the shift, and as modifiers through the head. Only the treatment named by `t=` owns its edge.

```
spec = {
    "X1": ContinuousNode(),
    "X2": ContinuousNode(),
    "X3": ContinuousNode(),
    "T": OrdinalNode(2, [LS("X1"), LS("X2")]),
    "Y": ContinuousNode([CS("X1", "X2", "X3"), VC("X2", "X3", t="T")]),
}
flow = CausalFlowDAG(spec, seed=0)
flow.fit(
    train,
    epochs=300,
    learning_rate=1e-2,
    batch_size=512,
    validation_data=val,
    seed=0,
    callbacks=EarlyStopping(),  # keep the best-validation weights
)
print(flow.to_matrix())
```

```
   X1 X2 X3   T                          Y
X1           LS      CS['X1', 'X2', 'X3']
X2           LS  CS['X1', 'X2', 'X3']+VCm
X3               CS['X1', 'X2', 'X3']+VCm
```

```
T                                   VC
Y
```

The adjacency view above is the paper's *meta-adjacency matrix*: every edge labelled with the term that carries it. `VC` marks the treatment edge and `VCm` the modifiers, which is how you can see at a glance that `X2` enters `Y` twice.

4. Reading the effect out

`varying_coef` gives $\beta(x)$ per row. It is **deterministic** and does not look at the outcome — it is a property of the fitted model, not of the rows' `Y` values.

```
beta_hat = flow.varying_coef(test, "Y")
beta_true = true_beta(test)
corr = float(np.corrcoef(beta_hat, beta_true)[0, 1])
beta0 = float(flow.nodes["Y"].shifts["T"].beta0)

print(f"corr(beta_hat, beta_true) = {corr:.3f}")
print(f"beta0 = {beta0:+.3f}   (true constant part {B0:+.1f})")
print(f"mean |error| = {np.abs(beta_hat - beta_true).mean():.3f}")

fig, axes = plt.subplots(1, 2, figsize=(10, 3.8))
axes[0].scatter(beta_true, beta_hat, s=6, alpha=0.35)
lims = [min(beta_true.min(), beta_hat.min()), max(beta_true.max(), beta_hat.max())]
axes[0].plot(lims, lims, "k--", lw=1)
axes[0].set_xlabel(r"true $\beta(x)$")
axes[0].set_ylabel(r"fitted $\hat\beta(x)$")
axes[0].set_title(f"recovery, corr = {corr:.3f}")

order = np.argsort(test["X2"].to_numpy())
axes[1].plot(test["X2"].to_numpy()[order], beta_true[order], "k-", lw=2, label="true")
axes[1].scatter(test["X2"], beta_hat, s=6, alpha=0.3, color="C0", label="fitted")
axes[1].set_xlabel("$X_2$")
axes[1].set_ylabel(r"$\beta$")
axes[1].set_title(r"$\beta$ against a modifier")
axes[1].legend()
fig.tight_layout()
plt.show()
```

```
corr(beta_hat, beta_true) = 0.996
beta0 = -0.960   (true constant part -1.0)
mean |error| = 0.098
```

```
/tmp/ipykernel_3800/2710869176.py:4: UserWarning: Converting a tensor with requires_grad=True to a s
Consider using tensor.detach() first. (Triggered internally at /__w/pytorch/pytorch/torch/csrc/auto
  beta0 = float(flow.nodes["Y"].shifts["T"].beta0)
```

The scatter against `X2` alone is a band rather than a line, because the true effect also depends on `X3` — exactly as it should be.

5. The read-out is an identity, not a summary

For a binary treatment, $\beta(x)$ *equals* the difference of the abducted latents between the two arms, with the outcome held fixed. That is a definition the implementation must satisfy, and it does, to float precision:

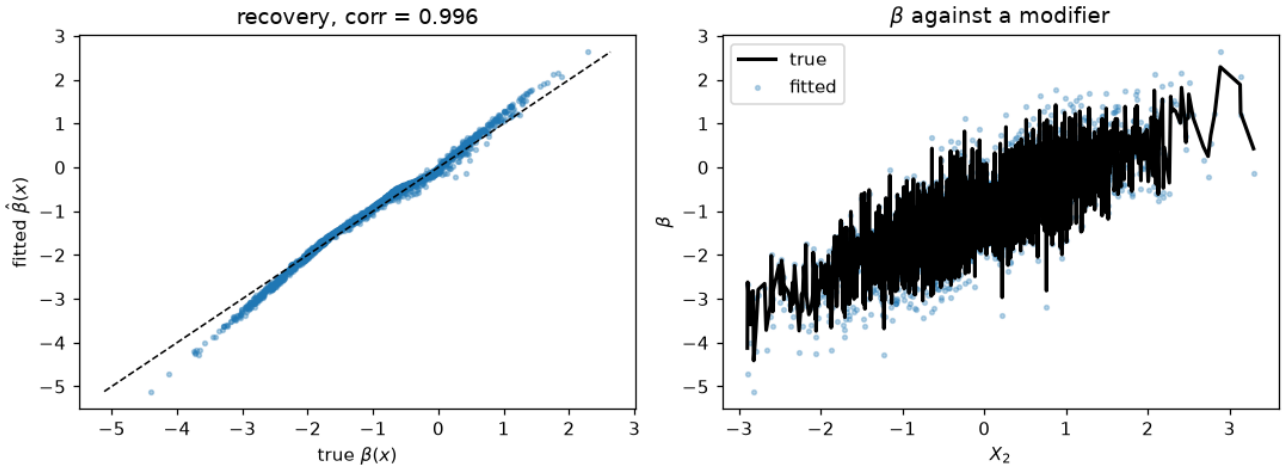


Figure 15: png

```
u1 = flow.abduct(test.assign(T=1.0), seed=0)["Y"].to_numpy()
u0 = flow.abduct(test.assign(T=0.0), seed=0)["Y"].to_numpy()
print(f"max |beta(x) - (u(T=1) - u(T=0))| = {np.abs(beta_hat - (u1 - u0)).max():.2e}")
```

```
max |beta(x) - (u(T=1) - u(T=0))| = 4.77e-07
```

6. Confounding: why `center=` exists

Everything above had a prognostic part flexible enough to absorb g. Real models are misspecified, and when the misfit correlates with **treatment assignment**, the effect head absorbs the confounding instead of the effect.

The configuration where this bites hardest is deliberately simple: one covariate, a strong propensity $e(x) = \sigma(2x)$, a constant true effect $\tau = -1$, and a quadratic prognostic part fitted with a linear term. `center="ps"` replaces t by $t - \hat{e}(x)$ using **cross-fitted** (out-of-fold) propensities from the training-frame column `ps`, so the head sees the part of the treatment that the covariates do not explain. Stage 1 — the propensities — is yours: any classifier, predicted out of fold, merged into the frame as a column. Here, five classical fits of the treatment spec.

```
TAU = -1.0


def confounded(n, seed):
    rng = np.random.default_rng(seed)
    x = rng.normal(size=n)
    t = (rng.logistic(size=n) > -2.0 * x).astype(float)  # strong confounding
    y = (rng.logistic(size=n) - 1.2 * x**2 - TAU * t) / 2.0  # quadratic prognostic
    return pd.DataFrame({"X": x, "T": t, "Y": y})


c_train, c_val, c_test = confounded(5400, 0), confounded(600, 7), confounded(3000, 1000)

# stage 1 of the centered design, the caller's job: cross-fitted P(T=1|X), each
# fold predicted by a treatment model that never saw it (the DML requirement)
fold_id = np.random.default_rng(0).permutation(len(c_train)) % 5
e_oof = np.empty(len(c_train))
for j in range(5):
    t_spec = {
```

```
        "X": ContinuousNode([I(transform="affine")]),
        "T": OrdinalNode(2, [LS("X")]),
    }
    proxy = CausalFlowDAG(t_spec, seed=0)
    proxy.fit_classical(c_train.ilocfold_id != j])
    e_oof[fold_id == j] = proxy.pmf(c_train.iloc[fold_id == j], "T")[:, 1]

for center in (False, "ps"):
    spec_c = {
        "X": ContinuousNode([I(transform="affine")]),
        "T": OrdinalNode(2, [LS("X")]),
        # linear prognostic term, though the truth is quadratic
        "Y": ContinuousNode([LS("X"), VC("X", center=center, t="T")]),
    }
    fc = CausalFlowDAG(spec_c, seed=0)
    fc.fit(
        # the out-of-fold propensities ride the frame as the column center= names
        c_train.assign(ps=e_oof) if center else c_train,
        epochs=250,
        learning_rate=1e-2,
        batch_size=512,
        validation_data=c_val,
        seed=0,
        callbacks=EarlyStopping(),  # keep the best-validation weights
    )
    b = fc.varying_coef(c_test, "Y")
    print(
        f"center={bool(center)!s:5s}  mean |beta - tau| = {np.abs(b - TAU).mean():.3f}"
        f"   mean beta = {b.mean():+.3f}  (true tau {TAU:+.1f})"
    )
```

```
center=False  mean |beta - tau| = 1.284   mean beta = -0.028  (true tau -1.0)


center=True   mean |beta - tau| = 0.308   mean beta = -0.692  (true tau -1.0)
```

Without centering the confounding swallows the effect almost entirely — the average estimate lands near zero when the truth is −1. With centering it recovers most of it, and the mean absolute error falls by roughly a factor of four. Centering costs nothing when the prognostic part is adequate, which is why it is worth reaching for whenever assignment is not random.

What to take away

you want	use	read it with
one interpretable effect	`LS("T")`	`ls_coefficients()`
an effect that varies with covariates	`VC(*modifiers, t="T")`	`varying_coef(df, node)`
the same under confounding + a misspecified prognostic part	`VC(..., center="ps")`	the same read-out
a shortlist of modifiers before you commit	`effect_modifier_scan` on a cheap all-`ls` fit	its `flag` column, read as screening

The alternative of writing `CS("T", "X2", "X3")` is equally *expressive* — any shift decomposes into arms — but nothing in the likelihood rewards a smooth difference between them, so the

implied effect is the difference of two unregularized networks. On this task class that reduced form measures corr ≈ 0.5 against the truth. The `VC` term exists to put the regularization where the question is. See `docs/varying-coefficients.md` for the semantics and `docs/scores.md` for the scan.